Querying with Qubits: A Quantum Annealing Approach to the Cardinality Estimation for Query Optimisation

Matea Qazolli

Master's Thesis in Computer Science
Faculty of Computer Science and Mathematics
Chair of Scalable Database Systems

Matriculation number 112754
Supervisor Prof. Stefanie Scherzinger

27th June 2026

Eigenständigkeitserklärung

Hiermit versichere ich, Matea Qazolli,

1. dass ich die vorliegende Arbeit selbstständig und ohne unzulässige Hilfe verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt, sowie die wörtlich und sinngemäß übernommenen Passagen aus anderen Werken kenntlich gemacht habe.

2. Außerdem erkläre ich, dass ich der Universität ein einfaches Nutzungsrecht zum Zwecke der Überprüfung mittels einer Plagiatssoftware in anonymisierter Form einräume.

Passau, 27. Juni 2026

Matea Qazolli

Abstract

Cardinality estimation is central to query optimisation because inaccurate estimates can lead a database optimiser to choose poor execution plans. Traditional database estimators are computationally lightweight, but often rely on assumptions such as attribute value independence. When query predicates involve correlated attributes, these assumptions can lead to substantial estimation errors. This motivates dependency-aware estimators that can represent relationships between attributes. Bayesian networks provide one way to model such relationships, and Chow–Liu trees are a tractable tree-structured form of Bayesian network used in this thesis as the classical baseline.

This thesis proposes AnnealBN-CE, an end-to-end framework that integrates QUBO-based Bayesian network structure learning with single-relation cardinality estimation. The framework converts a tabular relation into a finite solver representation, learns candidate structures using simulated annealing and simulated quantum annealing, and uses the resulting Bayesian networks to estimate selection-query cardinalities through probabilistic inference.

The framework is evaluated on controlled synthetic Bayesian-network data, real health-survey data, and synthetic binary transaction data. Its estimates are compared with PostgreSQL and the Chow–Liu baseline using query-level cardinalities and the q-error metric. The evaluation also studies robustness under different read–trial configurations by separating learned graph diversity from variation in the resulting cardinality estimates.

The results show that annealing-based Bayesian network structure learning can be integrated into an end-to-end cardinality-estimation workflow. In several settings, simulated annealing and simulated quantum annealing produced stable cardinality estimates even when multiple learned structures were obtained. However, the framework remains limited by scalability, its single-relation scope, and the absence of experiments on physical quantum-annealing hardware. Overall, this thesis presents AnnealBN-CE as an experimental blueprint for future correlation-aware cardinality estimators based on more expressive Bayesian network structures.

Note on LLMs

ChatGPT, using the GPT-5.5 Thinking model, was used as a writing aid during the preparation of this thesis. It was employed to improve clarity, grammar, style, and wording, and to obtain preliminary feedback on text structure. All suggestions were manually reviewed and approved or modified by the author. The author remains solely responsible for the final content, arguments, results, and conclusions of this thesis.

Contents

List of Figures

List of Tables

1. Introduction

A single inaccurate cardinality estimate can cause a database query optimiser to select an execution plan that is orders of magnitude slower than an alternative. Modern database management systems (DBMSs) are expected to answer SQL queries efficiently, even when the underlying data are large, complex, and highly correlated. Before a query is executed, the DBMS must decide how to execute it: which access methods to use, which join order to select, and which physical operators to apply. This decision is made by the query optimiser, which compares alternative query execution plans (QEPs) and selects the plan with the lowest estimated cost [18, 10, 25].

A central input to this decision process is the estimated size of the intermediate query results. If the optimiser severely underestimates or overestimates the number of tuples produced by a predicate or join, it may select a suboptimal execution plan. For example, an underestimated result size may lead the optimiser to choose an index-based nested-loop plan, although a scan-based or hash-join plan would have been more efficient. Therefore, reliable query optimisation depends on accurate cardinality estimation (CE) and selectivity estimation (SE) [18, 25].

Selectivity estimation and cardinality estimation are closely related. Selectivity estimation predicts the fraction of tuples satisfying a predicate, whereas cardinality estimation predicts the absolute number of result tuples [25]. Because cardinalities are typically calculated from selectivities and relation sizes, inaccuracies in selectivity estimates directly affect cardinality estimates and, consequently, query-plan selection.

Illustrative example. Accurate estimation is hard because real-world data often contain dependencies between attributes. Following the example of Markl et al. [20], let us consider a relation `Cars` with the attributes shown in Table 1.1. Suppose that the database statistics indicate 10 distinct values for `MAKE`, 100 distinct values for `MODEL`, and 10 distinct values for `COLOR`.

Assuming uniformity, the predicates `MAKE = Mazda`, `MODEL = CX-5`, and `COLOR = white` are assigned the following selectivities:

$$P(M) = 10^{-1}, \qquad P(D) = 10^{-2}, \qquad P(C) = 10^{-1}.$$

Table 1.1.: Attributes and assumed distinct-value counts in the `Cars` example.

Attribute	Description	Example value	Distinct values
`MAKE`	Manufacturer	`Mazda`	10
`MODEL`	Model name	`CX-5`	100
`COLOR`	Exterior colour	`white`	10

In this expression, M denotes `MAKE = Mazda`, D denotes `MODEL = CX-5`, and C denotes `COLOR = white`.

Now consider the following query, which retrieves all white Mazda CX-5 cars:

```
SELECT *
FROM Cars
WHERE MAKE = 'Mazda'
      AND MODEL = 'CX-5'
      AND COLOR = 'white';
```

Assuming attribute value independence, the optimiser estimates the combined selectivity by multiplying the individual selectivities:

$$\sigma_{\text{AVI}} = P(M)P(D)P(C) = 10^{-1} \cdot 10^{-2} \cdot 10^{-1} = 10^{-4}.$$

However, this estimate is inaccurate because `MAKE` and `MODEL` are not independent: the model `CX-5` is produced by `Mazda`. A more accurate estimate would explicitly model the statistical dependencies between these attributes, resulting in a joint selectivity of approximately

$$\sigma_{\text{dep}} \approx P(D)\,P(C) = 10^{-2} \cdot 10^{-1} = 10^{-3}.$$

Thus, the dependency-aware estimate is approximately 10^{-3}, compared with only 10^{-4} under the independence assumption. In other words, ignoring the dependency between `MAKE` and `MODEL` leads to a tenfold underestimation of the query selectivity. The example illustrates how ignoring attribute dependencies can lead to significant estimation errors and, consequently, inefficient query execution plans.

Existing approaches. The challenge of finding accurate cardinality estimates while maintaining low computational overhead has remained an open research problem in the database community since the 1970s [25]. Traditional cardinality estimation methods are commonly based on compact statistical synopses, such as histograms and most-common-value lists, together with simplifying assumptions such as uniformity and attribute value independence [25, 30]. These methods are computationally efficient and are, therefore, widely used in database

systems. However, they often fail to represent complex dependencies between attributes, especially when predicates involve correlated columns. More expressive multidimensional statistics can capture such dependencies, but they may introduce high construction and maintenance costs.

Recent research has therefore explored alternative approaches, including sampling-based and learning-based estimators. Sampling-based methods estimate query result sizes by executing queries on samples of the data, while learning-based methods use statistical or machine learning models to approximate the underlying data distribution. Although these approaches can improve estimation accuracy, they also introduce challenges related to storage overhead, inference latency, model retraining, and integration into real DBMSs [25, 18].

Bayesian networks for selectivity estimation. This thesis focuses on Bayesian networks (BNs) as a statistical modelling approach for selectivity estimation. BNs represent the joint distribution of database attributes as a directed acyclic graph (DAG), where each variable depends only on its parent variables [14]. This factorisation allows BNs to capture attribute dependencies while avoiding the full cost of representing the complete multidimensional distribution [14, 10]. As a result, BNs offer a promising balance between estimation accuracy and computational efficiency.

However, learning an optimal BN structure is an NP-complete problem [16]. Classical approaches often restrict the structure of the BN to reduce learning and inference costs. For example, Chow–Liu trees limit each node to at most one parent, which enables efficient learning but also limits the model's ability to represent more complex attribute dependencies [36]. This creates a trade-off: simple BN structures are efficient but may be insufficiently expressive, whereas richer BN structures may improve estimation accuracy but are more expensive to learn.

Research gap. Quantum annealing (QA) has recently been investigated as an alternative approach for Bayesian network structure learning (BNSL), where the goal is to search for high-scoring network structures under computational constraints [22, 36]. Existing work has mainly evaluated quantum-assisted BNSL in terms of network reconstruction quality [36]. However, it remains unclear whether Bayesian networks learned from QUBO-based annealing formulations can improve cardinality estimation accuracy in database query optimisation, where the learned structure must not only approximate the data distribution but also support inference for query predicates.

This thesis addresses this gap by adapting annealing-based Bayesian network structure learning methods to the cardinality estimation task and evaluating their accuracy trade-offs against classical and DBMS-based baselines. In particular, this thesis investigates whether richer BN structures learned using simulated annealing and simulated quantum annealing can improve estimation accuracy

compared with classical tree-based BN models and traditional DBMS estimators such as PostgreSQL [30].

Research questions. The central aim of this thesis is to investigate whether annealing-based Bayesian network structure learning can be integrated into a cardinality-estimation pipeline for single-relation selection queries, and whether the resulting learned structures provide empirical advantages over classical tree-structured Bayesian networks and PostgreSQL's built-in estimator. To evaluate this aim, the thesis addresses the following research questions:

Research Question 1: *To what extent can QUBO-based Bayesian network structure learning be integrated into an end-to-end cardinality-estimation workflow for single-relation selection queries?*

Research Question 2: *How do simulated annealing and simulated quantum annealing behave under varying read and trial budgets with respect to learned graph diversity, downstream cardinality-estimation stability, and query-level accuracy?*

Research Question 3: *Under which data and workload characteristics do annealing-learned Bayesian networks improve cardinality-estimation accuracy compared with PostgreSQL and the classical Chow–Liu Bayesian-network baseline?*

Contributions. The main contributions of this thesis are as follows:

- It proposes AnnealBN-CE, an end-to-end framework that integrates QUBO-based Bayesian network structure learning with query-level cardinality estimation for single-relation selection queries. The framework covers solver encoding, annealing-based structure learning, adjacency matrix decoding, Bayesian-network parameter fitting, probabilistic query evaluation, and cardinality-output generation.
- It adapts simulated annealing and simulated quantum annealing from the Bayesian network reconstruction setting to the downstream task of cardinality estimation. Instead of evaluating learned graphs only by structural similarity to a reference network, the thesis evaluates whether the learned structures produce useful query cardinality estimates.
- It reproduces and adapts a Chow–Liu Bayesian-network estimator as the classical tree-structured baseline. This establishes a dependency-aware but structurally restricted benchmark against which the more expressive annealing-learned Bayesian networks can be compared.
- It implements a reproducible evaluation workflow that compares PostgreSQL, the Chow–Liu baseline, simulated annealing, and simulated quantum annealing on the same tabular relations, query workloads, and q-error-based accuracy summaries.

- It analyses solver robustness by separating learned graph diversity from downstream cardinality-estimation variation. This distinction is important because different learned adjacency matrices do not necessarily lead to different query-level estimates.
- It identifies practical limitations and design requirements for future annealing-assisted cardinality estimators, including representation alignment, state-space control, solver-budget selection, and the extension from single-relation selection predicates to join-aware workloads.

Structure. The remainder of this thesis is organised as follows. Chapter 2 introduces the preliminaries required for the thesis, including selectivity and cardinality estimation, the q-error metric, traditional selectivity estimation, Bayesian networks, parameter learning and inference, Chow–Liu trees, Bayesian network structure learning, QUBO formulations, and annealing-based optimisation. Chapter 3 reviews related work on cardinality estimation, Bayesian-network-based estimators, and annealing-based Bayesian network structure learning. Chapter 4 presents AnnealBN-CE, the proposed framework for cardinality estimation with annealing-learned Bayesian networks. Chapter 5 introduces the classical Chow–Liu tree estimator used as the Bayesian-network baseline for comparison. Chapter 6 presents the experimentation and evaluation, including the implementation notes, execution platform, datasets, query workloads, annealing run configurations, evaluation procedure, robustness checks, and empirical results. Chapter 7 summarises the main findings, answers the research questions, discusses limitations, and outlines future work.

2. Preliminaries

This chapter introduces the technical concepts required for the methodology and evaluation of this thesis. It first defines selectivity and cardinality estimation in the single-relation setting and explains why independence assumptions can lead to estimation errors. It then introduces Bayesian networks as compact probabilistic models for representing dependencies between database attributes. The chapter further covers parameter learning, inference, Chow–Liu tree models, score-based Bayesian network structure learning, and the QUBO formulation used to express structure learning as an energy-minimisation problem. Finally, it introduces the annealing-based solvers considered in this thesis and the q-error metric used for evaluating cardinality estimation accuracy.

2.1. SELECTIVITY AND CARDINALITY ESTIMATION

Query optimisers use estimates of intermediate result sizes to compare alternative execution plans. This thesis focuses on single-relation, no-join selection queries. In this setting, the goal is to estimate the fraction of tuples in one relation that satisfy a given set of predicates [10, 9]. Join selectivity estimation is outside the scope of this work.

Let $\mathcal{R}(A_1, A_2, \ldots, A_n)$ be a relation with $N = |\mathcal{R}|$ tuples. A selection query q applies predicates to a subset of attributes. For equality predicates $A_1 = a_1, \ldots, A_k = a_k$, where $k \leq n$, the selectivity of q, denoted by $\sigma(q)$, is the fraction of tuples in $\mathcal{R}$ that satisfy all predicates. From a probabilistic point of view, selectivity can be interpreted as the joint probability of the queried attribute values [9, 10]:

$$\sigma(q) = P(A_1 = a_1, \ldots, A_k = a_k). \tag{2.1}$$

The corresponding cardinality is the number of tuples satisfying the query predicates. If the true selectivity is known, the true cardinality is

$$C(q) = \sigma(q) \cdot N. \tag{2.2}$$

In practice, the optimiser works with an estimated selectivity $\widehat{\sigma}(q)$. The estimated cardinality is therefore

$$\widehat{C}(q) = \widehat{\sigma}(q) \cdot N. \tag{2.3}$$

Thus, selectivity errors directly affect cardinality estimates and may lead the optimiser to choose suboptimal execution plans [10, 25].

A common simplification in traditional cardinality estimation is the *Attribute Value Independence* (AVI) assumption. AVI assumes that predicates over different attributes are statistically independent. Under this independence assumption, the joint selectivity of multiple predicates is estimated as the product of their marginal selectivities:

$$S(p_1 \wedge p_2 \wedge \cdots \wedge p_n) \approx \prod_{i=1}^{n} S(p_i) \tag{2.4}$$

$$P(A_1 = a_1, \ldots, A_k = a_k) \approx \prod_{i=1}^{k} P(A_i = a_i). \tag{2.5}$$

This approximation is computationally efficient, but it can be inaccurate when attributes are correlated. Such correlations are common in real-world data and are the type of dependency that Bayesian networks aim to represent [10, 9].

Example: Limitation of AVI in the WetGrass Network

To illustrate the limitation of AVI, consider a simplified fragment of the standard *WetGrass* Bayesian network with three variables: *Cloud*, *Sprinkler*, and *Rain* [5]. Let C denote *Cloud*, S denote *Sprinkler*, and R denote *Rain*. The variable C is a common parent of S and R, which makes S and R dependent through their shared parent. Here, t denotes the true state of a Boolean variable and f denotes the false state. The probability values used in the example are shown in Table 2.1.

Table 2.1.: Conditional probability distributions in the simplified WetGrass dependency example.

Quantity	**Probability of t**	**Probability of f**
$P(C)$	0.3	0.7
$P(S \mid C = t)$	0.2	0.8
$P(S \mid C = f)$	0.8	0.2
$P(R \mid C = t)$	0.6	0.4
$P(R \mid C = f)$	0.3	0.7

Assume that an optimiser needs to estimate the selectivity of the event $S = t \wedge R = t$. First, the marginal probability of $S = t$ is obtained by summing over the possible states of C:

$$\begin{aligned} P(S = t) &= P(C = t)P(S = t \mid C = t) + P(C = f)P(S = t \mid C = f) \\ &= (0.3 \cdot 0.2) + (0.7 \cdot 0.8) = 0.62. \end{aligned}$$

Similarly, the marginal probability of $R = t$ is

$$\begin{aligned} P(R = t) &= P(C = t)P(R = t \mid C = t) + P(C = f)P(R = t \mid C = f) \\ &= (0.3 \cdot 0.6) + (0.7 \cdot 0.3) = 0.39. \end{aligned}$$

Under the AVI assumption, the joint selectivity is estimated by multiplying these two marginal probabilities:

$$\begin{aligned} \sigma_{\text{AVI}} &= P(S = t)P(R = t) \\ &= 0.62 \cdot 0.39 = 0.2418. \end{aligned}$$

However, this estimate ignores the fact that S and R are dependent through the common parent C. A dependency-aware estimate marginalises over the possible states of C:

$$\begin{aligned} P(S = t, R = t) &= \sum_{c \in \{t,f\}} P(C = c)P(S = t \mid C = c)P(R = t \mid C = c) \\ &= (0.3 \cdot 0.2 \cdot 0.6) + (0.7 \cdot 0.8 \cdot 0.3) \\ &= 0.036 + 0.168 = 0.2040. \end{aligned}$$

Thus, the AVI-based estimate is larger than the dependency-aware estimate. Relative to the dependency-aware value, the overestimation is approximately

$$\frac{0.2418 - 0.2040}{0.2040} \approx 18.5\%.$$

This example demonstrates how independence assumptions can lead to estimation errors when attributes are correlated.

Since query optimisers rely on such estimates to choose execution plans, modelling attribute dependencies more explicitly can improve the reliability of selectivity and cardinality estimation [10, 9]. In this thesis, Bayesian networks are used because they provide a compact probabilistic representation of dependencies between attributes [10, 14].

2.2. EVALUATION METRIC: Q-ERROR

To evaluate estimation accuracy, this thesis uses the *q-error* metric, a multiplicative error metric commonly used in cardinality estimation [11, 10]. It is suitable for this setting because query result sizes can differ by several orders of magnitude. Absolute error metrics, such as mean squared error, can therefore be dominated by queries with large result sizes, while q-error compares estimates by their multiplicative deviation from the true value.

For a true value y and an estimated value $\widehat{y}$, the q-error is defined as

$$q(y, \widehat{y}) = \frac{\max(y, \widehat{y})}{\min(y, \widehat{y})}. \tag{2.6}$$

The standard definition assumes positive values of y and $\widehat{y}$. A q-error of 1 indicates an exact estimate. Larger values indicate larger multiplicative errors. For example, a q-error of 2 means that the estimate differs from the true value by a factor of 2.

Table 2.2.: Example q-errors for a true value of $y = 100$.

True value y	**Estimate** $\widehat{y}$	**q-error**
100	50	2
100	100	1
100	200	2

As shown in Table 2.2, q-error is symmetric with respect to underestimation and overestimation. It also allows estimates from queries with different result sizes or selectivities to be compared on the same multiplicative scale [10]. In this thesis, q-error is used as the main accuracy metric for evaluating PostgreSQL and the Chow–Liu tree Bayesian-network baseline. For annealing-learned Bayesian networks, the same q-error metric is used, but its application requires an additional aggregation procedure, which is described in Section 6.6.

2.3. TRADITIONAL SELECTIVITY ESTIMATION IN DATABASE SYSTEMS

Traditional database systems estimate selectivities using compact statistical summaries of the underlying data. These summaries are collected during statistics generation and are later used by the query optimiser to estimate predicate selectivities. The goal is to obtain estimates that are accurate for plan selection while keeping storage, construction, and maintenance costs low [18, 25].

A common approach is to maintain statistics separately for each attribute. These per-column statistics typically include distinct-value estimates, null-value

fractions, most-common-value lists, and histograms. Most-common-value lists store the frequencies of frequently occurring values, while histograms approximate the distribution of the remaining values by partitioning the attribute domain into buckets. Such statistics are effective for estimating simple predicates over individual attributes, such as equality and range predicates [25, 29].

PostgreSQL, which is used in this thesis as the traditional DBMS baseline, follows this general approach through planner statistics collected by `ANALYZE`. Its regular statistics include per-column information such as null fractions, distinct-value estimates, most-common-value lists, and histogram bounds [29]. PostgreSQL also supports extended statistics for selected attribute groups, including multivariate most-common-value statistics, dependency statistics, and distinct-value statistics [29]. However, these extended statistics must be explicitly defined for selected column groups. They therefore do not remove the general challenge of modelling dependencies between arbitrary combinations of attributes.

The main limitation of traditional per-column statistics is that they describe marginal distributions, such as $P(A = a)$, but do not directly represent joint distributions, such as $P(A_1 = a_1, A_2 = a_2)$. When a query contains predicates over multiple attributes, the optimiser must combine several single-column estimates. This is often done using simplifying assumptions such as uniformity or Attribute Value Independence (AVI) [10, 25]. As discussed in Section 2.1, such assumptions can lead to inaccurate estimates when attributes are correlated.

More expressive statistics, such as multidimensional histograms or multivariate most-common-value statistics, can represent dependencies between selected attribute groups. However, they also introduce additional construction, storage, and maintenance costs [25, 29]. This motivates the use of Bayesian networks, which model dependencies between attributes through a compact factorised representation of the joint distribution [10, 9, 14].

2.4. BAYESIAN NETWORKS

A *Bayesian network* (BN) is a probabilistic graphical model that compactly represents a joint probability distribution over a set of random variables $X = \{X_1, X_2, \ldots, X_n\}$ [14]. In this thesis, these random variables correspond to attributes of a database relation. Given a relation $\mathcal{R}(A_1, A_2, \ldots, A_n)$, each attribute can be treated as a random variable whose states correspond to observed values or discretised value ranges in the relation [10].

A Bayesian network is defined by two components: a directed acyclic graph (DAG) and a set of local conditional probability distributions [14]. The DAG defines the dependency structure of the model. Each node represents a random variable X_i, and a directed edge $X_j \to X_i$ indicates that X_j is a parent of X_i.

The set of parents of X_i is denoted by $\mathrm{Pa}(X_i)$. The graph must be acyclic, meaning that it contains no directed cycles. In this thesis, edges are interpreted as probabilistic dependencies used to factorise the joint distribution, rather than as necessarily causal relationships.

The local conditional probability distributions define the quantitative part of the model. For each variable X_i, the network stores a distribution $P(X_i \mid \mathrm{Pa}(X_i))$. If X_i has no parents, this reduces to the marginal distribution $P(X_i)$. For discrete variables, these local distributions are commonly represented as conditional probability tables (CPTs). A CPT contains probabilities of the form $P(X_i = x \mid \mathrm{Pa}(X_i) = \pi)$, where x is a possible state of X_i, and π is a joint configuration of its parent variables [14].

The main advantage of Bayesian networks is that they factorise the full joint distribution into local conditional distributions [14]:

$$P(X_1, X_2, \ldots, X_n) = \prod_{i=1}^{n} P(X_i \mid \mathrm{Pa}(X_i)). \tag{2.7}$$

This factorisation follows from the conditional independence assumptions encoded by the DAG. Instead of storing the full joint distribution explicitly, the model stores smaller local distributions. This is particularly useful for database applications, where relations may contain many attributes and the full joint distribution can become infeasible to represent directly [10].

Running Example: WetGrass Bayesian Network

The standard *WetGrass* network, taken from the data integration literature, is used as a running example throughout this thesis [5]. It consists of four binary variables: *Cloud* (C), *Sprinkler* (S), *Rain* (R), and *WetGrass* (W). In this network, C is a parent of both S and R, while S and R are parents of W. The structure is summarised in Table 2.3.

Table 2.3.: Structure of the WetGrass Bayesian network used as a running example.

Variable	Abbreviation	Parents
Cloud	C	$\emptyset$
Sprinkler	S	$\{C\}$
Rain	R	$\{C\}$
WetGrass	W	$\{S, R\}$

Using the Bayesian network factorisation from Eq. (2.7), the joint distribution of the WetGrass network factorises as

$$P(C, S, R, W) = P(C)\, P(S \mid C)\, P(R \mid C)\, P(W \mid S, R). \tag{2.8}$$

This factorisation is useful for two reasons. First, it shows how a Bayesian network represents the full joint distribution through smaller local conditional distributions. A full joint distribution over four binary variables requires $2^4 = 16$ probability entries, corresponding to 15 independent parameters because the entries must sum to one. The factorised representation in Eq. (2.8) requires only 9 independent parameters: one for $P(C)$, two for $P(S \mid C)$, two for $P(R \mid C)$, and four for $P(W \mid S, R)$.

Second, the example highlights a structural pattern that is important for this thesis. The variable W has two parents, S and R. A tree-structured Chow–Liu model can assign at most one parent to each non-root node and therefore cannot represent both incoming dependencies of W at the same time. This makes WetGrass a useful example for comparing tree-structured Bayesian networks with more general structures learned through annealing-based Bayesian network structure learning.

For selectivity estimation, Bayesian networks are useful because they provide a compact approximation of the joint distribution over database attributes. This allows predicate selectivities to be estimated while accounting for dependencies between attributes [10].

2.5. PARAMETER LEARNING AND DISTRIBUTION REPRESENTATION

After the structure of a Bayesian network has been specified or learned, the next step is to estimate its parameters. These parameters are the entries of the conditional probability tables (CPTs), which quantify the local distributions $P(X_i \mid \mathrm{Pa}(X_i))$ associated with each node [14]. Together with the Bayesian network structure, these parameters define the factorised joint distribution introduced in Eq. (2.7).

The CPT parameters are described using the following notation. Let r_i denote the number of possible states of variable X_i, and let q_i denote the number of possible joint configurations of its parent set $\mathrm{Pa}(X_i)$. If the parents are discrete variables, then

$$q_i = \prod_{X_j \in \mathrm{Pa}(X_i)} r_j,$$

where r_j is the number of states of parent variable X_j. If X_i has no parents, then $q_i = 1$. Let x_{ik} denote the k-th state of X_i, and let π_{ij} denote the j-th parent configuration. A CPT parameter is then defined as

$$\theta_{ijk} = P(X_i = x_{ik} \mid \mathrm{Pa}(X_i) = \pi_{ij}).$$

Parameter learning thus consists of estimating the values of θ_{ijk} directly from the dataset.

In this thesis, the datasets are treated as complete after preprocessing, meaning that each tuple contains an observed value for every variable used in the network. Let D be a dataset with N tuples, where each tuple represents one complete assignment of values to the variables $X_1, \ldots, X_n$. For complete discrete data, maximum likelihood estimation (MLE) reduces to counting value frequencies in the dataset [14].

For a root node X_i, which has no parents, the CPT reduces to a marginal distribution. The maximum likelihood estimate for state x_{ik} is

$$\widehat{P}(X_i = x_{ik}) = \frac{N(X_i = x_{ik})}{N},$$

where $N(X_i = x_{ik})$ denotes the number of tuples in which X_i takes state x_{ik}, and N is the total number of tuples in the dataset.

For a non-root node X_i, the probability of X_i is conditioned on the values of its parents. For parent configuration π_{ij}, the maximum likelihood estimate is

$$\widehat{\theta}_{ijk} = \widehat{P}(X_i = x_{ik} \mid \mathrm{Pa}(X_i) = \pi_{ij}) = \frac{N(X_i = x_{ik}, \mathrm{Pa}(X_i) = \pi_{ij})}{N(\mathrm{Pa}(X_i) = \pi_{ij})}. \quad (2.9)$$

Here, $N(X_i = x_{ik}, \mathrm{Pa}(X_i) = \pi_{ij})$ is the number of tuples in which X_i has state x_{ik} and its parents have configuration π_{ij}. The denominator $N(\mathrm{Pa}(X_i) = \pi_{ij})$ is the number of tuples with that parent configuration [14]. If a parent configuration is not observed in the data, the corresponding frequency estimate is not directly defined. This issue is relevant for sparse data and high-cardinality variables and is discussed further in Section 2.5.2.

This formulation shows that, for complete discrete data, parameter learning in a Bayesian network reduces to a frequency-counting problem. Because of the factorised structure of the network, the CPT of each variable can be estimated locally using only the variable and its parent set [14]. In relational settings, the required counts can be obtained naturally through grouping and aggregation operations, such as `GROUP BY` and `COUNT` [9].

The size of a CPT depends on both the number of states of the variable and the number of configurations of its parent set. If X_i has r_i possible states and its parents have q_i possible joint configurations, then the corresponding CPT contains $q_i r_i$ probability entries. Of these, $q_i(r_i - 1)$ are independent because the probabilities for each parent configuration must sum to one:

$$\mathrm{entries}(X_i) = q_i r_i, \qquad \mathrm{free}(X_i) = q_i(r_i - 1). \quad (2.10)$$

Since $q_i = \prod_{X_j \in \mathrm{Pa}(X_i)} r_j$, the CPT size grows multiplicatively with the cardinalities of the parent variables. If each variable has at most r states and X_i has p parents, then the CPT contains at most r^{p+1} entries. Thus, CPT size grows exponentially in the number of parents [14]. This motivates the use of restricted structures, such as Chow–Liu trees, as well as compact distribution representations and state-space reductions [9].

2.5.1. COMPACT REPRESENTATION WITH END-BIASED HISTOGRAMS

A practical challenge in Bayesian network-based selectivity estimation is the storage cost of conditional probability tables, especially when attributes have many distinct values or when continuous attributes are discretised into many states. As discussed in Section 2.5, the size of a CPT grows with both the number of states of the variable and the number of configurations of its parent set. High-cardinality attributes can therefore make exact CPT storage expensive [9].

One compact representation is the use of *end-biased histograms*. The idea is to preserve exact probabilities for frequent values while approximating less frequent values using histogram intervals [9]. Let $M = \{v_1, \ldots, v_K\}$ denote the K most common values of an attribute. The probabilities of these values are stored exactly. The remaining values are grouped into L intervals, typically using equi-height partitioning so that each interval contains approximately the same probability mass.

The remaining probability mass after storing the most common values is

$$R = 1 - \sum_{t=1}^{K} P(v_t).$$

Let B_ℓ denote one of the histogram intervals, and let $P(B_\ell)$ be the probability mass assigned to that interval. Under a uniformity assumption within the interval, the probability of a value $v \in B_\ell$ is approximated as

$$P(v) \approx \frac{P(B_\ell)}{|B_\ell|}, \qquad v \in B_\ell,$$

where $|B_\ell|$ is the number of distinct values contained in interval B_ℓ. If the remaining mass is split into L equi-height intervals, the probability mass allocated to each individual interval is approximated as

$$P(\text{int}_i) \approx \frac{M_{\text{rem}}}{L} \tag{2.11}$$

$$P(B_\ell) \approx \frac{R}{L} = \frac{1 - \sum_{t=1}^{K} P(v_t)}{L}.$$

Thus, frequent values are represented exactly, while less frequent values are approximated through intervals [9].

This representation reduces the effective number of stored states for an attribute from its raw cardinality to approximately $K + L$, where K values are stored explicitly and L intervals summarise the remaining values. If each variable is

represented using $K + L$ effective states, then a CPT for a node with p parents requires storage proportional to

$$(K + L)^{p+1}.$$

For an oriented Chow–Liu tree, the root node has no parent and each non-root node has exactly one parent. In that case, the approximate storage requirement for a network with n variables becomes

$$(K + L) + (n - 1)(K + L)^2,$$

where the first term corresponds to the root marginal distribution and the second term corresponds to the conditional distributions of the non-root nodes [9].

End-biased histograms therefore provide a compact approximation of probability distributions while preserving frequent values exactly. For selectivity estimation, this is useful because frequent values often have a strong effect on query selectivities, while less frequent values can be approximated more coarsely. The same principle also supports continuous attributes, which can be discretised into intervals before being represented in the histogram [9].

2.5.2. UNSEEN VALUES AND ZERO PROBABILITIES

A separate limitation of maximum likelihood estimation is that it can assign zero probability to states or parent-state configurations that are not observed in the data. This follows directly from the frequency-counting estimate in Eq. (2.9). In particular, if

$$N(X_i = x_{ik}, \mathrm{Pa}(X_i) = \pi_{ij}) = 0 \quad \text{and} \quad N(\mathrm{Pa}(X_i) = \pi_{ij}) > 0,$$

then the corresponding maximum likelihood estimate is $\widehat{\theta}_{ijk} = 0$.

This can negatively affect inference. If a query requires a state combination that was not observed in the training data, the estimated probability of that combination may become zero, even if the combination is possible in the underlying distribution. The problem becomes more likely when the data are sparse or when variables have many possible states.

A related issue occurs when a parent configuration is never observed at all. If $N(\mathrm{Pa}(X_i) = \pi_{ij}) = 0$, the conditional probability $P(X_i = x_{ik} \mid \mathrm{Pa}(X_i) = \pi_{ij})$ cannot be estimated directly from frequency counts because the denominator in Eq. (2.9) is zero.

Smoothing techniques, such as Laplace smoothing or Bayesian parameter estimation, can reduce this problem by assigning non-zero probability mass to unseen events. However, smoothing is not the focus of this thesis [14]. The issue is nevertheless relevant because the datasets used in this work contain discrete and discretised variables, and sparse state combinations can occur when the number of states or parent configurations increases.

2.6. INFERENCE FOR SELECTIVITY ESTIMATION

After the structure and parameters of a Bayesian network have been learned, the model can be used to estimate query selectivities. A selection query over a relation can be represented as a partial assignment of values to a subset of the variables in the network. For example, a query with predicates $A_1 = a_1, \ldots, A_k = a_k$ requires estimating the probability that all these predicate conditions hold [9].

Assume that the queried attributes are $A_1, \ldots, A_k$, and that the remaining attributes in the network are $A_{k+1}, \ldots, A_n$. The Bayesian network estimate of the query selectivity is obtained by marginalising over the non-query variables [9]:

$$P(A_1 = a_1, \ldots, A_k = a_k) = \sum_{a_{k+1}} \cdots \sum_{a_n} \prod_{j=1}^{n} P(A_j = a_j \mid \mathrm{Pa}(A_j) = pa_j). \quad (2.12)$$

Here, $a_{k+1}, \ldots, a_n$ range over all possible states of the variables that do not appear in the query. The term pa_j denotes the corresponding assignment of the parent variables of A_j. The queried variables $A_1, \ldots, A_k$ are fixed to their predicate values, while all remaining variables are summed out.

A standard exact method for computing such marginal probabilities is *variable elimination* (VE). VE eliminates non-query variables one at a time by combining the factors in which a variable appears and summing over its possible states [14, 9]. In general Bayesian networks, exact inference can be computationally expensive. The complexity of VE depends on the induced width w of the chosen elimination order. With bounded variable cardinalities, the cost is linear in the number of variables but exponential in w [14].

For tree-structured Bayesian networks, inference is substantially more efficient. In a tree, the induced width is 1, so VE can be performed in time linear in the number of variables, assuming bounded variable cardinalities. This makes tree-based models, such as Chow–Liu trees, attractive for selectivity estimation because they support exact inference while keeping computational cost low [9].

Inference in tree-structured networks can be further optimised by pruning parts of the tree that are irrelevant to a given query. The relevant subgraph is the minimal subtree containing the queried variables and the nodes on the paths needed to connect them. This subtree is often referred to as a *Steiner tree*. Nodes outside this subtree do not affect the queried marginal probability and can therefore be excluded before running inference [9].

Figure 2.1 illustrates this idea. If inference requires information involving nodes G, N, and H, the blue subtree contains the nodes and edges required for the computation, while irrelevant branches, such as S and P, can be pruned [9].

Overall, inference is the step that turns a learned Bayesian network into a selectivity estimator. Once the probability of the query predicates has been

computed using Eq. (2.12), it directly provides the estimated selectivity. The corresponding estimated cardinality is then obtained using Eq. (2.3).

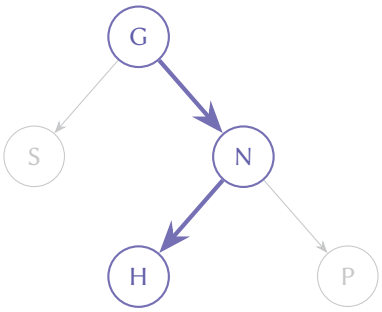

Figure 2.1.: Example of a Steiner tree, highlighted in blue, containing only the nodes and edges required for inference over G, N, and H. Illustration based on the pruning idea described by Halford et al. [9].

2.7. CHOW–LIU TREE BAYESIAN NETWORKS

The Chow–Liu algorithm is a classical method for learning a tree-structured Bayesian network from data [14]. Instead of searching over all possible directed acyclic graphs, it restricts the structure to a tree. After a root has been chosen and the edges have been oriented, the root node has no parent and every other node has exactly one parent. This restriction keeps structure learning efficient while still representing pairwise dependencies between attributes [14, 9].

The algorithm is based on *mutual information* (MI), which measures the statistical dependency between two variables [14]. For two discrete variables X and Y, mutual information is defined as

$$MI(X,Y) = \sum_{x,y} P(x,y) \log_2 \frac{P(x,y)}{P(x)P(y)}. \tag{2.13}$$

Here, $P(x,y)$ is the joint probability of $X = x$ and $Y = y$, while $P(x)$ and $P(y)$ are the corresponding marginal probabilities. In practice, these probabilities are estimated from the dataset using frequency counts. If X and Y are independent, then $MI(X,Y) = 0$. Larger values the stronger the statistical dependency.

To learn the structure, $MI(X_i, X_j)$ is computed for every pair of variables. These values are used as edge weights in a complete undirected graph over the variables. A maximum-weight spanning tree is then extracted, for example using Kruskal's or Prim's algorithm. Finally, a root is selected and the edges are oriented away from the root, producing a directed tree-structured Bayesian

network [14]. The parameters of the resulting network can then be estimated using the frequency-counting procedure described in Section 2.5.

The main computational cost is the pairwise mutual information computation. With n variables and N tuples, computing mutual information for all pairs using straightforward frequency counting requires $O(n^2 N)$ time. The spanning-tree step is polynomial in n and is usually not the dominant cost. This makes Chow–Liu learning much more tractable than learning an unrestricted Bayesian network structure [14, 9].

Chow–Liu trees are theoretically justified because they provide the best tree-structured approximation to the empirical distribution in terms of Kullback–Leibler divergence. Equivalently, among tree-structured models, the Chow–Liu tree maximises the total pairwise mutual information [14].

For selectivity estimation, Chow–Liu trees provide a compact and efficient Bayesian network baseline [9]. Since each non-root node has only one parent, the conditional probability tables remain small compared with more general Bayesian networks. Inference is also efficient because tree-structured networks have treewidth 1. Exact marginal probabilities can therefore be computed in linear time under bounded variable cardinalities, as discussed in Section 2.6 [9].

The main limitation is expressiveness. A Chow–Liu tree can capture pairwise dependencies, but it cannot fully represent cases where one variable depends jointly on multiple other variables. For example, in the WetGrass network introduced in Table 2.3, *WetGrass* has both *Sprinkler* and *Rain* as parents. A Chow–Liu tree can assign only one parent to each non-root node, so it cannot represent this joint dependency directly. This motivates the use of more general Bayesian network structure learning methods, which are discussed in the following sections.

2.8. BAYESIAN NETWORK STRUCTURE LEARNING

The Chow–Liu algorithm restricts Bayesian networks to tree structures. This makes learning efficient, but it also limits the dependencies that can be represented. More general Bayesian networks allow nodes to have multiple parents and can therefore represent richer dependency structures. Learning such a structure from data is known as *Bayesian network structure learning* (BNSL) [14, 16].

In BNSL, the goal is to learn a directed acyclic graph (DAG) G over variables $X_1, \ldots, X_n$ from a dataset D. In the context of this thesis, the variables correspond to attributes of a database relation. Once a structure has been learned, its parameters can be estimated using the frequency-counting procedure described in Section 2.5 [14].

Several approaches exist for BNSL, including constraint-based, score-based, and hybrid methods [16]. This thesis focuses on *score-based* structure learning.

In this setting, each candidate graph is evaluated by a scoring function, and the objective is to find the DAG with the best score [14]:

$$G^* = \arg \max_{G \in \mathcal{G}_{\mathrm{DAG}}} S(G; D), \tag{2.14}$$

where $\mathcal{G}_{\mathrm{DAG}}$ denotes the set of all directed acyclic graphs over the variables, and $S(G; D)$ is the score of structure G given dataset D.

The scoring function is central to score-based BNSL because it defines what it means for one candidate structure to be better than another. A useful score should reward structures that fit the observed data well, while penalising unnecessary complexity to avoid overfitting [14].

A useful property of many common Bayesian network scores is *decomposability*. A decomposable score can be written as a sum of local scores, one for each variable and its parent set, so that $S(G; D) = \sum_{i=1}^{n} s_i(\mathrm{Pa}_G(X_i); D)$ [14]. Here, $\mathrm{Pa}_G(X_i)$ denotes the parent set of X_i in graph G, and $s_i(\mathrm{Pa}_G(X_i); D)$ is the local score associated with variable X_i and its parents. Decomposability is important because it allows the quality of a candidate structure to be evaluated through local parent-set choices.

Learning an optimal Bayesian network structure is computationally difficult because the number of possible DAGs grows super-exponentially with the number of variables. Exhaustive enumeration is therefore infeasible except for very small networks [14, 16]. In practice, BNSL is often approached using structural restrictions, such as Chow–Liu trees, heuristic search methods, such as greedy search, or alternative optimisation formulations [14, 36].

The following subsections introduce two common decomposable scoring functions used in Bayesian network structure learning: the Bayesian Dirichlet score and the Bayesian Information Criterion [14].

2.8.1. BAYESIAN DIRICHLET SCORE

The Bayesian Dirichlet (BD) score is a Bayesian scoring function for discrete Bayesian networks. It evaluates a structure by computing the marginal likelihood of the data given the structure. In contrast to a score based only on one parameter estimate, the BD score integrates over the model parameters [14]. In this thesis, the annealing-based BNSL pipeline uses a BDeu-style local score as part of the QUBO construction, following the score-based formulation used for quantum-assisted Bayesian network reconstruction [36].

Assume complete discrete data. For each variable X_i, let r_i denote the number of possible states of X_i, and let q_i denote the number of possible configurations of its parent set $\mathrm{Pa}_G(X_i)$. Let N_{ijk} be the number of tuples in D where X_i is in its k-th state and its parents are in their j-th configuration. The corresponding Dirichlet hyperparameter is denoted by α_{ijk}. Furthermore, let $N_{ij} = \sum_{k=1}^{r_i} N_{ijk}$ and $\alpha_{ij} = \sum_{k=1}^{r_i} \alpha_{ijk}$.

Under the standard multinomial-Dirichlet assumptions, the BD marginal likelihood is given by

$$P(D \mid G) = \prod_{i=1}^{n} \prod_{j=1}^{q_i} \frac{\Gamma(\alpha_{ij})}{\Gamma(\alpha_{ij} + N_{ij})} \prod_{k=1}^{r_i} \frac{\Gamma(\alpha_{ijk} + N_{ijk})}{\Gamma(\alpha_{ijk})}. \tag{2.15}$$

Here, $\Gamma(\cdot)$ denotes the Gamma function. The BD score is decomposable because the marginal likelihood factorises over variables and parent configurations. In practice, the logarithm of this expression is usually used for numerical stability and to obtain additive local scores [14, 36].

The hyperparameters α_{ijk} encode prior assumptions about the conditional distributions. When these hyperparameters are chosen uniformly according to an equivalent sample size, the score corresponds to the Bayesian Dirichlet equivalent uniform (BDeu) score [14]. This BDeu-style score is important for this thesis because it provides the scoring component used in the annealing-based BNSL formulation.

2.8.2. BAYESIAN INFORMATION CRITERION

Another important score for Bayesian network structure learning is the Bayesian Information Criterion (BIC). Unlike the BD score, which integrates over parameters, BIC uses the maximum likelihood estimate of the parameters and penalises the number of free parameters in the model [14]. It is relevant in this thesis because it is a common score in score-based BNSL and is also used in related BNSL implementations considered during this work.

For complete discrete data, the BIC score can be written as

$$BIC(G; D) = \sum_{i=1}^{n} \sum_{j=1}^{q_i} \sum_{k=1}^{r_i} N_{ijk} \log_2 \frac{N_{ijk}}{N_{ij}} - \frac{\log_2 N}{2} \sum_{i=1}^{n} q_i (r_i - 1). \tag{2.16}$$

The first term is the log-likelihood of the data under the maximum likelihood parameter estimates. It rewards structures that fit the observed data well. The second term is a complexity penalty. It depends on the dataset size N and on the number of independent parameters in the network. For variable X_i, each parent configuration contributes $r_i - 1$ independent parameters, so the total number of free parameters is $\sum_{i=1}^{n} q_i(r_i - 1)$.

The BIC score therefore balances fit and complexity. Adding more parents to a node may improve the likelihood term, but it also increases q_i. This increases the number of parameters and strengthens the penalty. As a result, BIC discourages overly dense structures and helps reduce overfitting [14].

As with the BD score, BIC is decomposable. The contribution of each variable depends only on that variable and its parent set. This makes BIC suitable for score-based BNSL methods that evaluate alternative parent sets locally [14].

2.8.3. SEARCH PROCEDURES

Given a scoring function such as BD or BIC, structure learning becomes an optimisation problem over candidate DAGs. Because exhaustive search is infeasible except for very small networks, practical methods usually rely on heuristic or restricted search procedures [14, 16].

A common classical approach is greedy search. Starting from an initial structure, such as an empty graph, the algorithm repeatedly applies local modifications, including adding, deleting, or reversing edges, as long as the resulting graph remains acyclic. At each step, the modification that gives the largest improvement in the score is selected. The procedure stops when no local modification improves the score [14].

Greedy search is often efficient in practice; however, it does not guarantee finding the globally optimal structure because it can become trapped in local optima [14, 16]. This limitation motivates alternative formulations of BNSL as a combinatorial optimisation problem. In this thesis, such formulations are important because they make it possible to express BNSL as a Quadratic Unconstrained Binary Optimisation (QUBO) problem, which can then be addressed using annealing-based solvers [22, 36].

2.9. QUBO MATHEMATICAL FRAMEWORK

Quadratic Unconstrained Binary Optimisation (QUBO) is a mathematical framework for expressing optimisation problems over binary variables [7]. In a QUBO problem, the goal is to find a binary vector $x \in \{0,1\}^p$ that minimises an energy function $E(x)$, where p is the number of binary decision variables.

The QUBO objective is commonly written as

$$E(x) = \sum_{i=1}^{p} Q_{ii} x_i + \sum_{1 \le i < j \le p} Q_{ij} x_i x_j. \tag{2.17}$$

Here, Q_{ii} represents the linear contribution of variable x_i, while Q_{ij} represents the quadratic interaction between variables x_i and x_j. Since $x_i \in \{0,1\}$, it holds that $x_i^2 = x_i$. Therefore, diagonal entries of the QUBO matrix correspond to linear terms. The same objective is often written in matrix form as $E(x) = x^T Q x$, where Q is the QUBO matrix. In practical implementations, Q is often stored in upper-triangular form, with diagonal entries representing linear coefficients and upper off-diagonal entries representing pairwise interactions.

The value $E(x)$ is often interpreted as an *energy*. Solving a QUBO problem therefore means finding the binary assignment with minimum energy. This interpretation is useful because annealing-based solvers, including simulated annealing, quantum annealing, and simulated quantum annealing, search for low-energy configurations [7, 22].

Although QUBO is called "unconstrained", constraints can be incorporated into the objective function through penalty terms [7]. A constrained optimisation problem can therefore be rewritten as

$$E(x) = E_{\text{obj}}(x) + \sum_{\ell} \lambda_{\ell} P_{\ell}(x), \tag{2.18}$$

where $E_{\text{obj}}(x)$ is the original objective function, $P_{\ell}(x)$ is a penalty function for violating constraint ℓ, and λ_{ℓ} is a positive penalty weight. A valid penalty function is zero when the constraint is satisfied and positive when the constraint is violated. If the penalty weights are chosen sufficiently large, invalid solutions receive higher energy and are therefore discouraged.

In the context of score-based Bayesian network structure learning, QUBO provides a way to encode both the objective of selecting a high-quality network structure and the constraints required for a valid Bayesian network [22]. The binary variables do not represent database values directly. Instead, they represent structural decisions, such as whether a directed edge is included in the learned graph. Score terms assign energy contributions to candidate parent-set choices, while constraint terms enforce structural properties such as maximum in-degree and acyclicity [22].

In this thesis, the maximum number of parents per node is restricted to $m = 2$. Under this restriction, the parent-set score terms involve products of at most two arc variables and can therefore be represented directly in quadratic form. As a result, no additional quadratisation of higher-order score terms is required for the QUBO formulation used in this work [22].

This framework provides the mathematical basis for applying annealing-based solvers to Bayesian network structure learning. Once the objective and constraints have been encoded as a QUBO matrix, the learning problem becomes the task of finding a minimum-energy binary assignment. The next section describes how this framework is applied specifically to Bayesian network structure learning in practice.

2.10. QUBO FORMULATION FOR BAYESIAN NETWORK STRUCTURE LEARNING

The previous section introduced QUBO as a general framework for binary optimisation. This section describes how score-based Bayesian network structure learning can be formulated as a QUBO problem. The goal is to encode both the statistical quality of a candidate Bayesian network structure and the constraints required for the structure to be a valid directed acyclic graph [22, 36].

2.10.1. HAMILTONIAN COMPONENTS

Following the Hamiltonian-based formulation of O'Gorman et al. [22], the total objective is written as

$$H(d, y, r) = H_{\text{score}}(d) + H_{\max}(d, y) + H_{\text{cycle}}(d, r). \tag{2.19}$$

Here, H_{score} represents the score-derived cost of the candidate structure, $H_{\max}$ enforces the maximum in-degree constraint, and H_{cycle} enforces acyclicity. The binary variables d, y, and r represent arc decisions, slack variables, and ordering variables, respectively. Since QUBO is a minimisation framework, the Hamiltonian is constructed so that lower energy corresponds to a better valid Bayesian network structure [22].

The main decision variables are the arc variables. For $i \neq j$, the binary variable d_{ji} is set to 1 if the directed edge $X_j \to X_i$ is selected, and to 0 otherwise. Thus, the parent set of X_i is represented by the incoming arc variables d_{ji}. For a network with n variables, there are $n(n-1)$ possible directed arc variables [22].

2.10.2. SCORE HAMILTONIAN

The score Hamiltonian encodes the statistical quality of the selected parent sets. It relies on the decomposability of Bayesian network scores introduced in Section 2.8. For each variable X_i, the contribution to the total score depends only on X_i and its selected parent set [22, 36].

Let $J \subseteq \{1, \ldots, n\} \setminus \{i\}$ denote a candidate parent set for X_i. Since QUBO is minimised, a score that would normally be maximised is converted into a local cost. In this thesis, $c_i(J)$ denotes the local cost of assigning parent set J to X_i. O'Gorman et al. [22] denote this minimised local quantity by $s_i(J)$; this thesis uses $c_i(J)$ to emphasise that it is a cost in a minimisation problem. This cost can be obtained from a decomposable scoring function such as the Bayesian Dirichlet score or BIC, as described in Section 2.8.

For a maximum allowed parent-set size m, the score Hamiltonian for node X_i is written as

$$H^{(i)}_{\text{score}}(d_i) = \sum_{\substack{J \subseteq \{1,\ldots,n\} \setminus \{i\} \\ |J| \leq m}} w_i(J) \prod_{j \in J} d_{ji}. \tag{2.20}$$

Here, d_i denotes the incoming arc variables for X_i, and the empty product is defined as 1. The coefficients $w_i(J)$ are obtained from the local costs by an inclusion–exclusion transformation, $w_i(J) = \sum_{K \subseteq J} (-1)^{|J|-|K|} c_i(K)$ [22]. This transformation ensures that the active terms in $H^{(i)}_{\text{score}}$ sum to the correct local cost for the parent set selected by the arc variables.

In this thesis, the maximum number of parents per node is restricted to $m = 2$. Under this restriction, the score Hamiltonian contains only constant, linear, and quadratic terms:

$$H^{(i)}_{\text{score}}(d_i) = w_i(\emptyset) + \sum_{j \neq i} w_i(\{j\}) d_{ji} + \sum_{\substack{j<k \\ j,k \neq i}} w_i(\{j,k\}) d_{ji} d_{ki}. \tag{2.21}$$

The constant term $w_i(\emptyset)$ represents the baseline cost for the empty parent set, while the linear and quadratic terms add the corrections needed when one or two parents are selected. Because no parent set larger than two is allowed, the score Hamiltonian is already quadratic and can be represented directly in QUBO form. Constant terms such as $w_i(\emptyset)$ shift all energies by the same amount and do not affect which assignment minimises the Hamiltonian [22, 36].

2.10.3. STRUCTURAL CONSTRAINTS

The Hamiltonian must also enforce the structural constraints of a valid Bayesian network. The first constraint is the maximum in-degree constraint, which ensures that no node has more than m parents [22]. In this thesis, $m = 2$. For each variable X_i, the number of selected parents is given by $\sum_{j \neq i} d_{ji}$. To allow this number to be less than or equal to m, a non-negative slack variable y_i is introduced. This slack variable is encoded using binary slack bits as $y_i = \sum_{\ell=0}^{\mu-1} 2^\ell y_{i\ell}$, where $y_{i\ell} \in \{0, 1\}$ and $\mu = \lceil \log_2(m+1) \rceil$ [22]. For $m = 2$, this requires two slack bits per node.

The maximum in-degree penalty for node X_i is

$$H^{(i)}_{\max}(d_i, y_i) = \delta^{(i)}_{\max} \left(m - \sum_{j \neq i} d_{ji} - y_i \right)^2. \tag{2.22}$$

Here, $\delta^{(i)}_{\max} > 0$ is a penalty weight. If the number of selected parents is at most m, then y_i can be chosen so that the expression inside the square is zero. If the number of selected parents exceeds m, no non-negative slack value can satisfy the equality, and the term contributes positive energy. Structures that violate the maximum in-degree constraint are therefore penalised.

The second structural constraint is acyclicity. A Bayesian network must be a directed acyclic graph. A directed graph is acyclic if and only if there exists a topological ordering of its nodes such that every directed edge points from an earlier node to a later node in the ordering [22]. For each pair of nodes X_i and X_j, with $i < j$, an order variable r_{ij} is introduced. It is set to 1 if X_i precedes X_j in the ordering, and to 0 if X_j precedes X_i. There are $n(n-1)/2$ such binary order variables [22].

The cycle Hamiltonian is decomposed into a transitivity term $H_{\text{trans}}(r)$ and a consistency term $H_{\text{consist}}(d, r)$. The transitivity term penalises inconsistent order assignments. For three nodes X_i, X_j, and X_k, with $i < j < k$, one transitivity penalty is

$$H_{\text{trans}}^{(ijk)} = \delta_{\text{trans}}^{(ijk)} \left(r_{ik} + r_{ij} r_{jk} - r_{ij} r_{ik} - r_{jk} r_{ik} \right). \tag{2.23}$$

This term is zero when the ordering among the three variables is transitive and positive when the ordering is inconsistent [22]. The consistency term then ensures that selected directed edges agree with the ordering. For a pair of nodes X_i and X_j, with $i < j$, the consistency penalty is

$$H_{\text{consist}}^{(ij)} = \delta_{\text{consist}}^{(ij)} \left(d_{ji} r_{ij} + d_{ij} (1 - r_{ij}) \right). \tag{2.24}$$

If $r_{ij} = 1$, then X_i precedes X_j, so the edge $X_j \to X_i$ contradicts the ordering and is penalised. Conversely, if $r_{ij} = 0$, then X_j precedes X_i, so the edge $X_i \to X_j$ is penalised. By enforcing consistency between the edge variables and a transitive ordering, directed cycles are excluded [22].

The penalty weights $\delta_{\max}$, δ_{trans}, and δ_{consist} must be chosen so that violating a structural constraint is not beneficial compared with improving the score term [22]. If the penalties are too small, the minimiser of the Hamiltonian may correspond to an invalid graph. If they are too large, the score differences between valid structures may become less influential during optimisation. For the maximum in-degree term, one sufficient condition is $\delta_{\max}^{(i)} > \max_{j \neq i} \Delta_{ji}$, where Δ_{ji} denotes an upper bound on the possible decrease in energy caused by adding the arc $X_j \to X_i$ [22]. The consistency and transitivity penalties are chosen analogously, so that graphs containing cycles remain energetically less favourable than valid directed acyclic graph structures.

2.10.4. LOGICAL VARIABLES AND MAPPING SUMMARY

The logical binary variables in the QUBO consist of arc variables, order variables, and slack variables. There are $n(n-1)$ arc variables, $n(n-1)/2$ order variables, and $n\mu$ slack bits, where $\mu = \lceil \log_2(m+1) \rceil$. Since this thesis uses $m = 2$, we have $\mu = 2$, resulting in $2n$ slack bits. Therefore, the total number of logical binary variables grows as $O(n^2)$.

Due to connectivity constraints of current quantum annealers, logical QUBOs must undergo minor embedding, resulting in a physical qubit requirement that is significantly higher than the original logical variable count [22, 36].

Overall, the QUBO formulation transforms score-based Bayesian network structure learning into an energy-minimisation problem. Arc variables encode candidate directed edges, score-derived terms encode the local quality of parent-set choices, slack variables enforce the maximum in-degree constraint, and order variables enforce acyclicity through topological consistency [22].

For the setting used in this thesis, where each node is allowed at most two parents, all terms in the Hamiltonian are linear or quadratic. The resulting Hamiltonian can therefore be represented directly as a QUBO matrix. A minimum-energy assignment can be decoded into a Bayesian network structure [22, 36]. This structure is then parameterised using the counting-based procedure described earlier and subsequently used for selectivity estimation through inference.

2.11. ANNEALING-BASED SOLVERS

Once Bayesian network structure learning has been encoded as a QUBO, the learning task becomes an energy-minimisation problem. A candidate solution corresponds to a binary assignment of the QUBO variables, and its energy is determined by the objective encoded in the QUBO matrix [7, 22]. This thesis focuses experimentally on simulated annealing (SA) and simulated quantum annealing (SQA). Quantum annealing (QA) is introduced as background because it motivates the QUBO formulation and provides the physical model that SQA aims to simulate [22, 36].

The solvers share the same input QUBO formulation but differ in how they explore the energy landscape. SA relies on classical thermal fluctuations, while SQA uses a classical simulation of quantum annealing dynamics. Hardware-based QA is not evaluated in the experimental part of this thesis.

2.11.1. SIMULATED ANNEALING (SA)

Simulated annealing (SA) is a stochastic local search algorithm inspired by the physical process of thermodynamic annealing. It is designed to find global or near-global optima in discrete configuration spaces by navigating complex energy landscapes [17]. Unlike greedy descent methods, which can easily become trapped in local optima, SA can escape such traps by allowing probabilistic uphill moves, meaning transitions to higher-energy states. The probability of accepting these energy-increasing moves is governed by a computational temperature parameter T. At high initial temperatures, the algorithm explores the configuration space more freely and can traverse different local valleys (LVs). As T is gradually reduced towards zero according to a predefined cooling schedule, the probability of accepting uphill moves decreases, causing the system to relax into a low-energy state within its current local valley [17].

In this thesis, SA is used to solve a Quadratic Unconstrained Binary Optimisation (QUBO) problem. The constraints and parameters extracted from the dataset are mathematically encoded into the upper-triangular weight matrix Q. The SA algorithm then seeks the binary vector x that minimises the quadratic objective function $E(x) = x^T Q x$ [7, 22].

Because the energy landscape of a complex QUBO often contains numerous local minima separated by different escape barriers, a single stochastic run is usually insufficient. Comparative studies on classical and quantum annealing show that classical SA relies strongly on the size of a local valley's basin of attraction when discovering low-energy states [17]. To reduce the risk of converging only to valleys with wide basins rather than to lower-energy solutions, the optimisation procedure is executed across a predetermined number of *annealing reads*, where the number of reads is treated as a solver hyperparameter.

A single annealing read corresponds to one independent Markov chain Monte Carlo (MCMC) trajectory. Each annealing read begins from a random initial configuration and follows a complete thermal cooling schedule, yielding one final low-energy state and its associated energy value [17]. The solution with the minimum observed energy across all annealing reads is selected as the best candidate found by the solver, while the frequency of identical solutions, also called occurrences, provides useful insight into the stability of the search and the distribution of low-energy solutions.

2.11.2. QUANTUM ANNEALING (QA)

Quantum annealing (QA) is a quantum optimisation approach for searching for low-energy configurations of an Ising or QUBO model. Its theoretical basis is closely related to adiabatic quantum computation, where an optimisation problem is encoded into the energy landscape of a quantum system [36]. The objective is to reach a low-energy state of the problem Hamiltonian, ideally its ground state, which represents the optimal solution of the original problem [36].

In QA, the system evolves from an initial driver Hamiltonian H_D, whose ground state is easy to prepare, towards a problem Hamiltonian H_P, which encodes the optimisation objective. This evolution can be described as

$$H(t) = \Gamma(t)H_D + H_P, \tag{2.25}$$

where $\Gamma(t)$ decreases during the annealing process [36]. At the beginning, the driver Hamiltonian dominates and introduces quantum fluctuations. As the anneal progresses, the problem Hamiltonian becomes dominant and guides the system towards low-energy configurations. Under ideal adiabatic conditions, the system remains close to its instantaneous ground state and converges to the ground state of H_P [36].

This model is relevant for Bayesian network structure learning because the QUBO Hamiltonian described in Section 2.10 can be interpreted as a problem Hamiltonian. A low-energy binary assignment corresponds to a candidate Bayesian network structure [22, 36]. Although quantum annealing is commonly expressed in the Ising representation with spin variables $z_i \in \{-1, +1\}$, this

representation is equivalent to the QUBO representation over binary variables $x_i \in \{0, 1\}$, and the two can be converted into each other [7, 36].

Although hardware-based quantum annealing is not evaluated experimentally in this thesis, it is useful to note that running a QUBO on quantum annealing hardware requires an additional mapping step. Quantum annealers are not fully connected; their qubits are arranged according to a fixed hardware topology. Therefore, a logical QUBO graph may need to be embedded into the physical hardware graph using minor embedding [3, 4]. In this process, one logical variable may be represented by a chain of several physical qubits, which increases the number of physical qubits required for a given logical problem. This hardware overhead is one reason why the experimental part of this thesis focuses on simulated annealing and simulated quantum annealing rather than hardware-based quantum annealing.

2.11.3. SIMULATED QUANTUM ANNEALING (SQA)

Simulated quantum annealing (SQA) is a heuristic optimisation method that emulates aspects of quantum annealing using classical path-integral Monte Carlo (PIMC) simulations [15]. It approximates the behaviour of the transverse-field Ising model by simulating a system influenced by both quantum and thermal fluctuations. The mathematical basis of SQA is the Suzuki–Trotter transformation, which maps a d-dimensional quantum system to a $(d + 1)$-dimensional classical system. In this representation, each logical variable is represented by M Trotter slices, which can be understood as coupled replicas of the same variable. These replicas allow the simulation to approximate stochastic quantum fluctuations and to explore complex energy landscapes in a way that is inspired by quantum tunnelling [15].

The effective SQA model contains two types of interactions: interactions within each Trotter slice and interactions between neighbouring slices. Intra-slice interactions represent the classical problem objective, such as the QUBO energy of a candidate solution. Inter-slice couplings connect adjacent replicas and are used to model the effect of the transverse field. The quality of this approximation depends on the Trotter number M. A larger value of M can provide a more detailed approximation of quantum behaviour, but it also increases the computational cost of the simulation [15].

The convergence of SQA towards low-energy configurations is influenced by the number of Monte Carlo sweeps and by the temperature schedule. A sweep corresponds to a Monte Carlo update of the extended system, including the variables and their Trotter replicas. In the SQA implementation used in this thesis, the solver is run with a fixed sweep count and a geometric beta schedule, following the configuration used in the annealing-based BNSL pipeline [36]. The

schedule controls how the simulation moves from broader exploration towards more concentrated sampling of low-energy configurations.

As with simulated annealing, SQA is stochastic. It is therefore executed across multiple independent annealing reads. Each read produces one sampled low-energy assignment of the QUBO variables, which can then be decoded into a candidate Bayesian network structure. Running several reads increases the chance of observing high-quality structures, but it does not guarantee that the global optimum is found [22, 36]. In this thesis, SQA is used as a quantum-inspired sampling method for the QUBO-based BNSL formulation and is evaluated through the quality of the resulting Bayesian network structures in the downstream selectivity-estimation task.

3. Related Work

Traditional Cardinality Estimation. Cost-based query optimisers depend on cardinality estimates when comparing alternative execution plans. In the System R optimiser, access paths and join orders were already selected using estimated costs and catalog statistics [27]. Recent survey work still identifies cardinality estimation, cost modelling, and plan enumeration as central components of query optimisation [18]. This makes cardinality estimation a core optimiser problem and provides the starting point for this chapter.

A good amount of work has focused on compact statistics that can be maintained efficiently inside a DBMS. Early work by Piatetsky-Shapiro and Connell studied how summaries of attribute distributions can be used to estimate the number of tuples satisfying selection predicates [23]. PostgreSQL follows this traditional direction in practice. It estimates row counts using catalog statistics such as table cardinalities, histograms, most-common-value lists, null fractions, and distinct-value estimates [30, 29]. For conjunctive predicates, PostgreSQL may combine individual selectivities under an independence assumption, as illustrated in its row-estimation examples [30]. This makes PostgreSQL a suitable traditional baseline for this thesis.

The main limitation of such statistics is that they often rely on simplifying assumptions. Poosala and Ioannidis address this issue by studying selectivity estimation without the attribute value independence assumption [24]. Their work shows that multi-attribute predicates require information about dependencies between attributes. Markl et al. study a related problem for conjunctive predicates when only partial multivariate statistics are available [20]. Their maximum-entropy approach combines available statistics in a consistent way. These works are important because they show that dependency information is needed when predicates involve correlated attributes.

The effect of inaccurate estimates has also been studied from the perspective of query plans. Ioannidis and Christodoulakis analyse how errors in estimated relation sizes can propagate through join queries [13]. Leis et al. provide empirical evidence that large cardinality-estimation errors still occur in modern optimisers [19]. Although these studies focus on joins, they are relevant here because they show that estimation errors can affect later cost comparisons

and plan choices. The general lesson is that cardinality estimation should be evaluated not only as a statistical task, but also as part of query optimisation.

Learning-Based Cardinality Estimation. The limitations of traditional statistics have motivated a broad line of learning-based cardinality estimators. Prior work often distinguishes between query-driven and data-driven approaches. Query-driven methods learn from query features and observed cardinalities. Data-driven methods learn a model of the underlying data distribution and derive estimates from that model [32, 11]. Naru and DeepDB are representative examples of data-driven learned estimators. Naru uses deep autoregressive models to approximate the joint distribution of a relation without relying on explicit column-independence assumptions [35]. DeepDB follows a related direction by learning a probabilistic representation of the database instead of collecting query-cardinality pairs [12]. These systems show that learned cardinality estimation can be based on modelling the data distribution itself.

At the same time, learned estimators are not automatically practical for database systems. Wang et al. provide a critical evaluation of learned cardinality estimation in single-table settings [32]. They report that learned methods can be more accurate than traditional methods in static settings. They also identify practical barriers such as training cost, inference cost, hyperparameter tuning, and frequent data updates. In addition, they show that the performance of learned methods can be strongly affected by changes in correlation, skewness, or domain size [32]. Han et al. give a broader benchmark perspective by evaluating representative cardinality estimation methods inside PostgreSQL [11]. Their study considers end-to-end query performance and practical deployment aspects. They also argue that q-error alone does not fully reflect query plan quality, since different sub-plan estimates can affect the optimiser differently [11]. These results motivate estimators that are more expressive than traditional statistics, but still structured enough to remain practical.

Bayesian-Network-Based Selectivity Estimation. Bayesian networks and related graphical models provide one such structured alternative. They can model dependencies between attributes without materialising the full multidimensional distribution [6, 9]. Early work by Getoor et al. connects selectivity estimation with probabilistic modelling [6]. Tzoumas et al. later propose a lightweight graphical-model approach for selectivity estimation without independence assumptions and integrate it into PostgreSQL [31]. These works show the central trade-off for graphical models in query optimisation. They can capture useful correlations, but their structure must remain simple enough for practical construction and inference.

Halford, Saint-Pierre, and Morvan study this trade-off directly for Bayesian networks [9]. Their approach is motivated by the attribute value independence assumption and approximates the distribution of attribute values inside each relation with a Bayesian network. This makes their work especially relevant for the present thesis, because it focuses on intra-relation dependencies and therefore matches the single-relation scope of this work. To keep learning and inference efficient, Halford et al. restrict the Bayesian-network structure to a Chow–Liu tree. The theoretical basis for this model goes back to Chow and Liu [2]. This restriction makes the model compact and practical. However, it also limits expressiveness because each attribute can have at most one parent. Chow–Liu trees therefore provide an important Bayesian-network baseline, but they do not answer whether richer Bayesian-network structures can improve selectivity estimation.

The later work on linked Bayesian networks extends this line of research to cross-relation dependencies [10]. Halford and Saint-Pierre connect per-relation Bayesian networks to soften both the attribute value independence assumption and the join-uniformity assumption. This work is outside the main experimental scope of the present thesis, which focuses on single-relation selection queries. Still, it reinforces the same central point. To keep the estimator practical, the Bayesian-network structure must therefore be chosen carefully.

BayesCard also revisits Bayesian networks for cardinality estimation [34]. Wu et al. build an ensemble of Bayesian networks to model database table distributions and estimate query cardinalities. Their work positions Bayesian networks as a practical alternative to deep learned estimators and emphasises properties such as interpretability, smaller model size, faster updates, and system deployment [34]. For this thesis, BayesCard is important because it shows that Bayesian networks remain relevant for cardinality estimation beyond simple tree-structured baselines. Together with the work by Halford et al., it motivates the use of Bayesian networks as structured dependency-aware estimators.

Annealing-Based Bayesian Network Structure Learning. The remaining question is how richer Bayesian-network structures can be learned. Bayesian network structure learning is a broad research area. Kitson et al. survey constraint-based, score-based, and hybrid approaches for learning Bayesian-network structures [16]. This thesis is closest to the score-based setting. The difficulty of this task is also formal. Chickering shows that identifying a Bayesian-network structure with a sufficiently high BDe score is NP-complete, even when the number of parents per node is bounded [1]. This helps explain why richer Bayesian networks cannot simply replace Chow–Liu trees without additional optimisation strategies.

O'Gorman et al. propose one such strategy by formulating score-based Bayesian network structure learning as a Quadratic Unconstrained Binary Optimisation problem [22]. Their formulation combines graph quality and structural constraints in one optimisation objective. This connects BNSL to quantum annealing and also makes the formulation usable with classical heuristics such as simulated annealing. For the present thesis, this work is central because it provides a way to search for Bayesian-network structures that are richer than Chow–Liu trees.

Zardini et al. build directly on this formulation and provide an empirical implementation of O'Gorman et al.'s approach [36]. They evaluate the method on Bayesian-network reconstruction problems of increasing size. Their results show the effectiveness of O'Gorman's formulation for small BNSL instances, while larger instances remain difficult because of QUBO construction cost and hardware limitations [36]. They also investigate a divide-et-impera approach that improves scalability in their experiments, although the resulting structures are not always optimal [36]. This work is closely related to the present thesis because it studies annealing-based BNSL empirically. However, its evaluation focuses on reconstructing Bayesian-network structures, not on using the learned structures for query optimisation.

Research Gap. Taken together, the reviewed work shows a clear gap. Cardinality estimation has been studied for a long time, and learned estimators have shown that richer models can improve estimation accuracy in some settings. Bayesian-network-based estimators are attractive because they model dependencies in a structured way. Chow–Liu trees provide an efficient baseline, but their tree topology limits expressiveness. Annealing-based BNSL offers a possible way to search for richer Bayesian-network structures. However, existing annealing-based BNSL work mainly studies structure learning or reconstruction quality. To the best of our knowledge, it does not evaluate annealing-learned Bayesian networks as downstream cardinality estimators for query optimisation. This thesis addresses this gap by evaluating Bayesian networks learned with simulated annealing and simulated quantum annealing for single-relation cardinality estimation. The learned models are compared against PostgreSQL and a Chow–Liu Bayesian-network baseline using the same datasets, query workloads, and q-error-based evaluation.

4. Proposed Framework: AnnealBN-CE

This chapter presents AnnealBN-CE, the framework proposed in this thesis for cardinality estimation with annealing-learned Bayesian networks. It addresses the first research question by showing how Bayesian network structures learned from the QUBO formulation introduced in Section 2.10 can be integrated into a cardinality-estimation workflow.

The scope of the framework is limited to single-relation selection queries without joins. It therefore models dependencies among attributes within one relation, but does not estimate join cardinalities or dependencies across relations. The chapter focuses on the conceptual and algorithmic design of AnnealBN-CE. Its concrete datasets, run configurations, and empirical results are discussed in Chapter 6.

4.1. Framework Overview

AnnealBN-CE bridges the gap between annealing-based Bayesian network structure learning and cardinality estimation. It connects the QUBO formulation for Bayesian network structure learning described in Section 2.10 with Bayesian-network-based selectivity estimation introduced in Section 2.6. The challenge is that the solver returns binary assignments and energy values, whereas cardinality estimation requires a fitted probabilistic model to evaluate query predicates.

The framework operates on a single relation whose attributes are modelled as Bayesian network variables. To make the annealing step usable for cardinality estimation, AnnealBN-CE keeps two aligned representations of this relation. The solver representation is used for structure learning, where the data is encoded as finite integer states for local score computation and QUBO construction. The tabular representation is used after structure learning, so that parameter fitting and query evaluation remain linked to the original attributes and values of the relation.

At a high level, AnnealBN-CE consists of four conceptual components. First, the solver representation is transformed into a QUBO instance and submitted to the annealing solver. Second, the lowest-energy returned assignment is decoded

into a directed adjacency matrix, which represents the learned Bayesian network structure. Third, this matrix is mapped back to the attributes of the relation, and the conditional probability tables are fitted from the tabular data. Fourth, the fitted Bayesian network evaluates query predicates probabilistically, producing selectivities that are converted into cardinality estimates using Eq. (2.3).

The output of AnnealBN-CE is a cardinality estimate at query level. If repeated solver executions produce different learned structures, each structure can be fitted and evaluated separately. This makes it possible to study not only the resulting estimates, but also their stability across learned structures. The detailed execution procedure is described in Section 4.3.

4.2. THE ANNEALBN-CE FRAMEWORK

The previous section introduced the purpose and overall organisation of AnnealBN-CE. This section describes the individual components of the framework in more detail. The focus is on the transformation from the solver representation to a learned Bayesian network structure, and from this structure to query-level cardinality estimates.

4.2.1. FROM SOLVER REPRESENTATION TO QUBO INSTANCE

The first component of AnnealBN-CE constructs the QUBO instance used for annealing-based Bayesian network structure learning. It starts from the integer-encoded solver representation. This representation specifies the number of variables, the number of states of each variable, a reference solution array, and the encoded observations. When a ground-truth Bayesian network is known, the reference solution array encodes its adjacency structure for diagnostic comparison with the learned structure; otherwise, a dummy array is used. The array is not used by the learning pipeline.

Using the encoded observations and the number of states per variable, the implementation computes the local score terms and constraint penalties needed to populate the QUBO matrix Q. Candidate parent sets are restricted to the maximum in-degree $m = 2$, so each variable may have no parent, one parent, or two parents. The QUBO formulation uses arc variables d, slack variables y, and ordering variables r. The arc variables represent directed edge choices, the slack variables encode the maximum in-degree constraint, and the ordering variables are used to enforce acyclicity.

For a relation with n Bayesian network variables, the formulation contains $n(n-1)$ arc variables, $n\lceil \log_2(m+1) \rceil$ slack variables, and $n(n-1)/2$ ordering variables. Since this thesis restricts the maximum number of parents to $m = 2$, the number of slack variables is $2n$. These logical variables define the binary optimisation problem that is passed to the annealing solver.

Before the instance is submitted to the solver, the QUBO matrix Q is converted into a solver-compatible dictionary representation. This dictionary stores the non-zero coefficients of Q using pairs of logical QUBO variables as keys. The implementation also keeps the variable index, which records the order of the logical variables in the returned binary assignment. This index is needed in the next component to identify which entries correspond to arc variables and therefore to recover the learned graph structure.

4.2.2. ANNEALING OUTPUT AND GRAPH DECODING

The annealing solver receives the QUBO dictionary constructed in the previous component and searches for low-energy binary assignments. In this thesis, the solver is instantiated using simulated annealing or simulated quantum annealing, as introduced in Section 2.11. Each returned assignment represents a binary configuration of the logical QUBO variables, namely the arc variables d, the slack variables y, and the ordering variables r.

For each solver execution, the implementation selects the lowest-energy assignment returned by the solver. This assignment is converted into an ordered binary vector using the variable index kept during QUBO construction. This index records the fixed order of the logical QUBO variables in the vector representation. In the solver output, this vector is reported as `minX`. It contains values for all logical QUBO variables. The reported `minY` value of this assignment, computed as $x^\top Qx$, is used only as a solver-side diagnostic.

The graph structure is recovered from the arc-variable part of this ordered assignment. Since the QUBO index places the arc variables before the auxiliary variables, the first $n(n-1)$ entries correspond to directed edge decisions. These entries are extracted and reshaped into a directed adjacency matrix using the same variable order as the solver representation. With the adjacency convention used in this thesis, an entry $A_{ij} = 1$ denotes the directed edge $X_i \to X_j$.

The auxiliary slack and ordering variables are discarded after decoding because they are only needed to encode the structural constraints during optimisation. If repeated solver executions produce the same adjacency matrix, duplicate structures are removed before downstream evaluation. The output of this component is therefore a set of directed adjacency matrices that represent the learned Bayesian network structures.

4.2.3. BAYESIAN NETWORK CONSTRUCTION AND PARAMETER FITTING

The previous component outputs directed adjacency matrices that represent learned Bayesian network structures. Each matrix specifies which directed edges were selected by the annealing-based structure-learning step, but the matrix

entries must still be interpreted with respect to the attributes of the relation. This is done using the shared variable order of the solver representation and the tabular representation. Each selected matrix entry is therefore mapped to a directed edge between two relation attributes.

For each learned structure, the mapped edges determine the parent set of each Bayesian network variable. The framework then fits the conditional probability tables using the tabular representation and maximum likelihood estimation, as introduced in Section 2.5. In particular, AnnealBN-CE uses the frequency-counting formulation in Eq. (2.9) to turn the learned structure into a fitted probabilistic model. Cases in which a parent configuration is not observed in the data are discussed separately in Section 2.5.2.

After parameter fitting, each learned structure becomes a fitted Bayesian network with conditional probability tables. This fitted network is passed to the query-evaluation component, where it is used to compute selectivities and cardinality estimates.

4.2.4. Query Evaluation and Cardinality Output

After parameter fitting, the Bayesian network is used to evaluate query predicates probabilistically. The attributes that appear in a query predicate are matched to the corresponding Bayesian network variables, so that the predicate defines the event whose probability is estimated.

For a query $q = \{A_1 = a_1, \ldots, A_k = a_k\}$, the fitted Bayesian network estimates the joint probability

$$\widehat{P}(A_1 = a_1, \ldots, A_k = a_k).$$

This probability is interpreted as the estimated selectivity. This interpretation matches the semantics of conjunctive selection predicates, where all specified conditions must hold at the same time. Variables that are not part of the query are marginalised out by variable elimination during inference, as introduced in Section 2.6.

The estimated selectivity is then converted into an estimated cardinality using Eq. (2.3). The output of this component is a query-level cardinality estimate. If several unique adjacency matrices are obtained from repeated solver executions, each corresponding fitted Bayesian network is evaluated separately. As a result, downstream evaluation is based on the cardinality estimates produced for the single-relation queries.

4.3. AnnealBN-CE Execution Procedure

Figure 4.1 shows the execution procedure of AnnealBN-CE. It complements the component-level description in Section 4.2 by showing how the framework

is organised into a reproducible workflow. The procedure is designed to support controlled structure-learning runs, preserve solver outputs, reuse learned structures, and generate cardinality-estimation result files.

The procedure starts with the decision at ①. On a first execution, no stored matrices are available, so the workflow follows the solver branch. It starts from the selected TXT representation, namely the integer-encoded solver representation introduced in Section 4.2.1. This representation is used for QUBO construction and annealing-based structure learning.

The solver configuration is specified at ②. It consists of the annealing method, the number of annealing reads, denoted as reads in the figure, and the number of trials. The annealing method is selected from as either SA or SQA solvers introduced in Section 2.11. Each read produces a candidate binary assignment together with its energy value. The lowest-energy assignment among the reads is selected for decoding. The number of trials specifies how many solver runs are executed for the chosen method and read setting.

The solver outputs are shown at ③. For each trial, the lowest-energy assignment is decoded into a solution matrix, as described in Section 4.2.2. In this workflow, solution matrices are the decoded adjacency matrices returned by the structure-learning component. The associated solver diagnostics are stored with these outputs, so that each solution matrix remains linked to the solver-side information from the run.

Before downstream evaluation, the solution matrices are deduplicated. If the same adjacency matrix is obtained more than once for a solver configuration, it is kept once as a unique matrix, while an occurrence count records how often it appeared. This prevents repeated cardinality-estimation runs for identical structures. The unique matrices are also visualised as Bayesian networks, so that the learned dependencies can be inspected as graphs.

At ⑤, the workflow either stops after structure learning or continues to cardinality estimation. Stopping at this point is useful when the goal is to compare solver settings, inspect the learned Bayesian networks, or preserve the matrices for later use. If the workflow continues, the cardinality-estimation routine associated with the selected TXT representation is executed.

The right branch from ① supports reuse of stored unique matrices from earlier runs. The selected matrices are represented at ④ and are passed to the selected cardinality-estimation routine. The selected routine applies Bayesian network fitting and query evaluation to the selected unique matrices, as described in Sections 4.2.3 and 4.2.4.

Finally, at ⑥, the resulting cardinality estimates are written to a CSV file together with the corresponding queries. This file is then used for the query-level analysis in Chapter 6.

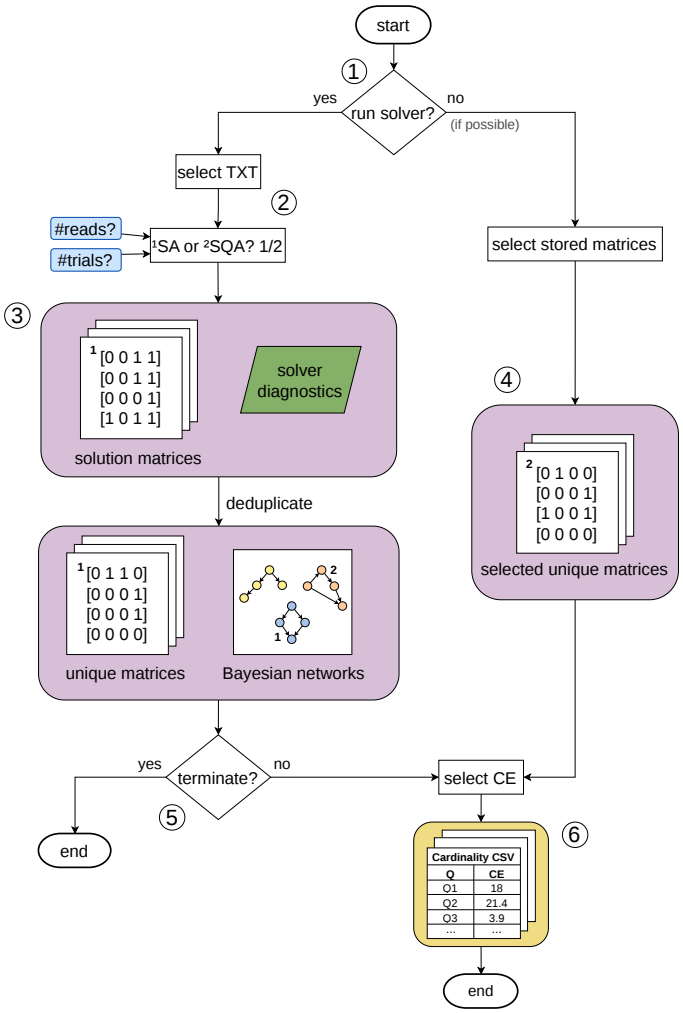

Figure 4.1.: Execution procedure of AnnealBN-CE. The numbered circles mark the main points of the workflow: ① deciding whether to run the solver or, if possible, reuse stored matrices; ② configuring the annealing method, reads, and trials; ③ storing solution matrices and solver diagnostics; ④ selecting stored unique matrices; ⑤ deciding whether to continue to cardinality estimation after structure learning; and ⑥ generating the cardinality CSV file. Purple blocks denote stored structure artefacts, including solution matrices, unique matrices, selected unique matrices, and BN visualisations. The green parallelogram denotes solver diagnostics, blue boxes denote solver hyperparameters, and the yellow block denotes the final cardinality output file. Small numerical labels inside the purple blocks indicate which adjacency matrix corresponds to which BN visualisation.

4.4. SUMMARY

This chapter introduced AnnealBN-CE, the proposed framework for cardinality estimation with annealing-learned Bayesian networks. The framework starts from an integer-encoded solver representation, constructs a QUBO instance for Bayesian network structure learning, and uses an annealing solver to obtain low-energy binary assignments. These assignments are decoded into adjacency matrices that represent Bayesian network structures.

The chapter then described how the decoded structures are mapped back to the attributes of the relation, fitted from tabular data using maximum likelihood estimation, and used to evaluate query predicates probabilistically. The resulting selectivity estimates are multiplied by the relation size to obtain query-level cardinality estimates.

Finally, the chapter presented the execution procedure used to organise solver runs, extract unique structures, store solver diagnostics and structure outputs, visualise the learned Bayesian networks, reuse stored structures, and generate cardinality result files.

Overall, AnnealBN-CE defines the methodological basis used in this thesis to connect annealing-based Bayesian network structure learning with single-relation cardinality estimation. Its modular design separates solver representation, structure learning, parameter fitting, and query evaluation, leaving room for future extensions to larger relations or join-aware workloads. The next chapter introduces the Chow–Liu Bayesian network benchmark, which serves as the classical comparison method.

5. Classical Baseline: Chow–Liu Tree Estimator

The previous chapter introduced AnnealBN-CE, the proposed framework for cardinality estimation with annealing-learned Bayesian networks. This chapter turns to the Chow–Liu estimator, which serves as the main classical Bayesian-network baseline for comparison with the proposed framework.

The Chow–Liu benchmark is a well-established tree-structured Bayesian-network estimator. It provides a natural comparison point for AnnealBN-CE because it also models dependencies between attributes probabilistically, but restricts the dependency structure to a tree. Under this restriction, each non-root variable has at most one parent. The model is therefore efficient to learn, but less expressive than the more general Bayesian network structures considered in the annealing-based framework.

The mathematical principle of Chow–Liu learning was introduced in Section 2.7. This chapter builds on that foundation by describing how the estimator is turned into a cardinality-estimation baseline for this thesis. The concrete datasets, query workloads, and empirical results are discussed later in Chapter 6.

5.1. BASELINE WORKFLOW

The Chow–Liu estimator represents the attributes of a relation using a tree-structured Bayesian network. The tree provides a compact synopsis of the relation by capturing pairwise dependencies between attributes, while preserving a structure that remains efficient for parameter fitting and inference. In this thesis, the estimator is used as the classical Bayesian-network baseline for comparison with the proposed framework, AnnealBN-CE.

The learning principle, histogram-based parameter representation, and inference procedure were introduced in Sections 2.5.1, 2.6 and 2.7. In this thesis, these inherited components are brought together into a reproducible cardinality-estimation workflow. The reproduced codebase supports a broader view of Bayesian-network-based selectivity estimation, including linked Bayesian networks over multiple relations. Here, the workflow is deliberately narrowed to the

single-relation estimator to match the scope of AnnealBN-CE. It takes a tabular relation and a query workload as input, fits a tree-structured Bayesian network, translates selection predicates into model evidence, evaluates the corresponding probabilities, and exports the resulting query-level cardinality estimates.

The workflow starts by treating the retained attributes of the relation as Bayesian-network variables. Pairwise dependency scores are computed between these variables, a maximum-weight spanning tree is extracted, and the tree is oriented into a directed Bayesian network. The resulting structure is a directed tree, so each non-root variable has at most one parent.

The model is then parameterised. The root variable is represented by a marginal distribution, while each non-root variable is represented by a conditional distribution given its parent. The parent-child relations determined by the learned tree therefore define which conditional distributions have to be fitted.

For query evaluation, selection predicates are translated into evidence over the corresponding relation attributes. For multi-predicate queries, inference is restricted to the relevant Steiner-tree subgraph as described in Section 2.6. The inferred probability is interpreted as the estimated selectivity and converted into an estimated cardinality using Eq. (2.3).

Further adaptations concern the fitting procedure, missing-value handling, and histogram robustness. The fitting procedure was adjusted so that conditional distributions are computed only after the Chow–Liu tree has been constructed, since the parent-child relations are defined by the learned structure. Missing values are handled explicitly so that they remain part of the fitted probabilistic model instead of being removed implicitly. The histogram representation was made more robust during fitting and inference by enforcing consistent numerical handling, treating empty inputs and null fractions explicitly, and making bucket lookup safe for boundary cases. These adaptations preserve the Chow–Liu learning principle while making the reproduced estimator reliable for query-level cardinality estimation on a single tabular relation.

5.2. BASELINE EXECUTION PROCEDURE

Figure 5.1 shows the simplified execution procedure of the Chow–Liu baseline. The figure abstracts the internal steps of the selected cardinality-estimation file to keep the focus on cardinality result file.

The procedure starts with the database selection at ①, which determines the tabular relation used for the baseline run. The corresponding Chow–Liu cardinality-estimation file is then selected at ②. This file executes the baseline workflow described in Section 5.1, including the query workload and the cardinality export.

The output is a cardinality CSV file. For each query, it contains the Chow–Liu estimate, the true cardinality, and the PostgreSQL estimate, denoted in the figure as CLCE, TCE, and PCE, respectively. This file is used for the query-level analysis in Chapter 6.

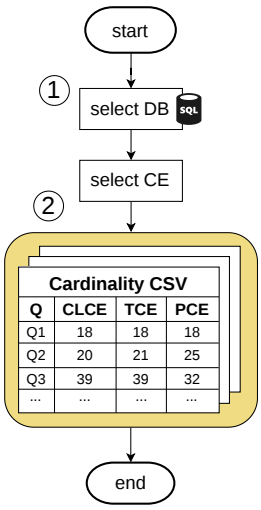

Figure 5.1.: Simplified execution procedure of the Chow–Liu baseline. The numbered circles mark the two main decisions: ① selecting the database and corresponding tabular relation, and ② selecting the Chow–Liu cardinality-estimation file. The selected file executes the baseline workflow and produces a cardinality CSV file with query-level Chow–Liu estimates, true cardinalities, and PostgreSQL estimates, denoted as CLCE, TCE, and PCE in the figure. Internal preprocessing, model fitting, and query evaluation are abstracted to keep the diagram focused on result-file generation.

5.3. SUMMARY

This chapter established how the reproduced Chow–Liu estimator is used as the classical Bayesian-network baseline in this thesis. Its existing components are kept unchanged in principle but are assembled into a reproducible workflow for query-level cardinality estimation.

The workflow starts from a tabular relation, learns and parameterises a tree-structured Bayesian network, translates selection predicates into model evidence,

and converts the inferred selectivities into cardinality estimates. It is restricted to the same single-relation setting as AnnealBN-CE, so that both estimators are evaluated within the same scope.

The chapter also described the adaptations needed to make the reproduced estimator reliable for this workflow. Finally, the simplified execution procedure showed how the baseline is run to produce cardinality CSV files used in the empirical evaluation.

6. Experimentation and Evaluation

The previous chapters introduced AnnealBN-CE as the proposed framework and the Chow–Liu tree estimator as the classical Bayesian-network baseline. This chapter moves from workflow design to empirical evaluation by applying these methods to concrete single-relation cardinality-estimation workloads.

The central question is whether Bayesian network structures learned by annealing-based methods can serve as effective statistical models for estimating the cardinalities of selection queries. The evaluation therefore focuses on query-level estimation quality: the learned graph is assessed through the cardinality estimates it produces, not as a graph object in isolation.

To answer this question, AnnealBN-CE is compared with PostgreSQL and the Chow–Liu baseline on fixed query workloads, with accuracy measured against the true cardinality. This comparison addresses whether the additional expressiveness of annealing-learned Bayesian networks provides an advantage over both baselines. Since simulated annealing and simulated quantum annealing are stochastic structure-learning methods, the chapter also examines graph diversity, solver stability, and the effect of different run configurations on downstream estimates. Where appropriate, robustness checks are used to assess whether observed estimation behaviour is specific to annealing-learned structures or could also arise from non-optimised graph choices.

The experiments are conducted on three datasets chosen to evaluate the framework under different data and domain characteristics: a controlled synthetic Bayesian network, a real health-survey subset, and synthetic binary transaction data. Across these settings, the chapter evaluates the accuracy and stability of AnnealBN-CE. It also analyses the strengths and weaknesses that emerge when the framework is applied to single-relation selection queries.

The remainder of this chapter is organised as follows. First, the implementation notes, execution platform, data representations, query workload design, annealing run configurations, and evaluation procedure are described in detail. These sections define the common experimental basis used across all datasets, including the q-error evaluation and the random-structure robustness check used where applicable.

The chapter then presents the dataset-specific experiments: the controlled synthetic WetGrass Bayesian-network dataset, the real NHANES health-survey subset, and the synthetic Market Basket transaction dataset. Each experiment follows the same general structure. The setup describes the dataset representation, preprocessing choices, query workload, and read–trial configurations. The results then report the cardinality estimates, q-error behaviour, graph diversity, and stability of the learned structures. Where applicable, a random-structure robustness check is reported as an additional negative control. Finally, a dataset-specific discussion relates the observed behaviour to the research questions.

For the two synthetically generated datasets, WetGrass and Market Basket, the evaluation is further split by relation size. The smaller relations are used to analyse sparse-frequency behaviour, while the larger relations are used to test whether the same underlying dependency patterns lead to more stable and accurate estimates when substantially more tuples are available.

The broader interpretation of trends, strengths, weaknesses, and limitations is reserved for the following chapter.

6.1. IMPLEMENTATION NOTES

The experimental implementation contains the code used to run the two execution procedures introduced in the preceding chapters: AnnealBN-CE and the Chow–Liu baseline. Python source files handle data conversion, Bayesian-network fitting, probabilistic inference, and query evaluation, and export the cardinality estimates to CSV files. Shell scripts manage repeated solver and evaluation runs, extract adjacency matrices, merge generated CSV files, and produce the final cardinality bar charts and q-error plots.

The implementation builds on two existing codebases. The Chow–Liu baseline reuses components from `MaxHalford/tldks-2020` [8], while the annealing-based structure-learning pipeline is based on `BNSL-QA-python` [26]. Both codebases were adapted for the current software environment and integrated into a common workflow for single-relation cardinality estimation.

The implementation size is reported for the source files that are part of the final experimental workflow. Blank lines and comments are excluded from the count. Newly written workflow scripts contribute `661` lines of code on the Chow–Liu side and `1016` lines of code on the annealing side. For adapted repository files, the changes are reported relative to the original repository versions. The adapted Chow–Liu files account for `284` changed lines. The adapted `BNSL-QA-python` files add `256` lines and remove `79` lines. Generated CSV files, datasets, plots, logs, and virtual environments are excluded from these counts.

The main libraries used in the implementation are `pandas`, `numpy`, `NetworkX`, `scikit-learn`, `pgmpy`, `dimod`, `dwave-neal`, `dwave-system`, `torch`, `SQLAlchemy`,

psycopg2, and graphviz. The final source code and the environment specification used for the experiments are included with the thesis materials.

6.2. EXECUTION PLATFORM

All experiments were executed locally on a MacBook Pro from 2021 equipped with an Apple M1 Pro 10-core CPU, 16 GB of unified memory, and a 512 GB SSD. The host architecture was arm64. The operating system used for the final experiments was macOS Tahoe version 26.3.1, and the implementation was executed with Python 3.13.11.

PostgreSQL was used as the database-system baseline in a Docker-based environment. The PostgreSQL version used in the experiments was PostgreSQL 12.22, running on Debian package version 12.22-1.pgdg120+1. Docker Desktop version 4.57.0 was used with its default resource settings to manage the containerised database environment. Pgweb 0.17.0 was used as an inspection interface for the PostgreSQL databases.

The Bayesian-network and annealing components were executed in the local Python environment, while PostgreSQL was isolated inside Docker. No hardware quantum annealer was used in the final experiments. Simulated annealing and simulated quantum annealing were executed as software-based solvers on the local platform.

6.3. EXPERIMENTAL DATA AND REPRESENTATIONS

The evaluation uses three datasets selected to cover different data and domain characteristics within the scope of single-relation selection queries. The datasets include controlled synthetic Bayesian-network data, real health-survey data, and synthetic binary transaction data. This selection allows the experiments to test AnnealBN-CE under different variable types, state-space sizes, dependency patterns, and data generation conditions.

For the experiments, each dataset is kept in two aligned representations: a CSV relation for cardinality estimation and a solver TXT representation for annealing-based structure learning. This distinction is important because the cardinality estimators operate on the tabular relation, whereas the bnslqa solver requires integer-encoded finite-state input.

The connection between the CSV relation and the solver TXT representation is handled through data encoding and state decoding. In this thesis, data encoding denotes the conversion from prepared values in the CSV relation to the integer state identifiers required by the solver TXT representation. State decoding denotes the reverse mapping from solver-side variable indices and state identifiers back to the variable names and values used for cardinality estimation.

Depending on the dataset, the starting point may be either a JSON problem specification or a CSV relation. For synthetic Bayesian-network data, the solver TXT generator receives a JSON problem specification and user-defined generation settings, such as the dataset name, dataset size, and generation mode. The JSON problem specification defines the variables, states, conditional probability tables, and reference adjacency matrix. The generated solver TXT representation contains the number of variables, the number of states for each variable, a problem name, an adjacency vector, and the integer-encoded examples. For datasets with a known Bayesian-network structure, the adjacency vector represents the reference graph. Otherwise, a dummy zero vector is used as a placeholder required by the solver TXT representation; this placeholder is not used as a learned structure for cardinality estimation. A concrete example of the JSON problem specification and the corresponding solver TXT representation is provided in Section A.1.

For tabular datasets, the CSV relation is encoded into the finite-state solver TXT representation. The concrete preparation choices are dataset-specific and may include categorical encoding, discretisation, or binary representation. These details are described in the corresponding experiment setup sections: Sections 6.7.1, 6.8.1, 6.9.1 and 6.9.5.

Table 6.1 summarises the datasets used in the experimental evaluation.

Table 6.1.: Overview of the datasets used in the experimental evaluation.

Dataset	**Data setting**	**Number of rows**	**Main characteristics**
WetGrass BN	Synthetic BN data	100 and 10,000	Controlled binary Bayesian-network data with known variables and dependency structure
NHANES subset	Real health-survey data	2,278	Real tabular data with categorical and discretised health-related attributes
Market Basket	Synthetic transaction data	100 and 10,000	Binary item-presence data over a fixed set of transaction attributes

6.4. QUERY WORKLOAD DESIGN

Each dataset is evaluated with a fixed workload of six single-relation selection queries. The workloads are intentionally kept compact and are designed to test the estimators along two diagnostic dimensions: predicate width and selectivity.

The first dimension, predicate width, refers to the number of variables constrained by a query. Single-predicate queries test marginal selectivities, while multi-predicate queries evaluate how well the estimator captures dependencies between variables. The workloads therefore range from marginal conditions to joint assignments over all modelled variables.

The second dimension, selectivity, is represented by queries with different result sizes. High-cardinality queries evaluate estimator behaviour on common value patterns, while low-cardinality queries evaluate sparse value combinations. Under a ratio-based error measure such as q-error, deviations on low-cardinality queries can have a stronger effect on the reported error. Queries with true cardinality zero are excluded, so the workload remains focused on non-empty selection results.

The query workloads are reported in SQL-style form. This representation is used for the PostgreSQL-based evaluation, while the Bayesian-network-based estimators use the same selection conditions as assignments over the variables of the fitted model. The dataset-specific representation of these assignments is described in the corresponding experiment setup sections.

The results and later observations are tied to these fixed workloads. Other predicate combinations, predicate widths, or selectivity ranges may lead to different estimator behaviour. The reported results should therefore be interpreted as evidence for the selected diagnostic workloads, not as an exhaustive evaluation of all possible single-relation selection queries. The complete SQL-style query lists are provided in Chapter B.

6.5. ANNEALING RUN CONFIGURATIONS

The annealing-based experiments are executed with different read–trial configurations to analyse how solver stochasticity propagates through the AnnealBN-CE workflow from learned structures to cardinality estimates. The configurations are organised around three experiment-level questions. First, does increasing the read budget make the learned structures more stable and improve the accuracy of the resulting cardinality estimates? Second, do additional trials mainly confirm already observed learned structures, or do they reveal further structural diversity? Third, are different learned structures equally effective for cardinality estimation, or do some produce estimates closer to the true cardinalities?

The two solver hyperparameters considered are the number of annealing reads, denoted by R, and the number of trials, denoted by T. A read corresponds to one candidate binary assignment sampled by the annealing solver. A trial is one complete solver run for a fixed dataset, annealing method, and read budget.

The concrete read–trial configurations for simulated annealing (SA) and simulated quantum annealing (SQA) used in the experiments are summarised per dataset in Table 6.2. The table separates the controlled setting from the evaluated configuration in each experiment. The controlled setting denotes the solver hyperparameter that is kept fixed, either R or T. The evaluated configuration lists the values tested for the other solver hyperparameter.

For the synthetically generated datasets, two relation sizes are evaluated. The smaller generated relations are included because, with fewer tuples, some state combinations may occur only rarely or not appear at all. This can make the learned structures more sensitive to the generated sample and is therefore relevant for the stability and diversity questions above. The larger $N = 10{,}000$ relations are motivated by the experimental setup of Zardini et al., where no improvement from larger dataset sizes was found in their performance comparison [36]. In this thesis, the larger generated relations are therefore used to test whether structures learned at this scale also lead to stable and accurate cardinality estimates.

The dataset-specific motivation for the selected configurations is discussed in the corresponding experiment setup sections: Sections 6.7.1, 6.8.1, 6.9.1 and 6.9.5. The reported observations are therefore tied to these configurations and workloads; other read–trial configurations may lead to different learned structures or different cardinality-estimation behaviour.

Table 6.2.: Annealing run configurations used in the experiments.

Dataset	**Dataset size**	**Controlled setting**	**Evaluated configuration**
WetGrass	$N = 100$	$T = 30$	$R \in \{10, 50, 100, 250, 500\}$
WetGrass	$N = 100$	$R = 100$	$T \in \{5, 10, 20, 50\}$
WetGrass	$N = 10{,}000$	$T = 20$	$R \in \{100, 500, 1000, 5000, 10000\}$
WetGrass	$N = 10{,}000$	$R = 1000$	$T \in \{1, 5, 10, 20, 50\}$
Market Basket	$N = 100$	$T = 30$	$R \in \{10, 50, 100, 250, 500\}$
Market Basket	$N = 100$	$R = 100$	$T \in \{5, 10, 20, 50\}$
Market Basket	$N = 10{,}000$	$T = 20$	$R \in \{100, 500, 1000, 5000, 10000\}$
Market Basket	$N = 10{,}000$	$R = 1000$	$T \in \{1, 5, 10, 20, 50\}$
NHANES	$N = 2{,}278$	$T = 10$	$R \in \{100, 500, 1000, 2500\}$
NHANES	$N = 2{,}278$	$R = 1000$	$T \in \{5, 10, 20\}$

6.6. EVALUATION PROCEDURE

This section describes two evaluation components used to analyse the AnnealBN-CE framework. The first component is the q-error evaluation, which measures the multiplicative deviation between estimated and true cardinalities. It is used to assess whether the Bayesian-network structures learned by the annealing methods lead to accurate cardinality estimates. The second component is a random-structure robustness check, which compares the annealing-learned structures with randomly generated admissible Bayesian-network structures. Since different read–trial configurations can lead to different learned structures, these two components assess both the accuracy of the resulting estimates and

whether this accuracy can be attributed to annealing-based structure learning rather than random structure selection.

Annealing q-error evaluation. Unlike PostgreSQL and the Chow–Liu baseline, an annealing read–trial configuration may yield one or several unique learned structures. After each unique structure has been fitted and evaluated on the fixed query workload, the q-error metric defined in Section 2.2 is computed for every query–structure pair. When several unique structures are produced, an additional aggregation step is applied to summarise the behaviour of the annealing method under the corresponding read–trial configuration.

For each query, the median q-error over the resulting unique learned structures is used as the main summary value. The median is chosen because these structures may differ in estimation quality, and individual poor structures can distort the arithmetic mean. The median therefore better represents the typical q-error observed for the query, while the mean q-error is retained as an additional diagnostic for sensitivity to outlier structures. Across the full query workload, minimum, median, and maximum q-errors are reported over these query-level summaries to describe the best, typical, and worst estimation behaviour.

Random-structure robustness check. The q-error evaluation shows how accurate the cardinality estimates produced by the annealing-learned structures are, but it does not show by itself whether this estimation accuracy is specific to the structure-learning step. Therefore, a random-structure robustness check is used as a negative control. It tests whether randomly generated admissible Bayesian-network structures can produce comparable cardinality estimates.

For this check, random acyclic adjacency matrices are generated for the same number of variables and under the same maximum-parent restriction as the annealing-learned structures. The random structures are generated by first sampling a node order and then selecting, for each node, up to the allowed number of parents from preceding nodes in that order. This guarantees acyclicity while respecting the maximum in-degree constraint. The generated matrices are then deduplicated, and each remaining unique matrix is evaluated with the same cardinality-estimation routine. Thus, the parameters are fitted on the same tabular relation, the same query workload is evaluated, and q-errors are computed against the same true cardinalities. The only intended difference is the origin of the structure: learned by the annealing method or generated randomly.

If the random structures achieve q-errors similar to the annealing-learned structures, then this estimation accuracy cannot be attributed confidently to the annealing-based structure-learning step. Conversely, if the annealing-learned structures achieve lower median and maximum q-errors than the random controls, this supports the interpretation that the solver identifies structures that

are useful for the cardinality-estimation task rather than structures whose performance could plausibly be reproduced by random generation.

6.7. EXPERIMENT: CONTROLLED SYNTHETIC BAYESIAN-NETWORK DATA

The WetGrass dataset follows the Bayesian-network example from *Principles of Data Integration* [5] and serves as a controlled synthetic benchmark in this thesis. The data are generated from the JSON-specified Bayesian network shown in Listing A.1. The network contains four binary categorical variables, `Cloud`, `Sprinkler`, `Rain`, and `WetGrass`. These variables become the four attributes of the generated tabular relation used for cardinality estimation. Since the underlying Bayesian network is specified in advance, WetGrass provides a controlled setting for analysing whether the annealing-based structure-learning procedure can recover the relevant dependency structure and whether the learned structures lead to accurate cardinality estimates.

The experiments use two generated relation sizes, $N = 100$ and $N = 10{,}000$. The smaller relation is used to analyse the behaviour of the estimators under sparse frequency counts, where individual tuple counts can visibly affect the estimated conditional probabilities. The larger relation is used to test whether the same generating distribution leads to more stable cardinality estimates when substantially more tuples are available.

The data generator supports two generation modes. In the expected mode, all complete state configurations are enumerated. For each configuration, its probability is computed from the conditional probability tables of the generating Bayesian network. The corresponding tuple count is then obtained by multiplying this probability by the requested relation size and converting the result to an integer count. This mode is treated as the variance-zero setting, because the generated relation follows the theoretical distribution as directly as possible for the chosen size. The expected mode is only approximately variance-zero, since tuple counts must be integer-valued. Therefore, the final number of generated tuples can differ slightly from the requested size, and low-probability configurations may appear only rarely or not at all.

In the stochastic mode, tuples are sampled one by one from the same conditional probability tables. This produces the requested number of tuples, but the empirical frequencies can deviate from the theoretical probabilities because of sampling noise. The stochastic mode is therefore treated as the nonzero-variance setting. Following prior work on annealing-based Bayesian network structure learning, where performance-related analyses were restricted to expected datasets after preliminary tests found negligible differences between expected and

non-expected datasets [36], the experiments in this thesis use the expected WetGrass relations.

The reference adjacency vector is derived from the JSON-specified WetGrass network and stored as the known solution for diagnostic comparison with the structures returned by the annealing methods.

No additional preprocessing is required for WetGrass. All variables are already binary and categorical, and the generated relations contain no missing values. The same binary categorical representation is used for the tabular relation and for the integer-encoded solver input.

The WetGrass query workload contains six positive-cardinality selection predicates over the four variables. The workload includes single-variable predicates and conjunctive predicates over multiple variables, allowing the evaluation to test both marginal and dependency-sensitive cardinality estimates. The full SQL query workload is reported in Section B.1.

The WetGrass evaluation is split into two experiment blocks, one for each relation size. The first block evaluates the smaller relation with $N = 100$, while the second evaluates the larger relation with $N = 10{,}000$. The concrete read–trial settings and their motivation are reported separately in the setup of each relation-size experiment.

6.7.1. EXPERIMENT SETUP: WETGRASS WITH $N = 100$

This experiment evaluates AnnealBN-CE on the smaller expected WetGrass relation with $N = 100$. This relation size is used to analyse the behaviour of the estimators under sparse frequency counts. Since the relation contains only 100 tuples, low-probability state combinations may occur only rarely or may be absent after integer rounding in the expected data generator. As a result, different learned structures can have a visible effect on the estimated probabilities and on the resulting query-level cardinality estimates.

For $N = 100$, the annealing budget is chosen to test solver sensitivity under sparse frequency counts. The read-variation experiment fixes $T = 30$ and varies $R \in \{10, 50, 100, 250, 500\}$, testing whether additional reads stabilise the learned structures and their downstream cardinality estimates. The trial-variation experiment fixes $R = 100$ and varies $T \in \{5, 10, 20, 50\}$, testing whether additional independent solver runs reveal further structural diversity and whether this diversity changes the resulting cardinality estimates.

The same WetGrass query workload introduced in Section 6.7 is used for all configurations. The full SQL query workload is reported in Section B.1. Since the underlying Bayesian network is known, the learned structures can also be compared diagnostically with the reference adjacency vector derived from the JSON specification.

Both configuration groups are executed for SA and SQA using the same generated tabular relation, solver representation, query workload, and parameter-fitting procedure. The resulting adjacency matrices are deduplicated before cardinality estimation, so the reported cardinality values correspond to distinct learned structures.

6.7.2. EXPERIMENT RESULTS: WETGRASS WITH $N = 100$

The WetGrass experiment with $N = 100$ evaluates the six fixed WetGrass workload queries. The true cardinality vector for these queries is

$$(18,\ 21,\ 39,\ 41,\ 2,\ 9) \tag{6.1}$$

The reference adjacency matrix of the generating WetGrass Bayesian network is given as

$$A_{\text{ref}} = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \tag{6.2}$$

The non-reference matrix that appears repeatedly across the stable configurations is

$$A_2 = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \tag{6.3}$$

Across the stable configurations, both A_{ref} and A_2 produce the same cardinality-estimation vector:

$$(17.46,\ 21.36,\ 39,\ 41.6,\ 2.4,\ 8.64) \tag{6.4}$$

This vector is almost identical to the true cardinalities in Eq. (6.1). For the trial-variation setting with fixed $R = 100$, SA returns two unique matrices at $T = 5$: the reference matrix A_{ref} and the non-reference matrix A_2. Both matrices return the stable cardinality vector in Eq. (6.4). For $T \in \{10, 20, 50\}$, SA returns only A_2, again with the same cardinality estimates. SQA shows the same downstream behaviour. At $T = 5$ and $T = 10$, it returns only A_2, while at $T = 20$ and $T = 50$, it returns both A_{ref} and A_2. In all cases, the resulting cardinality estimates remain the stable vector in Eq. (6.4).

For the read-variation setting with fixed $T = 30$, SA is already stable at $R = 50$. At this configuration, three unique matrices are returned, including the reference matrix, and all three produce the stable cardinality vector. At $R = 100$ and $R = 250$, SA returns two unique matrices, A_{ref} and A_2, with the

same cardinality estimates. At $R = 500$, only A_2 remains. The only unstable SA read configuration is $R = 10$, where eight unique matrices produce three distinct cardinality-estimation vectors.

SQA shows slightly more variation at low read budgets. At $R = 10$, eight unique matrices produce four distinct cardinality-estimation vectors. At $R = 50$, four unique matrices are returned; three produce the stable vector in Eq. (6.4), while one produces the alternative vector

$$(20.862,\ 16.824,\ 39,\ 37.82,\ 3.66,\ 13.176). \tag{6.5}$$

From $R = 100$ onward, SQA becomes stable at the cardinality level. At $R = 100$, it returns A_{ref} and A_2, while at $R = 250$ and $R = 500$, it returns only A_2. All these configurations produce the stable cardinality vector in Eq. (6.4).

The resulting query-level cardinality estimates are reported in Table 6.3. For SA and SQA, the table reports the stable cardinality vector obtained across the stable configurations. The classical Chow–Liu baseline and PostgreSQL are included as comparison estimators for the same WetGrass query workload.

Table 6.3.: Cardinality estimates for the WetGrass query workload with $N = 100$. SA and SQA report the stable cardinality vector, alongside the true cardinalities, the classical Chow–Liu baseline, and PostgreSQL.

Query	True	SA	SQA	Classical baseline	PostgreSQL
Q1	18	17.46	17.46	18.00	18
Q2	21	21.36	21.36	18.63	25
Q3	39	39.00	39.00	39.00	32
Q4	41	41.60	41.60	32.51	38
Q5	2	2.40	2.40	3.15	11
Q6	9	8.64	8.64	11.37	1

The q-error summary in Table 6.4 shows the same pattern at the metric level. The stable SA and SQA estimates have the lowest median q-error and the lowest maximum q-error among the compared estimators.

Table 6.4.: Q-error summary for the WetGrass query workload with $N = 100$. For SA and SQA, the summary is computed from the stable cardinality vector in Eq. (6.4).

Estimator	Min q-error	Median q-error	Max q-error	Exact queries
SA stable	1.000	1.024	1.200	16.7%
SQA stable	1.000	1.024	1.200	16.7%
Classical baseline	1.000	1.194	1.575	33.3%
PostgreSQL	1.000	1.205	9.000	16.7%

Overall, the $N = 100$ WetGrass results show that SA and SQA produce stable and low-error cardinality estimates once the read budget is sufficiently large, even when the learned structures are not always identical to the reference WetGrass structure.

6.7.3. DISCUSSION: WETGRASS WITH $N = 100$

The $N = 100$ WetGrass experiment shows that recovering the reference Bayesian-network structure is not sufficient by itself to obtain exact cardinality estimates. Several configurations return the reference adjacency matrix A_{ref}, but this matrix produces the same cardinality vector as the non-reference matrix A_2. The resulting estimates are close to the true cardinalities, but they are not exact for the full workload.

This observation is important because it separates two sources of error: structure learning and parameter estimation. In the $N = 100$ relation, the small number of tuples means that the fitted conditional probabilities are affected by sparse frequency counts and integer-valued data generation. As a result, even the reference graph does not necessarily reproduce the exact query cardinalities after parameter fitting. For AnnealBN-CE, this confirms that graph recovery alone is not the final objective; the relevant criterion is the cardinality behaviour after both structure learning and parameter fitting.

The read–trial behaviour addresses **Research Question 2**. At very low read budgets, especially $R = 10$, both annealing methods show graph-level and cardinality-level variation. SA produces three distinct cardinality behaviours, while SQA produces four. However, this variation disappears quickly as the read budget increases. From $R = 50$ onward for SA and from $R = 100$ onward for SQA, all returned structures in each configuration lead to the same stable cardinality vector. The trial-variation setting is even more stable: increasing the number of trials changes which structures are observed, but it does not change the downstream cardinality estimates.

The comparison between A_{ref} and A_2 also shows that different structures can be equivalent for the selected query workload. Whenever both matrices appear in the same configuration, they return the same cardinality-estimation vector. Thus, structural equality with the generating network is not required for the best observed downstream behaviour, and structural diversity does not automatically imply estimation instability.

The comparison with PostgreSQL and the classical Chow–Liu baseline addresses **Research Question 3**. The classical baseline estimates more individual queries exactly than the stable annealing pattern, but it has larger errors on several conjunctive predicates. PostgreSQL estimates Q1 exactly, but shows large errors on the low-cardinality predicates Q5 and Q6. In contrast, the stable SA and SQA estimates have the lowest median and maximum q-error. For this small WetGrass relation, the annealing-learned Bayesian networks therefore provide the most reliable overall cardinality estimates, although exact query-level cardinalities are not achieved across the full workload.

Overall, the $N = 100$ WetGrass experiment shows that solver stochasticity is most visible at low read budgets, while the downstream estimates become stable once sufficient reads are used. It also shows that, in a small-data setting, parameter estimation can limit cardinality accuracy even when the reference structure is recovered.

6.7.4. EXPERIMENT SETUP: WETGRASS WITH $N = 10{,}000$

This experiment evaluates AnnealBN-CE on the larger expected WetGrass relation with $N = 10{,}000$. This relation size is used to test whether the same Bayesian-network distribution leads to stable cardinality estimates when substantially more tuples are available. In contrast to the $N = 100$ setting, the larger relation reduces the effect of sparse frequency counts and provides a more stable empirical representation of the same underlying dependency structure.

For $N = 10{,}000$, the annealing budget is chosen to test whether the stability observed on the smaller relation persists when the same dependency pattern is represented by substantially more tuples. The read-variation experiment fixes $T = 20$ and varies $R \in \{100, 500, 1000, 5000, 10000\}$, testing whether additional reads reduce structural diversity and stabilise the resulting cardinality estimates. The trial-variation experiment fixes $R = 1000$ and varies $T \in \{1, 5, 10, 20, 50\}$, testing whether additional independent solver runs reveal further structural diversity and whether this diversity changes the downstream cardinality estimates.

The same WetGrass query workload and the same reference Bayesian-network structure are used as in the $N = 100$ experiment. This keeps the two WetGrass relation sizes directly comparable: only the generated relation size and the annealing read–trial configurations differ.

Both configuration groups are executed for SA and SQA using the same generated tabular relation, solver representation, query workload, and parameter-fitting procedure. As in the smaller WetGrass experiment, duplicate adjacency matrices are removed before cardinality estimation, so the reported cardinality values correspond to unique learned structures.

6.7.5. EXPERIMENT RESULTS: WETGRASS WITH $N = 10{,}000$

The WetGrass experiment with $N = 10{,}000$ produces exact cardinality estimates for both annealing methods across all evaluated read–trial configurations. The true cardinality vector for the six WetGrass workload queries is

$$(1{,}746,\ 2{,}136,\ 3{,}900,\ 4{,}160,\ 240,\ 864). \tag{6.6}$$

Both SA and SQA return this exact vector for every reported configuration.

For the read-variation setting with fixed $T = 20$, the first read budget $R = 100$ returns two unique adjacency matrices for both SA and SQA. The reference adjacency matrix of the generating WetGrass Bayesian network is

$$A_{\text{ref}} = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \tag{6.7}$$

The two unique matrices returned at $R = 100, T = 20$ are

$$A_1 = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \qquad A_2 = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}. \tag{6.8}$$

Neither A_1 nor A_2 is equal to A_{ref}. Nevertheless, both learned matrices produce the exact true cardinalities for all six workload queries.

The solver diagnostics also show the same reported energy for the reference structure and for the two learned structures. The reference WetGrass structure has reported energy $\text{expY} = -46{,}473.92187$. For SA at $R = 100, T = 20$, both learned matrices have reported energy $\text{minY} = -46{,}473.92187$. For SQA at the same read–trial configuration, both learned matrices have reported energy $\text{minY} = -46{,}473.921875$. Thus, up to the reported precision, the reference structure and the two learned structures have the same objective value.

For all larger read budgets in the fixed $T = 20$ read-variation setting, namely $R \in \{500, 1000, 5000, 10000\}$, both SA and SQA return only one unique adjacency matrix, A_2, and this matrix again produces the exact true cardinality

vector in Eq. (6.6). The trial-variation setting shows the same downstream behaviour. With fixed $R = 1000$ and $T \in \{1, 5, 10, 20, 50\}$, both SA and SQA return only the unique matrix A_2, and all configurations produce exact cardinality estimates for all six queries.

The resulting query-level cardinality estimates are reported in Table 6.5. For SA and SQA, the table reports the common exact cardinality vector obtained across the evaluated configurations. The Chow–Liu baseline and PostgreSQL are included as comparison estimators for the same WetGrass query workload.

Table 6.5.: Cardinality estimates for the WetGrass query workload with $N = 10{,}000$. The table compares the true cardinalities with the estimates obtained from SA, SQA, the Chow–Liu baseline, and PostgreSQL.

Query	True	SA	SQA	Classical baseline	PostgreSQL
Q1	1,746	1,746	1,746	1,746.00	1,746
Q2	2,136	2,136	2,136	1,897.26	2,476
Q3	3,900	3,900	3,900	3,900.00	3,219
Q4	4,160	4,160	4,160	3,270.51	3,782
Q5	240	240	240	316.50	1,135
Q6	864	864	864	1,102.74	121

The q-error summary in Table 6.6 confirms the cardinality-level behaviour. SA and SQA achieve q-error 1.000 for every query. The classical baseline estimates Q1 and Q3 exactly, but deviates on the conjunctive predicates Q2, Q4, Q5, and Q6. PostgreSQL estimates Q1 exactly, but shows larger errors on the dependency-sensitive and low-cardinality predicates, especially Q5 and Q6.

Overall, the $N = 10{,}000$ WetGrass results show exact query-level cardinality estimation for SA and SQA, even though the learned structures do not match the reference adjacency matrix.

Table 6.6.: Q-error summary for the WetGrass query workload with $N = 10{,}000$.

Estimator	Min q-error	Median q-error	Max q-error	Exact queries
SA	1.000	1.000	1.000	100.0%
SQA	1.000	1.000	1.000	100.0%
Classical baseline	1.000	1.199	1.319	33.3%
PostgreSQL	1.000	1.185	7.140	16.7%

6.7.6. DISCUSSION: WETGRASS WITH $N = 10{,}000$

The $N = 10{,}000$ WetGrass experiment highlights an important distinction between recovering the generating Bayesian-network structure and obtaining accurate cardinality estimates from a learned structure. Although the reference adjacency matrix is known in this controlled setting, neither SA nor SQA returns this exact structure in the evaluated configurations. Nevertheless, the learned structures are sufficient for exact query-level cardinality estimation on the six WetGrass workload predicates.

This result shows that exact graph recovery is not required for the objective studied in this thesis. The learned matrices differ from the reference WetGrass structure, but after parameter fitting they still represent the probabilities needed by the selected workload. For AnnealBN-CE, this supports the choice to evaluate learned Bayesian networks through their downstream cardinality estimates rather than through structural equality alone.

The solver diagnostics provide further evidence for this interpretation. The reference structure and the two learned matrices observed at $R = 100, T = 20$ have the same reported objective value up to the displayed precision. This suggests that the optimisation objective admits several equivalent structures for this dataset and encoding. Therefore, the structure-learning problem is not necessarily identifiable at the graph level, even when the resulting structures are equally useful for cardinality estimation.

The read–trial behaviour addresses **Research Question 2**. At the lowest read budget, SA and SQA return two different non-reference structures, but this graph-level variation does not lead to different cardinality estimates. From $R = 500$ onward, both methods return only one unique structure, A_2, and the trial-variation setting shows the same stable behaviour. Thus, in this experiment, solver stochasticity can affect the learned graph at low read budgets, but it does not propagate into cardinality-level instability.

The comparison with PostgreSQL and the classical Chow–Liu baseline addresses **Research Question 3**. Both baselines estimate some queries exactly, but they deviate on dependency-sensitive predicates. PostgreSQL shows especially large errors for Q5 and Q6, while the classical baseline remains closer but still deviates on several conjunctive queries. In contrast, SA and SQA are exact for the full workload. For this controlled WetGrass setting, the annealing-learned Bayesian networks therefore provide the most accurate cardinality estimates, even without recovering the exact generating structure.

6.8. REAL HEALTH-SURVEY DATA

This experiment evaluates AnnealBN-CE on a real health-survey relation prepared from the National Health and Nutrition Examination Survey (NHANES)

age prediction subset from the UCI Machine Learning Repository [21]. Because no ground-truth Bayesian-network structure is available for this relation, the experiment focuses on the stability of the learned structures and on the resulting cardinality-estimation accuracy.

6.8.1. EXPERIMENT SETUP

The prepared NHANES relation provides a real health-related tabular dataset with three categorical attributes and one numerical health-related attribute. It is therefore used as the real-data setting in the single-relation experiments.

The local file used in the experiments contains $2{,}278$ tuples and no missing values. From the original dataset, four attributes are retained: `age_group`, `BMXBMI`, `RIAGENDR`, and `DIQ010`. This reduction is necessary because the QUBO formulation grows with the number of Bayesian-network variables and states. The selected attributes keep the structure-learning problem tractable while preserving both demographic and health-related information.

Before generating the solver representation, the numerical attribute `BMXBMI` is discretised using BMI thresholds based on WHO adult BMI indicators [33]. The WHO indicators define underweight as $\mathrm{BMI} < 18.5$, overweight as $\mathrm{BMI} \geq 25$, and obesity as $\mathrm{BMI} \geq 30$. Based on these thresholds, `BMXBMI` is encoded into four states, so that the exact BMI value is replaced by its interval membership:

$$0 : \mathrm{BMI} < 18.5,\ 1 : 18.5 \leq \mathrm{BMI} < 25,\ 2 : 25 \leq \mathrm{BMI} < 30,\ 3 : \mathrm{BMI} \geq 30.$$

The remaining attributes are categorical and are encoded as integer states for the solver representation. The attribute `age_group` distinguishes between `Adult` participants younger than 65 years and `Senior` participants aged 65 years or older. These two values are encoded as the states 0 and 1, respectively. The gender attribute `RIAGENDR` contains the two values 1.0 and 2.0, encoded as the states 0 and 1. The diabetes-indicator attribute `DIQ010` contains the three values 1.0, 2.0, and 3.0, encoded as the states 0, 1, and 2.

Table 6.7.: Prepared NHANES attributes and solver encodings.

Attribute	**Prepared states and solver encoding**
`age_group`	`Adult`, `Senior` $\rightarrow 0, 1$
`BMXBMI`	BMI bins: < 18.5, $[18.5, 25)$, $[25, 30)$, $\geq 30 \rightarrow 0, 1, 2, 3$
`RIAGENDR`	$1.0, 2.0 \rightarrow 0, 1$
`DIQ010`	$1.0, 2.0, 3.0 \rightarrow 0, 1, 2$

Since no ground-truth Bayesian network is available for this dataset, the solver representation uses a dummy zero adjacency vector in the header of the solver file.

After preparing the solver representation, the experiment uses a fixed query workload to evaluate cardinality-estimation accuracy. The workload contains six positive-cardinality selection predicates, denoted by Q1–Q6. It ranges from a single-attribute predicate to selective conjunctive predicates over the selected NHANES attributes. The full SQL predicates are listed in Section B.2.

For NHANES, the annealing budget is chosen to test how sensitive the cardinality-estimation accuracy of AnnealBN-CE is to solver effort. The budget choice is guided by the compact encoded structure-learning space: four selected attributes, state counts $2, 4, 2, 3$, and maximum in-degree two. This motivates a coarse sensitivity grid rather than a dense parameter sweep: the experiment checks whether increasing the number of reads R or the number of trials T changes the learned structures enough to change the resulting q-errors.

The read sweep fixes $T = 10$ and evaluates $R \in \{100, 500, 1000, 2500\}$, forming a coarse read range from a low to a high setting. The trial sweep fixes $R = 1000$, one of the intermediate read settings, and evaluates $T \in \{5, 10, 20\}$. Together, the two sweeps distinguish the effect of sampling more candidate assignments within one solver run from the effect of repeating complete stochastic solver runs.

6.8.2. EXPERIMENT RESULTS

Across all NHANES read–trial configurations, SA and SQA each returned a single unique learned structure, and the corresponding query-level cardinality estimates were identical. Table 6.8 therefore reports the estimates from the lowest tested total sampling budget, $R = 100$ and $T = 10$, as representative for SA and SQA. The cardinality estimates are then summarised with q-error in Table 6.9 to compare the estimators at the metric level.

Table 6.8.: True cardinalities and estimator outputs for the NHANES query workload.

Query	True	SA	SQA	Classical baseline	PostgreSQL
Q1	2,199	2,199	2,199	2,199	2,199
Q2	9	9	9	9	7
Q3	4	4	4	3.43	1
Q4	675	675	675	657.57	635
Q5	8	8	8	6.81	5
Q6	75	75.6	75.6	58.78	53

The annealing methods show near-exact estimates across the workload. The larger classical Chow–Liu baseline and PostgreSQL deviations occur on selective conjunctive predicates over demographic and health-related attributes. The

largest deviation is PostgreSQL's estimate for Q3, the senior-obese-diabetes predicate, where the true cardinality 4 is underestimated as 1.

Table 6.9.: Q-error summary for the NHANES query workload.

Estimator	Minimum q-error	Median q-error	Maximum q-error
SA	1.000	1.000	1.008
SQA	1.000	1.000	1.008
Classical baseline	1.000	1.096	1.276
PostgreSQL	1.000	1.350	4.000

The q-error summary confirms the same pattern. SA and SQA achieve a perfect median q-error of 1.000, with a maximum q-error of 1.008. The classical baseline remains close overall, while PostgreSQL shows the widest error range, reaching a maximum q-error of 4.000.

6.8.3. DISCUSSION

The NHANES experiment provides a compact real-data case in which the annealing-based estimators behave consistently. The encoded structure-learning problem has only four Bayesian-network variables and maximum in-degree two, which keeps the QUBO-based BNSL instance in the range where the formulation is expected to be manageable [36]. Across the tested read–trial configurations, increasing R or T did not change the cardinality estimates. This observation is important for **Research Question 2**: in this dataset, solver stochasticity does not propagate into downstream cardinality variability.

The comparison between SA and SQA is therefore neutral for NHANES. Both methods produce the same cardinality behaviour and the same q-error summary. The result does not indicate an advantage of SQA over SA in this compact setting. Instead, the encoded problem appears simple enough for both annealing methods to find structures that are equally useful for the selected workload.

This result addresses **Research Question 3**. As expected under PostgreSQL's independence-based selectivity assumptions, its errors increase on selective conjunctive predicates involving dependencies between multiple attributes. The classical Chow–Liu baseline reduces these errors, which indicates that modelling dependencies with a Bayesian network is useful. However, its tree restriction still limits the dependencies that can be represented simultaneously. The annealing-learned structures allow up to two parents per node, giving them more flexibility to represent dependency patterns among queried attributes.

The encoding choice is also relevant. Discretising `BMXBMI` into BMI categories removes exact numeric BMI values, but it preserves the distinctions used in the query workload and reduces the state space seen by the solver. In this case, the

representation keeps the information needed for accurate cardinality estimation while making the structure-learning problem stable across solver configurations.

6.9. SYNTHETIC BINARY TRANSACTION DATA

The *Synthetic Market Basket Transactions* dataset is based on the standard market-basket example from *Introduction to Data Mining*, where transactions are represented as binary item vectors [28]. It is used here as a binary co-occurrence dataset for cardinality estimation. Each tuple represents one transaction, and each variable indicates whether a specific item occurs in that transaction. This makes the dataset suitable for evaluating how the cardinality estimators handle item co-occurrence patterns.

The relation contains six item variables: `Beer`, `Bread`, `Cola`, `Diapers`, `Eggs`, and `Milk`. Each variable is binary. A value of 1 denotes that the item is present in the transaction, while a value of 0 denotes that it is absent.

The dataset is generated synthetically from the five base transaction templates listed in Section A.2. Each template represents a typical item combination as a binary vector over the six items. Larger datasets are obtained by sampling these templates with replacement. This preserves the template-based item co-occurrence patterns while allowing datasets of different sizes to be created in a controlled way.

The experiments use two generated relation sizes, $N = 100$ and $N = 10{,}000$. The smaller relation is used to analyse the behaviour of the estimators under sparse frequency counts, where individual learned structures can have a visible effect on query-level cardinality estimates. The larger relation is used to test whether the same template-based dependencies lead to more stable estimates when substantially more tuples are available. The two sizes therefore separate a small-data stability setting from a larger-data stability setting.

Since all variables are already binary, no discretisation or additional categorical encoding is required. The same binary values are used in the tabular relation for cardinality estimation and in the integer-encoded solver representation.

The query workload contains six positive-cardinality selection predicates over the binary item variables. It includes a single-item predicate, two-item co-occurrence predicates, a three-item co-occurrence predicate, and a selective predicate over all six variables. The full workload is listed in Section B.3.

Since no ground-truth Bayesian network is available for this dataset, the solver representation uses a dummy zero adjacency vector in the header of the solver file.

The Market Basket evaluation is split into two experiment blocks, one for each relation size. The first block evaluates the smaller relation with $N = 100$, while the second evaluates the larger relation with $N = 10{,}000$. The concrete

read–trial settings and their motivation are reported separately in the setup of each relation-size experiment.

6.9.1. EXPERIMENT SETUP: MARKET BASKET WITH $N = 100$

This experiment evaluates AnnealBN-CE on the smaller synthetic Market Basket relation with $N = 100$ tuples. This relation size is used to analyse the behaviour of the estimators under sparse frequency counts, where individual learned structures can have a visible effect on query-level cardinality estimates.

For $N = 100$, the annealing budget is chosen to test solver sensitivity under sparse frequency counts. Since the relation contains only 100 tuples, different learned structures may affect the estimated probabilities of multi-item predicates more visibly. The read-variation experiment fixes $T = 30$ and varies $R \in \{10, 50, 100, 250, 500\}$, testing whether additional reads stabilise the learned structures and their downstream cardinality estimates. The trial-variation experiment fixes $R = 100$ and varies $T \in \{5, 10, 20, 50\}$, testing whether additional independent solver runs reveal further structural diversity and whether this diversity changes the resulting cardinality estimates.

Both configuration groups are executed for SA and SQA using the same binary data representation, query workload, and parameter-fitting procedure. The resulting unique adjacency matrices are deduplicated before cardinality estimation, so these reported values correspond to distinct learned structures.

6.9.2. EXPERIMENT RESULTS: MARKET BASKET WITH $N = 100$

The results for this experiment are organised around three complementary perspectives. First, the estimated cardinalities of SA, SQA, the classical Chow–Liu baseline, and PostgreSQL are compared with the true cardinalities for each query. Second, the same estimates are evaluated using the q-error metric introduced in Section 2.2. Third, the read-variation and trial-variation results are used to characterise the sensitivity of SA and SQA to their read–trial configurations, focusing on learned-structure diversity and the resulting distinct estimates.

Cardinality-estimate comparison. Table 6.10 reports the cardinality estimates obtained with SA, SQA, the classical Chow–Liu baseline, and PostgreSQL for the six workload predicates. For the two annealing methods, the table reports the *stable cardinality pattern.* In this subsection, stable refers to the downstream cardinality vector that remains when the read-variation configurations no longer produce multiple distinct estimate patterns. This term therefore denotes consistency across configurations, not necessarily the most accurate estimate for every individual query.

For both SA and SQA, the stable cardinality pattern is

$$(81.39,\ 63,\ 58.39,\ 21,\ 39.78,\ 18.61). \tag{6.9}$$

This vector is the only cardinality pattern observed for SA from $R = 250$ onward and for SQA from $R = 100$ onward in the read-variation experiment. The SA and SQA columns are kept separate to make the comparison between the two annealing methods explicit, although their stable estimates are identical for this workload.

Table 6.10.: Cardinality estimates for the Market Basket query workload with $N = 100$. SA and SQA report the stable cardinality pattern, alongside the true cardinalities, the classical Chow–Liu baseline, and PostgreSQL.

Query	True	SA	SQA	Classical baseline	PostgreSQL
Q1	84	81.39	81.39	84.00	84
Q2	63	63.00	63.00	63.00	52
Q3	61	58.39	58.39	61.00	65
Q4	21	21.00	21.00	21.00	4
Q5	45	39.78	39.78	48.65	54
Q6	16	18.61	18.61	8.51	2

Q-error comparison. The second perspective evaluates the estimates from Table 6.10 using the q-error metric introduced in Section 2.2. For each estimator, Table 6.11 reports the minimum, median, and maximum q-error over the six workload queries. The final column gives the percentage of queries for which the estimated cardinality is exactly equal to the true cardinality.

Table 6.11.: Q-error summary for the Market Basket query workload with $N = 100$. For SA and SQA, the summary is computed from the stable cardinality pattern of each annealing method.

Estimator	Min q-error	Median q-error	Max q-error	Exact queries
SA stable	1.000	1.038	1.163	33.3%
SQA stable	1.000	1.038	1.163	33.3%
Classical baseline	1.000	1.000	1.880	66.7%
PostgreSQL	1.000	1.206	8.000	16.7%

To complement the aggregate statistics in Table 6.11, Fig. 6.1 shows the full q-error profile of each estimator over the six workload queries. The q-errors

are sorted independently for each estimator and plotted by query rank, where rank 1 corresponds to the smallest q-error and rank 6 to the largest q-error. This representation abstracts away from the individual query labels and makes the distribution of errors visible, including the worst-case behaviour of each estimator. It is therefore useful for assessing whether an estimator performs consistently across the workload or whether its aggregate statistics are affected by a small number of high-error queries.

The SA and SQA curves overlap because both annealing methods produce the same stable cardinality pattern for this experiment. The sorted plot also shows that the main difference between the estimators appears in the upper tail of the q-error distribution, especially for the multi-predicate queries.

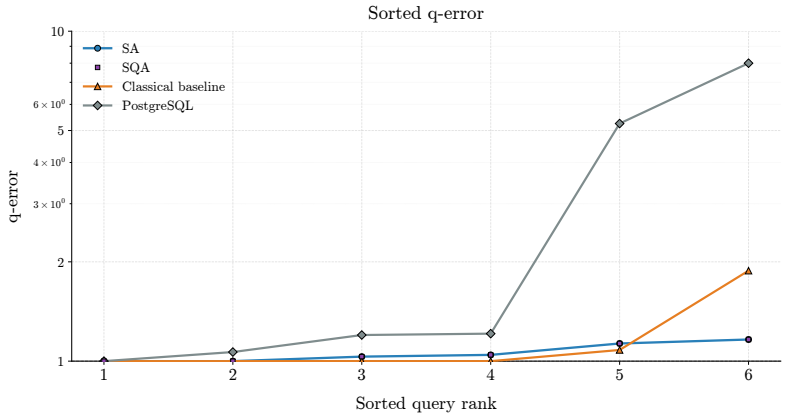

Figure 6.1.: Sorted q-error profiles for the Market Basket query workload with $N = 100$. The SA and SQA profiles are computed from the stable cardinality pattern in Eq. (6.9); the classical baseline and PostgreSQL are included for comparison.

The analysis now turns to the third perspective, namely the **read-variation** and **trial-variation** results.

The first part considers **read variation** by fixing the number of trials to $T = 30$ and varying the read budget as $R \in \{10, 50, 100, 250, 500\}$. This isolates the effect of the read budget on the structures returned by SA and SQA and on the cardinality estimates obtained from those structures.

Table 6.12 reports the resulting structure-level and cardinality-level variation. For each configuration, the table gives the number of unique adjacency matrices after deduplication, the largest number of distinct cardinality estimates observed for any individual query, the median q-error, and the percentage of exact estimates. The median q-error and exact-estimate percentage are computed over all query–structure pairs generated by the unique matrices in the correspond-

ing configuration. The detailed per-query cardinality values are reported in Table B.12 for SA and in Table B.13 for SQA.

Table 6.12.: Read-variation results for the Market Basket query workload with $N = 100$ and fixed $T = 30$. The table reports structural diversity, cardinality-estimation variation, median q-error, and exact-estimate percentages for each read budget.

Method	Reads	Unique matrices	Max distinct per query	Median q-error	Exact estimates
SA	10	27	4	1.000	59.9%
SA	50	20	2	1.032	43.3%
SA	100	17	2	1.032	37.3%
SA	250	11	1	1.038	33.3%
SA	500	12	1	1.038	33.3%
SQA	10	30	8	1.000	66.1%
SQA	50	27	2	1.000	50.6%
SQA	100	18	1	1.038	33.3%
SQA	250	12	1	1.038	33.3%
SQA	500	10	1	1.038	33.3%

Across increasing read budgets, both methods show a reduction in cardinality-level variation. For SA, the number of unique matrices changes from 27 at $R = 10$ to 12 at $R = 500$, while the maximum number of distinct estimates per query decreases from 4 to 1. For SQA, the number of unique matrices changes from 30 to 10, and the maximum number of distinct estimates per query decreases from 8 to 1. Thus, from $R = 250$ onward for SA and from $R = 100$ onward for SQA, the downstream cardinality estimates are stable at the query level, even though more than one unique adjacency matrix may still be present.

The second part considers **trial variation** by fixing the read budget to $R = 100$ and varying the number of trials as $T \in \{5, 10, 20, 50\}$. This configuration isolates how additional independent solver executions affect learned-structure diversity and the resulting cardinality estimates.

Table 6.13 reports the resulting structure-level and cardinality-level variation for SA and SQA. The table uses the same quantities as the read-variation summary: the number of unique adjacency matrices after deduplication, the largest number of distinct cardinality estimates observed for any individual query, the median q-error, and the percentage of exact estimates. The median q-error and exact-estimate percentage are computed over all query–structure pairs generated by the unique matrices in the corresponding configuration. The detailed per-query cardinality values are reported in Table B.14 for SA and in Table B.15 for SQA.

Table 6.13.: Trial-variation results for the Market Basket query workload with $N = 100$ and fixed $R = 100$. The table reports structural diversity, cardinality-estimation variation, median q-error, and exact-estimate percentages for each trial count.

Method	Trials	Unique matrices	Max distinct per query	Median q-error	Exact estimates
SA	5	4	1	1.038	33.3%
SA	10	8	1	1.038	33.3%
SA	20	13	1	1.038	33.3%
SA	50	22	2	1.038	36.4%
SQA	5	5	1	1.038	33.3%
SQA	10	9	1	1.038	33.3%
SQA	20	17	2	1.038	37.3%
SQA	50	28	1	1.038	33.3%

The trial-variation results in Table 6.13 show that increasing the number of trials increases the number of unique matrices for both methods. For SA, the number of unique matrices increases from 4 at $T = 5$ to 22 at $T = 50$. For SQA, it increases from 5 to 28. In contrast, cardinality-level variation remains limited: the maximum number of distinct estimates per query is 1 for three of the four SA configurations and three of the four SQA configurations. The median q-error remains 1.038 in all trial-variation configurations, while the exact-estimate percentage changes only slightly.

6.9.3. RANDOM-STRUCTURE ROBUSTNESS CHECK: MARKET BASKET WITH $N = 100$

The random-structure robustness check evaluates whether the stable annealing results are specific to the learned Bayesian-network structures or could also be obtained from randomly generated admissible structures. The annealing configurations used for this comparison are selected by stability rather than by individual exact estimates. Specifically, the representative configuration is the first read–trial setting for which all unique learned matrices lead to a single downstream cardinality-estimation behaviour. This criterion selects $R = 250, T = 30$ for SA and $R = 100, T = 30$ for SQA.

Both selected annealing configurations produce the stable cardinality pattern reported in Eq. (6.9). As a negative control, 1000 admissible random matrices were generated under the same acyclicity and maximum-parent restrictions. After deduplication, 960 unique random structures remained. All structures were

evaluated with the same parameter-fitting and cardinality-estimation procedure. The comparison is summarised in Table 6.14.

Table 6.14.: Random-structure robustness check for the Market Basket query workload with $N = 100$. The selected SA and SQA configurations are the first stable read–trial settings for each annealing method. The column "Distinct CE vectors" reports the number of distinct cardinality-estimation vectors obtained from the evaluated structures.

Structure source	Structures	Distinct CE vectors	Median q-error	Max q-error	Exact estimates
SA, $R = 250, T = 30$	11	1	1.04	1.16	33.33%
SQA, $R = 100, T = 30$	18	1	1.04	1.16	33.33%
Random structures	960	838	1.14	18.21	25.28%

The random-control structures show substantially higher variability than the selected annealing configurations. They produce 838 distinct cardinality-estimation vectors, whereas the selected SA and SQA configurations each produce only one. Their median q-error is 1.14, their maximum q-error is 18.21, and their exact-estimate rate is 25.28%. In contrast, the stable annealing configurations have a median q-error of 1.04, a maximum q-error of 1.16, and an exact-estimate rate of 33.33%.

The main difference is the worst-case behaviour. Random structures can lead to large q-errors, while the selected annealing structures keep the maximum q-error at 1.16. No random structure achieved both a median q-error and a maximum q-error at least as good as the selected stable annealing configuration. The robustness check therefore supports the interpretation that the annealing methods are not merely selecting random admissible matrices. For this workload, the learned structures are more stable and more reliable for cardinality estimation than the random-control structures.

6.9.4. DISCUSSION: MARKET BASKET WITH $N = 100$

The Market Basket experiment with $N = 100$ provides a small binary co-occurrence case in which structural choices can visibly affect query-level cardinality estimates. The stable SA and SQA pattern is not exact for every query, but it remains close to the true cardinalities and keeps the maximum q-error at 1.163. This is lower than the maximum q-error of both PostgreSQL and the classical Chow–Liu baseline. The result is therefore relevant for **Research Question 3**: the annealing methods do not dominate the classical baseline by median q-error, but they provide the best worst-case behaviour on this workload.

The estimator comparison shows the benefit of dependency modelling. PostgreSQL estimates the single-item query Q1 exactly, but strongly underestimates the selective conjunctive queries Q4 and Q6. The classical Chow–Liu baseline captures several item dependencies well and achieves the best median q-error, but it underestimates Q6, which involves all six item variables. In contrast, the annealing-learned structures are less exact on some simpler queries, but avoid the large error on Q6. This indicates that the additional structural flexibility of the annealing-learned Bayesian networks is most useful for the most selective multi-predicate query.

The read–trial analysis addresses **Research Question 2**. Increasing the read budget reduces cardinality-level variation: from $R = 250$ onward for SA and from $R = 100$ onward for SQA, all unique learned structures produce the same cardinality-estimation behaviour. Increasing the number of trials has a different effect. It increases the number of unique matrices, but the median q-error remains unchanged and the number of distinct cardinality-estimation vectors stays low. Thus, structural diversity does not necessarily imply downstream estimation instability.

The random-structure robustness check is the key evidence that the stable annealing results are not simply the result of arbitrary structure selection. The random-control structures produce 838 distinct cardinality-estimation vectors and reach a maximum q-error of 18.21, whereas the selected stable SA and SQA configurations each produce one vector and keep the maximum q-error at 1.16. No random structure achieves both a median q-error and a maximum q-error at least as good as the selected stable annealing configuration. This supports the interpretation that, for this workload, the annealing methods identify structures that are more reliable for cardinality estimation than random admissible structures.

Overall, the $N = 100$ Market Basket results support the use of AnnealBN-CE on binary transaction data. They show that the framework can produce stable downstream estimates despite solver stochasticity, and that its main advantage in this small-data setting is the reduction of worst-case q-error on selective multi-predicate queries.

6.9.5. Experiment Setup: Market Basket with $N = 10{,}000$

This experiment evaluates AnnealBN-CE on the larger synthetic Market Basket relation with $N = 10{,}000$ tuples. This relation size is used to test whether the template-based item co-occurrence patterns lead to stable cardinality estimates when substantially more tuples are available. In contrast to the $N = 100$ setting, the larger relation reduces the effect of sparse frequency counts and allows the stability of the learned structures to be analysed at a larger sample size.

For $N = 10{,}000$, the annealing budget is chosen to test whether the stability observed on the smaller relation persists when the same dependency patterns are represented by substantially more tuples. The read-variation experiment fixes $T = 20$ and varies $R \in \{100, 500, 1000, 5000, 10000\}$, testing whether additional reads reduce structural diversity and stabilise the resulting cardinality estimates. The trial-variation experiment fixes $R = 1000$ and varies $T \in \{1, 5, 10, 20, 50\}$, testing whether additional independent solver runs reveal further structural diversity and whether this diversity changes the downstream cardinality estimates.

Both configuration groups are executed for SA and SQA using the same binary data representation, query workload, and parameter-fitting procedure. As in the $N = 100$ experiment, the resulting unique adjacency matrices are deduplicated before cardinality estimation, so the reported cardinality values correspond to distinct learned structures.

6.9.6. EXPERIMENT RESULTS: MARKET BASKET WITH $N = 10{,}000$

The results for the larger Market Basket relation are organised in the same way as the $N = 100$ experiment. First, the estimated cardinalities of SA, SQA, the classical Chow–Liu baseline, and PostgreSQL are compared with the true cardinalities for each query. Second, the same estimates are evaluated using the q-error metric introduced in Section 2.2. Third, the read-variation and trial-variation results are used to analyse whether structural diversity still propagates to distinct cardinality estimates when substantially more tuples are available.

Across the evaluated configurations, both annealing methods produce an almost identical downstream cardinality-estimation behaviour. The common stable cardinality pattern is

$$(8{,}055.72,\ 5{,}934,\ 5{,}935.72,\ 2{,}016,\ 3{,}991.44,\ 1{,}944.28). \tag{6.10}$$

This vector is very close to the true cardinalities for all six queries. The only exception occurs for SQA at $R = 100, T = 20$, where one of the 17 unique matrices returns the exact cardinality vector. This alternative behaviour is not observed again for larger read budgets.

Cardinality-estimate comparison. Table 6.15 reports the cardinality estimates obtained with SA, SQA, the classical Chow–Liu baseline, and PostgreSQL for the six workload queries. For SA and SQA, the table reports the stable cardinality pattern in Eq. (6.10). The two annealing methods return the same stable estimates, which are close to the true cardinalities for all six queries.

Table 6.15.: Cardinality estimates for the Market Basket query workload with $N = 10{,}000$. SA and SQA report the stable cardinality pattern, alongside the true cardinalities, the classical Chow–Liu baseline, and PostgreSQL.

Query	True	SA	SQA	Classical baseline	PostgreSQL
Q1	8,057	8,055.72	8,055.72	8,057.00	8,057
Q2	5,934	5,934.00	5,934.00	5,934.00	4,719
Q3	5,937	5,935.72	5,935.72	5,937.00	6,350
Q4	2,016	2,016.00	2,016.00	2,016.00	406
Q5	3,994	3,991.44	3,991.44	4,486.53	5,116
Q6	1,943	1,944.28	1,944.28	957.21	232

Q-error comparison. The second perspective evaluates the estimates from Table 6.15 using q-error. For SA and SQA, the q-error summary is computed from the stable cardinality pattern in Eq. (6.10). The annealing estimates are nearly identical to the true cardinalities, giving both methods a median q-error of 1.0002 and a maximum q-error of 1.0007. The classical baseline estimates four queries exactly, but its maximum q-error increases because it underestimates Q6. PostgreSQL has the largest maximum q-error, mainly due to its underestimation of Q4 and Q6.

Table 6.16.: Q-error summary for the Market Basket query workload with $N = 10{,}000$. For SA and SQA, the summary is computed from the stable cardinality pattern in Eq. (6.10).

Estimator	Min q-error	Median q-error	Max q-error	Exact queries
SA stable	1.0000	1.0002	1.0007	33.3%
SQA stable	1.0000	1.0002	1.0007	33.3%
Classical baseline	1.0000	1.0000	2.0299	66.7%
PostgreSQL	1.0000	1.2692	8.3750	16.7%

The analysis now turns to the third perspective: the **read-variation** and **trial-variation** results.

The first part considers **read variation** by fixing the number of trials to $T = 20$ and varying the read budget as $R \in \{100, 500, 1000, 5000, 10000\}$. This isolates the effect of the read budget on the structures returned by SA and SQA and on the cardinality estimates obtained from those structures.

Table 6.17 reports the resulting structure-level and cardinality-level variation. For each configuration, the table gives the number of unique adjacency matrices

after deduplication, the largest number of distinct cardinality estimates observed for any individual query, the median q-error, and the percentage of exact estimates. The detailed per-query cardinality values are reported in Table B.16 for SA and in Table B.17 for SQA.

Table 6.17.: Read-variation results for the Market Basket query workload with $N = 10{,}000$ and fixed $T = 20$. The table reports the read budget, number of unique adjacency matrices, maximum distinct cardinality estimates per query, median q-error, and exact-estimate percentage for each configuration.

Method	R	**Unique**	**Distinct**	**Median**	**Exact**
SA	100	14	1	1.0002	33.3%
SA	500	12	1	1.0002	33.3%
SA	1000	8	1	1.0002	33.3%
SA	5000	2	1	1.0002	33.3%
SA	10000	1	1	1.0002	33.3%
SQA	100	17	2	1.0002	37.3%
SQA	500	12	1	1.0002	33.3%
SQA	1000	8	1	1.0002	33.3%
SQA	5000	4	1	1.0002	33.3%
SQA	10000	3	1	1.0002	33.3%

The read-variation results show that SA is already stable at the lowest read budget. At $R = 100$, SA produces 14 unique matrices, but all of them return the same cardinality-estimation vector. As R increases, the number of unique matrices falls from 14 to 1, while the cardinality estimates remain unchanged.

SQA shows the same overall pattern, with one minor exception at $R = 100$. At this configuration, 17 unique matrices are obtained and two distinct cardinality-estimation behaviours appear: 16 matrices return the stable pattern in Eq. (6.10), while one matrix returns the exact cardinality vector. From $R = 500$ onward, SQA also returns only the stable cardinality pattern. Thus, increasing the read budget mainly reduces structural diversity, while the downstream estimates remain essentially unchanged.

The second part considers **trial variation** by fixing the read budget to $R = 1000$ and varying the number of trials as $T \in \{1, 5, 10, 20, 50\}$. This isolates the effect of additional independent solver executions on the structures returned by SA and SQA and on the cardinality estimates obtained from those structures.

Table 6.18 reports the corresponding trial-variation results. The table uses the same quantities as the read-variation summary: the number of unique adjacency matrices after deduplication, the largest number of distinct cardinality estimates observed for any individual query, the median q-error, and the percentage

of exact estimates. The detailed per-query cardinality values are reported in Table B.18 for SA and in Table B.19 for SQA.

Table 6.18.: Trial-variation results for the Market Basket query workload with $N = 10{,}000$ and fixed $R = 1000$. The table reports the number of trials, number of unique adjacency matrices, maximum distinct cardinality estimates per query, median q-error, and exact-estimate percentage for each configuration.

Method	T	Unique	Distinct	Median	Exact
SA	1	1	1	1.0002	33.3%
SA	5	4	1	1.0002	33.3%
SA	10	6	1	1.0002	33.3%
SA	20	8	1	1.0002	33.3%
SA	50	10	1	1.0002	33.3%
SQA	1	1	1	1.0002	33.3%
SQA	5	4	1	1.0002	33.3%
SQA	10	6	1	1.0002	33.3%
SQA	20	8	1	1.0002	33.3%
SQA	50	10	1	1.0002	33.3%

The trial-variation results show that additional trials increase the number of unique matrices, but do not change the downstream cardinality-estimation behaviour. For both SA and SQA, the number of unique matrices increases from 1 at $T = 1$ to 10 at $T = 50$. However, every configuration produces only one distinct cardinality-estimation vector. The median q-error remains 1.0002, and the exact-estimate rate remains 33.3% across all trial counts.

Overall, the $N = 10{,}000$ Market Basket experiment shows that the template-based item co-occurrence patterns are estimated almost exactly once the relation contains substantially more tuples. Structural diversity is still present, especially when additional trials are executed, but this diversity does not propagate into different downstream cardinality estimates.

6.9.7. DISCUSSION: MARKET BASKET WITH $N = 10{,}000$

The Market Basket experiment with $N = 10{,}000$ shows a much more stable cardinality-estimation behaviour than the corresponding $N = 100$ experiment. Both SA and SQA produce the same stable cardinality pattern, and this pattern is very close to the true cardinalities for all six workload queries. The stable annealing estimates have a median q-error of 1.0002 and a maximum q-error of only 1.0007. This means that the remaining deviations are very small in multiplicative terms. The result is therefore relevant for **Research Question 3**:

on this workload, the annealing-based estimators provide the strongest worst-case accuracy among the competitor estimators.

The comparison with PostgreSQL and the classical Chow–Liu baseline shows that the larger relation size does not remove the need to model dependencies. PostgreSQL estimates the single-item query Q1 exactly, but it still produces large errors for selective conjunctive predicates, especially Q4 and Q6. The classical Chow–Liu baseline performs very well on several queries and estimates four of the six predicates exactly. However, it strongly underestimates Q6, the most selective query involving all six item variables. This suggests that its tree restriction is sufficient for many pairwise co-occurrence patterns, but not for the full multi-item dependency represented by Q6.

The annealing-learned structures behave differently. They do not estimate every query exactly, but they avoid large errors across the full workload. This is important because cardinality estimation is often most critical for selective multi-predicate queries, where large underestimates or overestimates can have a strong effect on query optimisation. In this experiment, the advantage of the annealing-based estimators is therefore not the number of exact estimates, but their consistently low worst-case q-error.

The read–trial analysis addresses **Research Question 2**. Increasing the read budget mainly reduces structural diversity, but it does not materially change the downstream cardinality estimates. For SA, all read-variation configurations already produce only one distinct cardinality-estimation behaviour, even when several unique matrices are found. For SQA, the only exception occurs at $R = 100, T = 20$, where one of the 17 unique matrices returns the exact cardinality vector. From $R = 500$ onward, SQA also returns only the stable cardinality pattern. Thus, solver stochasticity still produces some graph diversity, but this diversity almost never propagates into different cardinality estimates.

The trial-variation results confirm the same interpretation. For both SA and SQA, increasing the number of trials from $T = 1$ to $T = 50$ increases the number of unique matrices from 1 to 10. However, every trial-variation configuration produces only one distinct cardinality-estimation vector. This separates structural diversity from estimation instability: additional trials reveal more learned structures, but these structures are equivalent for the selected cardinality-estimation workload.

Overall, the $N = 10{,}000$ Market Basket experiment strengthens the conclusion drawn from the smaller relation. With more tuples, the template-based co-occurrence patterns are estimated more reliably, and the effect of sparse frequency counts is reduced. In this setting, AnnealBN-CE produces highly accurate and stable cardinality estimates. Its main advantage appears in the reduction of worst-case q-error for selective multi-item predicates, while the

read–trial results show that the downstream estimates remain stable despite the presence of multiple learned structures.

With this final Market Basket experiment, the empirical evaluation is complete. The following chapter summarises the main findings across all datasets, relates them back to the research questions, and outlines limitations and directions for future work.

7. Conclusions and Future Work

7.1. SUMMARY

This thesis addressed single-relation cardinality estimation for selection queries with correlated attributes. Traditional database estimators can become inaccurate when they combine predicate selectivities under independence assumptions. Bayesian networks offer a dependency-aware alternative by modelling the joint distribution over attributes, provided that the learned structure captures the dependencies relevant to the query workload.

AnnealBN-CE was proposed as an end-to-end quantum-inspired annealing-based framework that bridges the $\mathcal{NP}$-complete problem of Bayesian-network structure learning, expressed as a quadratic unconstrained binary optimisation (QUBO) formulation, and the challenging task of query-level cardinality estimation. The framework maps a tabular relation to an aligned integer-encoded solver representation, learns candidate structures with simulated annealing (SA) and simulated quantum annealing (SQA), and uses the resulting Bayesian networks to estimate selection-query cardinalities through probabilistic inference.

The empirical evaluation compared AnnealBN-CE with two reference estimators: PostgreSQL as the database-system baseline and Chow–Liu as the classical Bayesian-network baseline. PostgreSQL represents the behaviour of a traditional optimiser estimator based on stored planner statistics, while Chow–Liu provides a tractable tree-structured dependency model. The comparison was conducted on controlled synthetic Bayesian-network data, real health-survey data, and synthetic binary transaction data, using the same query workloads, true cardinalities, and q-error metric for all estimators. Beyond accuracy, the evaluation examined how different annealing configurations affect learned structure diversity and downstream cardinality-estimation stability.

The results demonstrate that annealing-based Bayesian-network structure learning can be integrated into a complete cardinality-estimation workflow. They also show that solver stochasticity must be interpreted at the level of downstream estimates rather than graph structure alone. Across several experiments, different learned adjacency matrices produced identical or nearly identical car-

dinality estimates for the evaluated workload, indicating that structural diversity does not necessarily imply cardinality-estimation instability.

Overall, the findings indicate that AnnealBN-CE is a promising direction for dependency-aware cardinality estimation. Across the evaluated settings, annealing-learned Bayesian networks can produce accurate and stable estimates when the learned structures capture dependencies relevant to the query predicates. This positions AnnealBN-CE as an exploratory prototype and as a blueprint for future correlation-aware cardinality estimators.

7.2. CONCLUSIONS

The first research question examined the integration of QUBO-based Bayesian-network structure learning into an end-to-end cardinality-estimation workflow for single-relation selection queries. This integration was shown to be feasible. AnnealBN-CE bridges the solver-side quadratic unconstrained binary optimisation formulation and the database-side estimation task by decoding learned structures into Bayesian-network topologies, fitting them on the tabular relation, and using probabilistic inference to estimate selection-query cardinalities. This yields a reproducible experimental workflow from structure learning to query-level cardinality estimation.

The second research question focused on how simulated annealing and simulated quantum annealing behave under varying read and trial budgets with respect to learned graph diversity, downstream cardinality-estimation stability, and query-level accuracy. The experiments show that this behaviour must be interpreted at two levels. SA and SQA can return several distinct adjacency matrices, especially when the read–trial configuration changes. However, different structures do not necessarily lead to different cardinality estimates. In several configurations, different learned structures produced identical or nearly identical estimates for the evaluated workload. This shows that the relevant question is not only whether the solver returns one specific graph, but whether the learned structure captures the dependencies that matter for the query predicates.

The third research question addressed the conditions under which annealing-learned Bayesian networks improve cardinality-estimation accuracy compared with PostgreSQL and the classical Chow–Liu Bayesian-network baseline. The answer is conditional. AnnealBN-CE is most effective when the learned structure captures dependencies that are relevant to the evaluated workload, especially for selective multi-attribute predicates where independence assumptions or tree-structured models can be insufficient. At the same time, PostgreSQL and the Chow–Liu baseline remain competitive when the queried dependencies are simple, mostly marginal, or well approximated by a tree. The results therefore do not show that annealing-learned Bayesian networks should replace exist-

ing estimators. Instead, they show that such structures can be useful on the evaluated dependency-sensitive workloads.

A further conclusion is that graph reconstruction alone is not sufficient for evaluating annealing-based Bayesian-network structure learning in this setting. For cardinality estimation, the learned graph must be judged by its effect on the resulting query estimates. A structure that differs from a reference or generating graph may still capture enough relevant dependency information to produce accurate estimates for a given workload. Conversely, a structurally plausible graph is not automatically useful if it does not improve the resulting cardinalities. This workload-oriented view is important when structure learning is used inside a cardinality-estimation pipeline.

Modern database settings increasingly involve complex data, wider relations, and attributes whose relationships cannot always be captured by simple independence assumptions or by one fixed tree-structured model. This is where the perspective introduced by AnnealBN-CE becomes most relevant. The framework treats cardinality estimation as a dependency-modelling problem and uses annealing-learned Bayesian networks to retain relationship information that simpler statistics may lose. Its stochastic behaviour is therefore not only a source of variability, but also a way to expose that the same data can often be represented by more than one useful structure. In this sense, AnnealBN-CE combines the expressive power of probabilistic modelling with the search capabilities of annealing-based optimisation. AnnealBN-CE is not a replacement for PostgreSQL statistics, but a step towards selective, correlation-aware estimation for cases where traditional assumptions are insufficient. It provides a blueprint for future estimators that model complex attribute relationships more explicitly and use this information to improve query-level cardinality estimates.

7.3. LIMITATIONS

Single-relation scope. A central limitation of AnnealBN-CE is its single-relation scope. The framework estimates selection predicates over one tabular relation and does not model joins or cross-relation dependencies. This keeps the prototype focused on the integration of annealing-based Bayesian-network structure learning with cardinality estimation, but excludes a major source of database-optimisation error: inaccurate estimates propagating through joins and intermediate results.

Scalability of the structure-learning formulation. The current structure-learning formulation has limited scalability. Its QUBO representation grows quickly with the number of variables, possible parent sets, and attribute states. Bayesian-network parameter tables also become larger when variables have

many states or when nodes have several parents. This limits the direct use of the framework for wider schemas, high-cardinality attributes, and larger dependency structures.

Numerical-attribute discretisation. The framework also depends on how numerical attributes are represented. The annealing solver requires a finite integer-encoded state space, so numerical attributes must be discretised before structure learning. This makes the problem tractable, but it can introduce information loss because the learned Bayesian network models the discretised tabular relation rather than the original continuous values. As a result, discretisation can affect which dependencies are visible to the structure-learning procedure and, consequently, the resulting cardinality estimates.

Absence of physical quantum-annealing hardware. The experiments are limited to simulated annealing and simulated quantum annealing. They therefore do not demonstrate hardware-level quantum advantage or show whether current quantum-annealing hardware would provide better solution quality, lower runtime, or a better cost–accuracy trade-off than classical solvers.

Fixed workload scope and PostgreSQL statistics state. The empirical evaluation provides insights for the fixed query workloads used in this thesis. Other predicate combinations may expose different dependency patterns and therefore lead to different accuracy or stability results. The PostgreSQL estimates were collected with `EXPLAIN (FORMAT JSON)` by extracting the planner's `Plan Rows` value under the statistics state available at evaluation time. They should therefore be interpreted as planner estimates from the current prototype environment, not as results from a fully controlled PostgreSQL evaluation protocol.

7.4. FUTURE WORK

A more ambitious direction is to use AnnealBN-CE as an optimiser-side disagreement detector. Instead of treating the framework only as a replacement estimator, a database system could first produce its standard cardinality estimate, while AnnealBN-CE is applied to correlation-sensitive predicate sets. If the Bayesian-network estimate differs substantially from the planner estimate, the query could be marked as dependency-sensitive before execution. The optimiser could then allocate a larger estimation budget, compare alternative plans under different cardinality assumptions, or fall back to a more conservative plan choice. This would turn AnnealBN-CE from a standalone prototype into a mechanism for detecting when conventional statistics may be misleading.

A second direction is to make the framework query-scoped rather than relation-wide. Instead of learning one Bayesian-network structure over all attributes of a relation, future work could learn smaller overlapping structures for attributes that occur together in query predicates. For join queries, these local structures could be connected through selected primary-key/foreign-key paths or other dependency-relevant attributes. This would address both the single-relation limitation and the scalability limitation of the current QUBO formulation, because the solver would operate on the query-relevant part of the schema rather than on one large global model.

The stochasticity of annealing solvers could also be used as an estimation signal. Future versions of AnnealBN-CE could retain several low-energy structures instead of selecting only one adjacency matrix. If these structures produce similar cardinality estimates, the estimate could be treated as stable and the solver could stop early. If they disagree, the system could increase the read or trial budget, report an uncertainty interval, or fall back to a conservative estimate. This would connect solver behaviour directly to the confidence of the cardinality estimate.

Further work is also needed on data representation. Numerical attributes are currently discretised before structure learning, which makes the solver input finite but may hide relevant dependency patterns. Future experiments should compare fixed discretisation with adaptive, data-driven, or query-aware binning strategies. The goal would be to preserve more information from continuous attributes while still producing a finite state space for QUBO construction, Bayesian-network fitting, and probabilistic inference.

Finally, future evaluation should test whether the proposed workflow remains useful under stricter and more realistic conditions. This includes testing the QUBO formulation on physical quantum-annealing hardware and comparing it with SA and SQA using the same downstream q-error evaluation. PostgreSQL estimates should also be collected under a controlled statistics protocol, with documented `ANALYZE` or `VACUUM ANALYZE` steps before extracting `Plan Rows` from `EXPLAIN (FORMAT JSON)`. Beyond q-error, future experiments should also include plan-aware metrics, so that improvements in cardinality estimation can be linked to their effect on query optimisation.

Bibliography

[1] D. M. Chickering. 1996. Learning Bayesian Networks is NP-Complete. In *Learning from Data: Artificial Intelligence and Statistics V*. Lecture Notes in Statistics. Vol. 112. D. Fisher and H.-J. Lenz, editors. Springer, New York, NY, 121–130. doi:10.1007/978-1-4612-2404-4_12 (cit. on p. 32).

[2] C. K. Chow and C. N. Liu. 1968. Approximating Discrete Probability Distributions with Dependence Trees. *IEEE Transactions on Information Theory*, 14, 3, 462–467. doi:10.1109/TIT.1968.1054142 (cit. on p. 32).

[3] D-Wave Systems Inc. 2026. D-Wave Documentation: Minor Embedding. Accessed: 2026-05-12. D-Wave Systems Inc. https://docs.dwavequantum.com/en/latest/quantum_research/embedding_intro.html (cit. on p. 28).

[4] D-Wave Systems Inc. 2026. D-Wave Documentation: Topologies. Accessed: 2026-05-12. D-Wave Systems Inc. https://docs.dwavequantum.com/en/latest/quantum_research/topologies.html (cit. on p. 28).

[5] A. Doan, A. Y. Halevy and Z. G. Ives. 2012. *Principles of Data Integration*. (1st ed.). Morgan Kaufmann, Waltham, MA. 520 pp. ISBN: 978-0124160446 (cit. on pp. 7, 11, 52).

[6] L. Getoor, B. Taskar and D. Koller. 2001. Selectivity Estimation using Probabilistic Models. In *Proceedings of the 2001 ACM SIGMOD International Conference on Management of Data*, 461–472. doi:10.1145/375663.375727 (cit. on p. 31).

[7] F. Glover, G. Kochenberger and Y. Du. 2019. Quantum Bridge Analytics I: A Tutorial on Formulating and Using QUBO Models. *4OR*, 17, 335–371. doi:10.1007/s10288-019-00424-y (cit. on pp. 21, 22, 26, 28).

[8] M. Halford. 2020. tldks-2020: Implementation of Selectivity Estimation with Linked Bayesian Networks. https://github.com/MaxHalford/tldks-2020. Original-date: 2020-12-16, commit hash e8d1d7d, Apache License 2.0. (2020). Retrieved 21st May 2026 from (cit. on p. 46).

[9] M. Halford, P. Saint-Pierre and F. Morvan. 2019. An Approach Based on Bayesian Networks for Query Selectivity Estimation. In *Database Systems for Advanced Applications (DASFAA 2019)* (Lecture Notes in Computer Science). G. Li, J. Yang, J. Gama, J. Natwichai and Y. Tong, editors. Vol. 11447. hal-02493882. Springer, Chiang Mai, Thailand, 3–19 (cit. on pp. 6–8, 10, 13–18, 31, 32).

[10] M. Halford, P. Saint-Pierre and F. Morvan. 2020. Selectivity Estimation with Attribute Value Dependencies Using Linked Bayesian Networks. In *Transactions on Large-Scale Data- and Knowledge-Centered Systems XLVI*. Lecture Notes in Computer Science. Vol. 12410. A. Hameurlain and A. M. Tjoa, editors. Springer, Berlin, Heidelberg, 154–188. doi:10.1007/978-3-662-62386-2_6 (cit. on pp. 1, 3, 6–12, 32).

[11] Y. Han et al. 2022. Cardinality Estimation in DBMS: A Comprehensive Benchmark Evaluation. *Proceedings of the VLDB Endowment*, 15, 4, 752–765. doi:10.14778/3503585.3503586 (cit. on pp. 9, 31).

[12] B. Hilprecht, A. Schmidt, M. Kulessa, A. Molina, K. Kersting and C. Binnig. 2020. DeepDB: Learn from Data, not from Queries! *Proceedings of the VLDB Endowment*, 13, 7, 992–1005. doi:10.14778/3384345.3384349 (cit. on p. 31).

[13] Y. E. Ioannidis and S. Christodoulakis. 1991. On the Propagation of Errors in the Size of Join Results. In *Proceedings of the 1991 ACM SIGMOD International Conference on Management of Data* (Denver, CO, USA). ACM, 268–277. doi:10.1145/115790.115835 (cit. on p. 30).

[14] F. V. Jensen and T. D. Nielsen. 2007. *Bayesian Networks and Decision Graphs*. (2nd ed.). *Information Science and Statistics*. Springer, New York. ISBN: 978-0-387-68281-5. doi:10.1007/978-0-387-68282-2 (cit. on pp. 3, 8, 10–13, 15–21).

[15] A. D. King et al. 2021. Scaling Advantage over Path-Integral Monte Carlo in Quantum Simulation of Geometrically Frustrated Magnets. *Nature Communications*, 12, 1, 1113. doi:10.1038/s41467-021-20901-5 (cit. on p. 28).

[16] N. K. Kitson, A. C. Constantinou, Z. Guo, Y. Liu and K. Chobtham. 2023. A survey of Bayesian Network structure learning. *Artificial Intelligence Review*, 56, 8721–8814. doi:10.1007/s10462-022-10351-w (cit. on pp. 3, 18, 19, 21, 32).

[17] Y. Koshka and M. A. Novotny. 2020. Comparison of D-Wave Quantum Annealing and Classical Simulated Annealing for Local Minima Determination. *IEEE Journal on Selected Areas in Information Theory*, 1, 2, 515–525. doi:10.1109/JSAIT.2020.3014192 (cit. on pp. 26, 27).

[18] H. Lan, Z. Bao and Y. Peng. 2021. A Survey on Advancing the DBMS Query Optimizer: Cardinality Estimation, Cost Model, and Plan Enumeration. *Data Science and Engineering*, 6, 86–101. doi:10.1007/s41019-020-00149-7 (cit. on pp. 1, 3, 9, 30).

[19] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper and T. Neumann. 2015. How Good Are Query Optimizers, Really? *Proceedings of the VLDB Endowment*, 9, 3, 204–215. doi:10.14778/2850583.2850594 (cit. on p. 30).

[20] V. Markl, N. Megiddo, M. Kutsch, T. M. Tran, P. J. Haas and U. Srivastava. 2005. Consistently Estimating the Selectivity of Conjuncts of Predicates. In *Proceedings of the 31st International Conference on Very Large Data Bases* (VLDB 2005). VLDB Endowment, 373–384. https://dl.acm.org/doi/10.5555/1083592.1083638 (cit. on pp. 1, 30).

[21] NA, N. 2019. National Health and Nutrition Health Survey 2013–2014 (NHANES) Age Prediction Subset. UCI Machine Learning Repository. Accessed: 2026-05-13. (2019). doi:10.24432/C5BS66 (cit. on p. 61).

[22] B. O'Gorman, R. Babbush, A. Perdomo-Ortiz, A. Aspuru-Guzik and V. Smelyanskiy. 2015. Bayesian network structure learning using quantum annealing. *The European Physical Journal Special Topics*, 224, 1, 163–188. doi:10.1140/epjst/e2015-02349-9 (cit. on pp. 3, 21–27, 29, 33).

[23] G. Piatetsky-Shapiro and C. Connell. 1984. Accurate Estimation of the Number of Tuples Satisfying a Condition. In *Proceedings of the 1984 ACM SIGMOD International Conference on Management of Data*. ACM, 256–276. doi:10.1145/602259.602294 (cit. on p. 30).

[24] V. Poosala and Y. E. Ioannidis. 1997. Selectivity Estimation Without the Attribute Value Independence Assumption. In *Proceedings of the 23rd International Conference on Very Large Data Bases* (VLDB 1997). Morgan Kaufmann, 486–495. https://www.vldb.org/conf/1997/P486.PDF (cit. on p. 30).

[25] D. Repas, Z. Luo, M. Schoemans and M. Sakr. 2023. Selectivity Estimation of Inequality Joins in Databases. *Mathematics*, 11, 6, 1383. doi:10.3390/math11061383 (cit. on pp. 1–3, 7, 9, 10).

[26] M. Rizzoli. [n. d.] BNSL-QA-python: Bayesian Network Structure Learning with Quantum Annealing (2022). https://github.com/massimo-rizzoli/BNSL-QA-python. Original-date: 2022-09-12, commit hash 66f3ae1, GNU General Public License v2.0. (). Retrieved 21st May 2026 from (cit. on p. 46).

[27] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie and T. G. Price. 1979. Access Path Selection in a Relational Database Management System. In *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data*. ACM, 23–34. doi:10.1145/582095.582099 (cit. on p. 30).

[28] P.-N. Tan, M. Steinbach and V. Kumar. 2006. *Introduction to Data Mining*. Pearson, Boston, MA. ISBN: 978-0-321-32136-7 (cit. on pp. 64, 87).

[29] The PostgreSQL Global Development Group. 2026. *PostgreSQL 18 Documentation, Chapter 14.2: Statistics Used by the Planner*. Accessed: 2026-05-12. PostgreSQL Global Development Group. https://www.postgresql.org/docs/current/planner-stats.html (cit. on pp. 10, 30).

[30] The PostgreSQL Global Development Group. 2026. PostgreSQL 18 Documentation, Chapter 69.1: Row Estimation Examples. Accessed: 2026-03-31. PostgreSQL Global Development Group. https://www.postgresql.org/docs/current/row-estimation-examples.html (cit. on pp. 2, 4, 30).

[31] K. Tzoumas, A. Deshpande and C. S. Jensen. 2011. Lightweight graphical models for selectivity estimation without independence assumptions. *Proceedings of the VLDB Endowment*, 4, 11, 852–863. doi:10.14778/3402707.3402724 (cit. on p. 31).

[32] X. Wang, C. Qu, W. Wu, J. Wang and Q. Zhou. 2021. Are We Ready For Learned Cardinality Estimation? *Proceedings of the VLDB Endowment*, 14, 9, 1640–1654. doi:10.14778/3461535.3461552 (cit. on p. 31).

[33] World Health Organization. [n. d.] Moderate and severe thinness, underweight, overweight. https://apps.who.int/nutrition/landscape/help.aspx?helpid=420&menu=0. Accessed: 2026-05-27. () (cit. on p. 61).

[34] Z. Wu, A. Shaikhha, R. Zhu, K. Zeng, Y. Han and J. Zhou. 2021. BayesCard: Revitalizing Bayesian Networks for Cardinality Estimation. *arXiv preprint arXiv:2012.14743*. arXiv: 2012.14743 [cs.DB] (cit. on p. 32).

[35] Z. Yang et al. 2019. Deep Unsupervised Cardinality Estimation. *Proceedings of the VLDB Endowment*, 13, 3, 279–292. doi:10.14778/3368289.3368294 (cit. on p. 31).

[36] E. Zardini, M. Rizzoli, S. Dissegna, E. Blanzieri and D. Pastorello. 2022. Reconstructing Bayesian networks on a quantum annealer. *Quantum Information and Computation*, 22, 15-16, 1320–1350. doi:10.26421/qic22.15-16-4 (cit. on pp. 3, 19–29, 33, 50, 53, 63).

A. Supplementary Experimental Details

A.1. WETGRASS JSON SPECIFICATION AND SOLVER ENCODING

The WetGrass dataset is generated from the JSON specification shown in Listing A.1. The specification defines the variables, their states, parent sets, conditional probability tables, the reference adjacency matrix, and the topological order used during generation. For each binary variable, the conditional probability table stores the probability of the first listed state. The probability of the second state is given by the complement.

Listing A.1.: JSON specification used to generate the WetGrass dataset.

```
{
  "name": "WetGrass",
  "variables": {
    "Cloud": {
      "states": ["t", "f"],
      "parents": [],
      "cpt": [0.30]
    },
    "Sprinkler": {
      "states": ["on", "off"],
      "parents": ["Cloud"],
      "cpt": [[0.20], [0.80]]
    },
    "Rain": {
      "states": ["t", "f"],
      "parents": ["Cloud"],
      "cpt": [[0.60], [0.30]]
    },
    "WetGrass": {
      "states": ["t", "f"],
      "parents": ["Sprinkler",
      ↪  "Rain"],
      "cpt": [
        [[1.00], [1.00]],
        [[1.00], [0.10]]
      ]
    }
  },
  "solution": [
    [0, 1, 1, 0],
    [0, 0, 0, 1],
    [0, 0, 0, 1],
    [0, 0, 0, 0]
  ],
  "toporder": [
    "Cloud",
    "Sprinkler",
    "Rain",
    "WetGrass"
  ]
}
```

The same generated data is also written to the integer-encoded TXT format used by the `bnslqa` solver. The first line contains the number of variables and the number of states per variable. The second line gives the problem name. The third

line contains the adjacency vector required by the solver format. The following lines contain the encoded examples. An excerpt is shown in Listing A.2.

Listing A.2.: Excerpt of the WetGrass solver TXT representation.

```
4 2 2 2 2                       0 1 0 0
WetGrass                        0 1 0 0
1 1 0 0 0 1 0 0 1 0 0 0         0 1 0 0
0 0 0 0                         0 1 0 0
0 0 0 0                         0 1 0 0
0 0 0 0                         0 1 1 0
0 0 0 0                         0 1 1 1
0 0 1 0                         0 1 1 1
0 0 1 0                         0 1 1 1
0 1 0 0
```

A.2. SYNTHETIC MARKET BASKET GENERATION

The Synthetic Market Basket Transactions dataset is generated from the five transaction templates shown in Table A.1. The templates follow the standard market-basket example from *Introduction to Data Mining*, where transactions are represented as binary item vectors [28]. A value of 1 means that the item is present in the transaction, while a value of 0 means that it is absent.

Table A.1.: Base transaction template used to generate the Synthetic Market Basket dataset.

Template	Beer	Bread	Cola	Diapers	Eggs	Milk
T_1	0	1	0	0	0	1
T_2	1	1	0	1	1	0
T_3	1	0	1	1	0	1
T_4	1	1	0	1	0	1
T_5	0	1	1	1	0	1

To generate a dataset of size N, rows are sampled with replacement from the template set $\{T_1, T_2, T_3, T_4, T_5\}$. In this thesis, the generated dataset sizes are $N = 100$ and $N = 10{,}000$. The generated rows are written both as a CSV relation for cardinality estimation and as an integer-encoded TXT file for the `bnslqa` solver. Since the variables are already binary, the solver representation uses the same values 0 and 1.

B. Experimental Query Workloads

This appendix lists the fixed query workloads used in the experimental evaluation. Each workload consists of six single-relation selection queries. Only the SQL-style selection predicates are shown here; the corresponding Bayesian-network assignments follow from the dataset preparation described in the experiment setup sections.

B.1. WETGRASS QUERY WORKLOAD

Table B.1.: WetGrass query workload.

Query	SQL-style selection predicate
Q1	`wetgrass = 'f'`
Q2	`cloud = 't' AND wetgrass = 't'`
Q3	`rain = 't' AND wetgrass = 't'`
Q4	`sprinkler = 'on' AND rain = 'f'`
Q5	`cloud = 't' AND sprinkler = 'on' AND rain = 'f'`
Q6	`cloud = 't' AND sprinkler = 'off' AND rain = 'f' AND wetgrass = 'f'`

B.2. NHANES QUERY WORKLOAD

Table B.2.: NHANES query workload.

Query	SQL-style selection predicate
Q1	`DIQ010 = 2`
Q2	`BMXBMI >= 30.0 AND DIQ010 = 1`
Q3	`age_group = 'Senior' AND BMXBMI >= 30.0 AND DIQ010 = 1`
Q4	`age_group = 'Adult' AND BMXBMI >= 18.5 AND BMXBMI < 25.0 AND DIQ010 = 2`
Q5	`age_group = 'Adult' AND BMXBMI >= 25.0 AND BMXBMI < 30.0 AND DIQ010 = 1`
Q6	`age_group = 'Senior' AND RIAGENDR = 1 AND BMXBMI >= 25.0 AND BMXBMI < 30.0 AND DIQ010 = 2`

B.3. MARKET BASKET QUERY WORKLOAD

Table B.3.: Market Basket query workload.

Query	SQL-style selection predicate
Q1	`Bread = 1`
Q2	`Beer = 1 AND Diapers = 1`
Q3	`Milk = 1 AND Diapers = 1`
Q4	`Eggs = 1 AND Milk = 0`
Q5	`Bread = 1 AND Diapers = 1 AND Milk = 1`
Q6	`Beer = 1 AND Bread = 0 AND Cola = 1 AND Diapers = 1 AND Eggs = 0 AND Milk = 1`

B.4. ADDITIONAL WETGRASS ROBUSTNESS TABLES

This appendix reports the complete per-query cardinality estimates obtained for additional WetGrass read–trial configurations. The tables show the distinct cardinality values returned by the unique learned structures for the selected configurations. Cardinality values are rounded to at most two decimal places. The # distinct column gives the number of different cardinality estimates returned for each query and configuration. Values in parentheses indicate how many unique matrices returned the preceding cardinality estimate.

B.5. WETGRASS WITH 100 TUPLES

Table B.4.: Distinct SA cardinality estimates for the WetGrass query workload with $N = 100$, fixed $T = 30$, and varying read counts.

Reads	Unique matrices	Query	True	# distinct	Distinct SA cardinalities returned
10	8	Q1	18	3	18 (3); 17.46 (3); 20.86 (2)
10	8	Q2	21	3	18.63 (3); 21.36 (3); 16.82 (2)
10	8	Q3	39	1	39 (8)
10	8	Q4	41	3	41 (3); 41.6 (3); 37.82 (2)
10	8	Q5	2	3	3.97 (3); 2.4 (3); 3.66 (2)
10	8	Q6	9	3	11.37 (3); 8.64 (3); 13.18 (2)
50	3	Q1	18	1	17.46 (3)
50	3	Q2	21	1	21.36 (3)
50	3	Q3	39	1	39 (3)
50	3	Q4	41	1	41.6 (3)
50	3	Q5	2	1	2.4 (3)
50	3	Q6	9	1	8.64 (3)
100	2	Q1	18	1	17.46 (2)
100	2	Q2	21	1	21.36 (2)

Continued on next page

Table B.4.: Per-query distinct SA cardinality estimates for the WetGrass query workload with 100 tuples, fixed $T = 30$, and varying read counts. (continued)

Reads	Unique matrices	Query	True	# distinct	Distinct SA cardinalities returned
100	2	Q3	39	1	39 (2)
100	2	Q4	41	1	41.6 (2)
100	2	Q5	2	1	2.4 (2)
100	2	Q6	9	1	8.64 (2)
250	2	Q1	18	1	17.46 (2)
250	2	Q2	21	1	21.36 (2)
250	2	Q3	39	1	39 (2)
250	2	Q4	41	1	41.6 (2)
250	2	Q5	2	1	2.4 (2)
250	2	Q6	9	1	8.64 (2)
500	1	Q1	18	1	17.46 (1)
500	1	Q2	21	1	21.36 (1)
500	1	Q3	39	1	39 (1)
500	1	Q4	41	1	41.6 (1)
500	1	Q5	2	1	2.4 (1)
500	1	Q6	9	1	8.64 (1)

Table B.5.: Distinct SA cardinality estimates for the WetGrass query workload with $N = 100$, fixed $R = 100$, and varying numbers of trials.

Trials	Unique matrices	Query	True	# distinct	Distinct SA cardinalities returned
5	1	Q1	18	1	17.46 (1)
5	1	Q2	21	1	21.36 (1)
5	1	Q3	39	1	39 (1)
5	1	Q4	41	1	41.6 (1)
5	1	Q5	2	1	2.4 (1)
5	1	Q6	9	1	8.64 (1)
10	2	Q1	18	1	17.46 (2)
10	2	Q2	21	1	21.36 (2)
10	2	Q3	39	1	39 (2)
10	2	Q4	41	1	41.6 (2)
10	2	Q5	2	1	2.4 (2)
10	2	Q6	9	1	8.64 (2)
20	2	Q1	18	1	17.46 (2)
20	2	Q2	21	1	21.36 (2)
20	2	Q3	39	1	39 (2)
20	2	Q4	41	1	41.6 (2)
20	2	Q5	2	1	2.4 (2)
20	2	Q6	9	1	8.64 (2)
50	2	Q1	18	1	17.46 (2)
50	2	Q2	21	1	21.36 (2)
50	2	Q3	39	1	39 (2)

Continued on next page

Table B.5.: Per-query distinct SA cardinality estimates for the WetGrass query workload with 100 tuples, fixed $R = 100$, and varying numbers of trials. (continued)

Trials	Unique matrices	Query	True	# distinct	Distinct SA cardinalities returned
50	2	Q4	41	1	41.6 (2)
50	2	Q5	2	1	2.4 (2)
50	2	Q6	9	1	8.64 (2)

Table B.6.: Distinct SQA cardinality estimates for the WetGrass query workload with $N = 100$, fixed $T = 30$, and varying read counts.

Reads	Unique matrices	Query	True	# distinct	Distinct SQA cardinalities returned
10	8	Q1	18	3	18 (3); 17.46 (3); 20.86 (2)
10	8	Q2	21	4	18.63 (2); 24.6 (1); 21.36 (3); 16.82 (2)
10	8	Q3	39	2	39 (7); 40.28 (1)
10	8	Q4	41	4	41 (2); 39.49 (1); 41.6 (3); 37.82 (2)
10	8	Q5	2	4	3.97 (2); 4.65 (1); 2.4 (3); 3.66 (2)
10	8	Q6	9	4	11.37 (2); 5.4 (1); 8.64 (3); 13.18 (2)
50	4	Q1	18	2	17.46 (3); 20.86 (1)
50	4	Q2	21	2	21.36 (3); 16.82 (1)
50	4	Q3	39	1	39 (4)
50	4	Q4	41	2	41.6 (3); 37.82 (1)
50	4	Q5	2	2	2.4 (3); 3.66 (1)
50	4	Q6	9	2	8.64 (3); 13.18 (1)
100	2	Q1	18	1	17.46 (2)
100	2	Q2	21	1	21.36 (2)
100	2	Q3	39	1	39 (2)
100	2	Q4	41	1	41.6 (2)
100	2	Q5	2	1	2.4 (2)
100	2	Q6	9	1	8.64 (2)
250	1	Q1	18	1	17.46 (1)
250	1	Q2	21	1	21.36 (1)
250	1	Q3	39	1	39 (1)
250	1	Q4	41	1	41.6 (1)
250	1	Q5	2	1	2.4 (1)
250	1	Q6	9	1	8.64 (1)
500	1	Q1	18	1	17.46 (1)
500	1	Q2	21	1	21.36 (1)
500	1	Q3	39	1	39 (1)
500	1	Q4	41	1	41.6 (1)
500	1	Q5	2	1	2.4 (1)
500	1	Q6	9	1	8.64 (1)

Table B.7.: Distinct SQA cardinality estimates for the WetGrass query workload with $N = 100$, fixed $R = 100$, and varying numbers of trials.

Trials	Unique matrices	Query	True	# distinct	Distinct SQA cardinalities returned
5	1	Q1	18	1	17.46 (1)
5	1	Q2	21	1	21.36 (1)
5	1	Q3	39	1	39 (1)
5	1	Q4	41	1	41.6 (1)
5	1	Q5	2	1	2.4 (1)
5	1	Q6	9	1	8.64 (1)
10	1	Q1	18	1	17.46 (1)
10	1	Q2	21	1	21.36 (1)
10	1	Q3	39	1	39 (1)
10	1	Q4	41	1	41.6 (1)
10	1	Q5	2	1	2.4 (1)
10	1	Q6	9	1	8.64 (1)
20	2	Q1	18	1	17.46 (2)
20	2	Q2	21	1	21.36 (2)
20	2	Q3	39	1	39 (2)
20	2	Q4	41	1	41.6 (2)
20	2	Q5	2	1	2.4 (2)
20	2	Q6	9	1	8.64 (2)
50	2	Q1	18	1	17.46 (2)
50	2	Q2	21	1	21.36 (2)
50	2	Q3	39	1	39 (2)
50	2	Q4	41	1	41.6 (2)
50	2	Q5	2	1	2.4 (2)
50	2	Q6	9	1	8.64 (2)

B.6. WETGRASS WITH 10000 TUPLES

Table B.8.: Distinct SA cardinality estimates for the WetGrass query workload with $N = 10{,}000$, fixed $T = 20$, and varying read counts.

Reads	Unique matrices	Query	True	# distinct	Distinct SA cardinalities returned
100	2	Q1	1800	1	1746 (2)
100	2	Q2	2100	1	2136 (2)
100	2	Q3	3900	1	3900 (2)
100	2	Q4	4100	1	4160 (2)
100	2	Q5	200	1	240 (2)
100	2	Q6	900	1	864 (2)
500	1	Q1	1800	1	1746 (1)
500	1	Q2	2100	1	2136 (1)
500	1	Q3	3900	1	3900 (1)
500	1	Q4	4100	1	4160 (1)
500	1	Q5	200	1	240 (1)

Continued on next page

Table B.8.: Per-query distinct SA cardinality estimates for the WetGrass query workload with 10000 tuples, fixed $T = 20$, and varying read counts. (continued)

Reads	Unique matrices	Query	True	# distinct	Distinct SA cardinalities returned
500	1	Q6	900	1	864 (1)
1000	1	Q1	1800	1	1746 (1)
1000	1	Q2	2100	1	2136 (1)
1000	1	Q3	3900	1	3900 (1)
1000	1	Q4	4100	1	4160 (1)
1000	1	Q5	200	1	240 (1)
1000	1	Q6	900	1	864 (1)
5000	1	Q1	1800	1	1746 (1)
5000	1	Q2	2100	1	2136 (1)
5000	1	Q3	3900	1	3900 (1)
5000	1	Q4	4100	1	4160 (1)
5000	1	Q5	200	1	240 (1)
5000	1	Q6	900	1	864 (1)
10000	1	Q1	1800	1	1746 (1)
10000	1	Q2	2100	1	2136 (1)
10000	1	Q3	3900	1	3900 (1)
10000	1	Q4	4100	1	4160 (1)
10000	1	Q5	200	1	240 (1)
10000	1	Q6	900	1	864 (1)

Table B.9.: Per-query distinct SA cardinality estimates for the WetGrass query workload with 10000 tuples, fixed $R = 1000$, and varying numbers of trials.

Trials	Unique matrices	Query	True	# distinct	Distinct SA cardinalities returned
1	1	Q1	1800	1	1746 (1)
1	1	Q2	2100	1	2136 (1)
1	1	Q3	3900	1	3900 (1)
1	1	Q4	4100	1	4160 (1)
1	1	Q5	200	1	240 (1)
1	1	Q6	900	1	864 (1)
5	1	Q1	1800	1	1746 (1)
5	1	Q2	2100	1	2136 (1)
5	1	Q3	3900	1	3900 (1)
5	1	Q4	4100	1	4160 (1)
5	1	Q5	200	1	240 (1)
5	1	Q6	900	1	864 (1)
10	1	Q1	1800	1	1746 (1)
10	1	Q2	2100	1	2136 (1)
10	1	Q3	3900	1	3900 (1)
10	1	Q4	4100	1	4160 (1)

Continued on next page

Table B.9.: Per-query distinct SA cardinality estimates for the WetGrass query workload with 10000 tuples, fixed $R = 1000$, and varying numbers of trials. (continued)

Trials	Unique matrices	Query	True	# distinct	Distinct SA cardinalities returned
10	1	Q5	200	1	240 (1)
10	1	Q6	900	1	864 (1)
20	1	Q1	1800	1	1746 (1)
20	1	Q2	2100	1	2136 (1)
20	1	Q3	3900	1	3900 (1)
20	1	Q4	4100	1	4160 (1)
20	1	Q5	200	1	240 (1)
20	1	Q6	900	1	864 (1)
50	1	Q1	1800	1	1746 (1)
50	1	Q2	2100	1	2136 (1)
50	1	Q3	3900	1	3900 (1)
50	1	Q4	4100	1	4160 (1)
50	1	Q5	200	1	240 (1)
50	1	Q6	900	1	864 (1)

Table B.10.: Per-query distinct SQA cardinality estimates for the WetGrass query workload with 10000 tuples, fixed $T = 20$, and varying read counts.

Reads	Unique matrices	Query	True	# distinct	Distinct SQA cardinalities returned
100	2	Q1	1800	1	1746 (2)
100	2	Q2	2100	1	2136 (2)
100	2	Q3	3900	1	3900 (2)
100	2	Q4	4100	1	4160 (2)
100	2	Q5	200	1	240 (2)
100	2	Q6	900	1	864 (2)
500	1	Q1	1800	1	1746 (1)
500	1	Q2	2100	1	2136 (1)
500	1	Q3	3900	1	3900 (1)
500	1	Q4	4100	1	4160 (1)
500	1	Q5	200	1	240 (1)
500	1	Q6	900	1	864 (1)
1000	1	Q1	1800	1	1746 (1)
1000	1	Q2	2100	1	2136 (1)
1000	1	Q3	3900	1	3900 (1)
1000	1	Q4	4100	1	4160 (1)
1000	1	Q5	200	1	240 (1)
1000	1	Q6	900	1	864 (1)
5000	1	Q1	1800	1	1746 (1)
5000	1	Q2	2100	1	2136 (1)
5000	1	Q3	3900	1	3900 (1)
5000	1	Q4	4100	1	4160 (1)
5000	1	Q5	200	1	240 (1)

Continued on next page

Table B.10.: Per-query distinct SQA cardinality estimates for the WetGrass query workload with 10000 tuples, fixed $T = 20$, and varying read counts. (continued)

Reads	Unique matrices	Query	True	# distinct	Distinct SQA cardinalities returned
5000	1	Q6	900	1	864 (1)
10000	1	Q1	1800	1	1746 (1)
10000	1	Q2	2100	1	2136 (1)
10000	1	Q3	3900	1	3900 (1)
10000	1	Q4	4100	1	4160 (1)
10000	1	Q5	200	1	240 (1)
10000	1	Q6	900	1	864 (1)

Table B.11.: Per-query distinct SQA cardinality estimates for the WetGrass query workload with 10000 tuples, fixed $R = 1000$, and varying numbers of trials.

Trials	Unique matrices	Query	True	# distinct	Distinct SQA cardinalities returned
1	1	Q1	1800	1	1746 (1)
1	1	Q2	2100	1	2136 (1)
1	1	Q3	3900	1	3900 (1)
1	1	Q4	4100	1	4160 (1)
1	1	Q5	200	1	240 (1)
1	1	Q6	900	1	864 (1)
5	1	Q1	1800	1	1746 (1)
5	1	Q2	2100	1	2136 (1)
5	1	Q3	3900	1	3900 (1)
5	1	Q4	4100	1	4160 (1)
5	1	Q5	200	1	240 (1)
5	1	Q6	900	1	864 (1)
10	1	Q1	1800	1	1746 (1)
10	1	Q2	2100	1	2136 (1)
10	1	Q3	3900	1	3900 (1)
10	1	Q4	4100	1	4160 (1)
10	1	Q5	200	1	240 (1)
10	1	Q6	900	1	864 (1)
20	1	Q1	1800	1	1746 (1)
20	1	Q2	2100	1	2136 (1)
20	1	Q3	3900	1	3900 (1)
20	1	Q4	4100	1	4160 (1)
20	1	Q5	200	1	240 (1)
20	1	Q6	900	1	864 (1)
50	1	Q1	1800	1	1746 (1)
50	1	Q2	2100	1	2136 (1)
50	1	Q3	3900	1	3900 (1)
50	1	Q4	4100	1	4160 (1)

Continued on next page

Table B.11.: Per-query distinct SQA cardinality estimates for the WetGrass query workload with 10000 tuples, fixed $R = 1000$, and varying numbers of trials. (continued)

Trials	Unique matrices	Query	True	# distinct	Distinct SQA cardinalities returned
50	1	Q5	200	1	240 (1)
50	1	Q6	900	1	864 (1)

B.7. ADDITIONAL MARKET BASKET ROBUSTNESS TABLES

This appendix reports the complete per-query cardinality estimates obtained for additional Market Basket read–trial configurations. The tables show the distinct cardinality values returned by the unique learned structures for the selected configurations. Cardinality values are rounded to at most two decimal places. The # distinct column gives the number of different cardinality estimates returned for each query and configuration. Values in parentheses indicate how many unique matrices returned the preceding cardinality estimate.

B.8. MARKET BASKET WITH 100 TUPLES

Table B.12.: Detailed per-query distinct SA cardinality estimates for the Market Basket query workload with $N = 100$, fixed $T = 30$, and varying read counts. The value in parentheses gives the number of unique matrices that returned the corresponding cardinality estimate.

Reads	Unique matrices	Query	True	# distinct	Distinct SA cardinalities returned
10	27	Q1	84	3	84 (12); 81.39 (12); 77.95 (3)
10	27	Q2	63	1	63 (27)
10	27	Q3	61	4	61 (11); 66.82 (1); 58.39 (12); 57.65 (3)
10	27	Q4	21	2	21 (24); 18.3 (3)
10	27	Q5	45	4	45 (11); 50.82 (1); 39.78 (12); 35.6 (3)
10	27	Q6	16	3	16 (12); 18.61 (12); 22.05 (3)
50	20	Q1	84	2	84 (3); 81.39 (17)
50	20	Q2	63	1	63 (20)
50	20	Q3	61	2	61 (3); 58.39 (17)
50	20	Q4	21	1	21 (20)
50	20	Q5	45	2	45 (3); 39.78 (17)
50	20	Q6	16	2	16 (3); 18.61 (17)
100	17	Q1	84	2	84 (1); 81.39 (16)
100	17	Q2	63	1	63 (17)
100	17	Q3	61	2	61 (1); 58.39 (16)
100	17	Q4	21	1	21 (17)
100	17	Q5	45	2	45 (1); 39.78 (16)

Continued on next page

Table B.12.: Detailed per-query distinct SA cardinality estimates for the Market Basket query workload with $N = 100$, fixed $T = 30$, and varying read counts. The value in parentheses gives the number of unique matrices that returned the corresponding cardinality estimate (continued).

Reads	Unique matrices	Query	True	# distinct	Distinct SA cardinalities returned
100	17	Q6	16	2	16 (1); 18.61 (16)
250	11	Q1	84	1	81.39 (11)
250	11	Q2	63	1	63 (11)
250	11	Q3	61	1	58.39 (11)
250	11	Q4	21	1	21 (11)
250	11	Q5	45	1	39.78 (11)
250	11	Q6	16	1	18.61 (11)
500	12	Q1	84	1	81.39 (12)
500	12	Q2	63	1	63 (12)
500	12	Q3	61	1	58.39 (12)
500	12	Q4	21	1	21 (12)
500	12	Q5	45	1	39.78 (12)
500	12	Q6	16	1	18.61 (12)

Table B.13.: Detailed per-query distinct SQA cardinality estimates for the Market Basket query workload with $N = 100$, fixed $T = 30$, and varying read counts. The value in parentheses gives the number of unique matrices that returned the corresponding cardinality estimate.

Reads	Unique matrices	Query	True	# distinct	Distinct SQA cardinalities returned
10	30	Q1	84	5	84 (18); 81.39 (6); 82.06 (3); 77.95 (2); 83.73 (1)
10	30	Q2	63	3	63 (26); 61.82 (3); 66.42 (1)
10	30	Q3	61	7	61 (17); 58.39 (6); 64.78 (3); 64.44 (1); 58.19 (1); 62.72 (1); 66.82 (1)
10	30	Q4	21	5	21 (25); 17.12 (2); 25.39 (1); 19.58 (1); 18.09 (1)
10	30	Q5	45	8	45 (16); 39.78 (6); 47.79 (3); 42.39 (1); 45.07 (1); 40.67 (1); 50.82 (1); 50.33 (1)
10	30	Q6	16	6	16 (17); 18.61 (6); 16.99 (3); 22.05 (2); 13.12 (1); 10.67 (1)
50	27	Q1	84	2	81.39 (20); 84 (7)
50	27	Q2	63	1	63 (27)
50	27	Q3	61	2	58.39 (20); 61 (7)
50	27	Q4	21	1	21 (27)
50	27	Q5	45	2	39.78 (20); 45 (7)
50	27	Q6	16	2	18.61 (20); 16 (7)
100	18	Q1	84	1	81.39 (18)
100	18	Q2	63	1	63 (18)
100	18	Q3	61	1	58.39 (18)

Continued on next page

Table B.13.: Detailed per-query distinct SQA cardinality estimates for the Market Basket query workload with $N = 100$, fixed $T = 30$, and varying read counts. The value in parentheses gives the number of unique matrices that returned the corresponding cardinality estimate (continued).

Reads	Unique matrices	Query	True	# distinct	Distinct SQA cardinalities returned
100	18	Q4	21	1	21 (18)
100	18	Q5	45	1	39.78 (18)
100	18	Q6	16	1	18.61 (18)
250	12	Q1	84	1	81.39 (12)
250	12	Q2	63	1	63 (12)
250	12	Q3	61	1	58.39 (12)
250	12	Q4	21	1	21 (12)
250	12	Q5	45	1	39.78 (12)
250	12	Q6	16	1	18.61 (12)
500	10	Q1	84	1	81.39 (10)
500	10	Q2	63	1	63 (10)
500	10	Q3	61	1	58.39 (10)
500	10	Q4	21	1	21 (10)
500	10	Q5	45	1	39.78 (10)
500	10	Q6	16	1	18.61 (10)

Table B.14.: Detailed per-query distinct SA cardinality estimates for the Market Basket query workload with $N = 100$, fixed $R = 100$, and varying trial counts. The value in parentheses gives the number of unique matrices that returned the corresponding cardinality estimate.

Trials	Unique matrices	Query	True	# distinct	Distinct SA cardinalities returned
5	4	Q1	84	1	81.39 (4)
5	4	Q2	63	1	63 (4)
5	4	Q3	61	1	58.39 (4)
5	4	Q4	21	1	21 (4)
5	4	Q5	45	1	39.78 (4)
5	4	Q6	16	1	18.61 (4)
10	8	Q1	84	1	81.39 (8)
10	8	Q2	63	1	63 (8)
10	8	Q3	61	1	58.39 (8)
10	8	Q4	21	1	21 (8)
10	8	Q5	45	1	39.78 (8)
10	8	Q6	16	1	18.61 (8)
20	13	Q1	84	1	81.39 (13)
20	13	Q2	63	1	63 (13)
20	13	Q3	61	1	58.39 (13)
20	13	Q4	21	1	21 (13)
20	13	Q5	45	1	39.78 (13)

Continued on next page

Table B.14.: Detailed per-query distinct SA cardinality estimates for the Market Basket query workload with $N = 100$, fixed $R = 100$, and varying trial counts. The value in parentheses gives the number of unique matrices that returned the corresponding cardinality estimate (continued).

Trials	Unique matrices	Query	True	# distinct	Distinct SA cardinalities returned
20	13	Q6	16	1	18.61 (13)
50	22	Q1	84	2	81.39 (21); 84 (1)
50	22	Q2	63	1	63 (22)
50	22	Q3	61	2	58.39 (21); 61 (1)
50	22	Q4	21	1	21 (22)
50	22	Q5	45	2	39.78 (21); 45 (1)
50	22	Q6	16	2	18.61 (21); 16 (1)

Table B.15.: Detailed per-query distinct SQA cardinality estimates for the Market Basket query workload with $N = 100$, fixed $R = 100$, and varying trial counts. The value in parentheses gives the number of unique matrices that returned the corresponding cardinality estimate.

Trials	Unique matrices	Query	True	# distinct	Distinct SQA cardinalities returned
5	5	Q1	84	1	81.39 (5)
5	5	Q2	63	1	63 (5)
5	5	Q3	61	1	58.39 (5)
5	5	Q4	21	1	21 (5)
5	5	Q5	45	1	39.78 (5)
5	5	Q6	16	1	18.61 (5)
10	9	Q1	84	1	81.39 (9)
10	9	Q2	63	1	63 (9)
10	9	Q3	61	1	58.39 (9)
10	9	Q4	21	1	21 (9)
10	9	Q5	45	1	39.78 (9)
10	9	Q6	16	1	18.61 (9)
20	17	Q1	84	2	81.39 (16); 84 (1)
20	17	Q2	63	1	63 (17)
20	17	Q3	61	2	58.39 (16); 61 (1)
20	17	Q4	21	1	21 (17)
20	17	Q5	45	2	39.78 (16); 45 (1)
20	17	Q6	16	2	18.61 (16); 16 (1)
50	28	Q1	84	1	81.39 (28)
50	28	Q2	63	1	63 (28)
50	28	Q3	61	1	58.39 (28)
50	28	Q4	21	1	21 (28)
50	28	Q5	45	1	39.78 (28)
50	28	Q6	16	1	18.61 (28)

B.9. MARKET BASKET WITH 10000 TUPLES

Table B.16.: Per-query distinct SA cardinality estimates for the Market Basket query workload with 10000 tuples, fixed $T = 20$, and varying read counts.

Reads	Unique matrices	Query	True	# distinct	Distinct SA cardinalities returned
100	14	Q1	8,057	1	8,055.72 (14)
100	14	Q2	5,934	1	5,934 (14)
100	14	Q3	5,937	1	5,935.72 (14)
100	14	Q4	2,016	1	2,016 (14)
100	14	Q5	3,994	1	3,991.44 (14)
100	14	Q6	1,943	1	1,944.28 (14)
500	12	Q1	8,057	1	8,055.72 (12)
500	12	Q2	5,934	1	5,934 (12)
500	12	Q3	5,937	1	5,935.72 (12)
500	12	Q4	2,016	1	2,016 (12)
500	12	Q5	3,994	1	3,991.44 (12)
500	12	Q6	1,943	1	1,944.28 (12)
1000	8	Q1	8,057	1	8,055.72 (8)
1000	8	Q2	5,934	1	5,934 (8)
1000	8	Q3	5,937	1	5,935.72 (8)
1000	8	Q4	2,016	1	2,016 (8)
1000	8	Q5	3,994	1	3,991.44 (8)
1000	8	Q6	1,943	1	1,944.28 (8)
5000	2	Q1	8,057	1	8,055.72 (2)
5000	2	Q2	5,934	1	5,934 (2)
5000	2	Q3	5,937	1	5,935.72 (2)
5000	2	Q4	2,016	1	2,016 (2)
5000	2	Q5	3,994	1	3,991.44 (2)
5000	2	Q6	1,943	1	1,944.28 (2)
10000	1	Q1	8,057	1	8,055.72 (1)
10000	1	Q2	5,934	1	5,934 (1)
10000	1	Q3	5,937	1	5,935.72 (1)
10000	1	Q4	2,016	1	2,016 (1)
10000	1	Q5	3,994	1	3,991.44 (1)
10000	1	Q6	1,943	1	1,944.28 (1)

Table B.17.: Per-query distinct SQA cardinality estimates for the Market Basket query workload with $N = 10{,}000$, fixed $T = 20$, and varying read counts.

Reads	Unique matrices	Query	True	# distinct	Distinct SQA cardinalities returned
100	17	Q1	8,057	2	8,055.72 (16); 8,057 (1)
100	17	Q2	5,934	1	5,934 (17)
100	17	Q3	5,937	2	5,935.72 (16); 5,937 (1)

Continued on next page

Table B.17.: Per-query distinct SQA cardinality estimates for the Market Basket query workload with $N = 10{,}000$, fixed $T = 20$, and varying read counts (continued).

Reads	Unique matrices	Query	True	# distinct	Distinct SQA cardinalities returned
100	17	Q4	2,016	1	2,016 (17)
100	17	Q5	3,994	2	3,991.44 (16); 3,994 (1)
100	17	Q6	1,943	2	1,944.28 (16); 1,943 (1)
500	12	Q1	8,057	1	8,055.72 (12)
500	12	Q2	5,934	1	5,934 (12)
500	12	Q3	5,937	1	5,935.72 (12)
500	12	Q4	2,016	1	2,016 (12)
500	12	Q5	3,994	1	3,991.44 (12)
500	12	Q6	1,943	1	1,944.28 (12)
1000	8	Q1	8,057	1	8,055.72 (8)
1000	8	Q2	5,934	1	5,934 (8)
1000	8	Q3	5,937	1	5,935.72 (8)
1000	8	Q4	2,016	1	2,016 (8)
1000	8	Q5	3,994	1	3,991.44 (8)
1000	8	Q6	1,943	1	1,944.28 (8)
5000	4	Q1	8,057	1	8,055.72 (4)
5000	4	Q2	5,934	1	5,934 (4)
5000	4	Q3	5,937	1	5,935.72 (4)
5000	4	Q4	2,016	1	2,016 (4)
5000	4	Q5	3,994	1	3,991.44 (4)
5000	4	Q6	1,943	1	1,944.28 (4)
10000	3	Q1	8,057	1	8,055.72 (3)
10000	3	Q2	5,934	1	5,934 (3)
10000	3	Q3	5,937	1	5,935.72 (3)
10000	3	Q4	2,016	1	2,016 (3)
10000	3	Q5	3,994	1	3,991.44 (3)
10000	3	Q6	1,943	1	1,944.28 (3)

Table B.18.: Per-query distinct SA cardinality estimates for the Market Basket query workload with 10000 tuples, fixed $R = 1000$, and varying numbers of trials.

Trials	Unique matrices	Query	True	# distinct	Distinct SA cardinalities returned
1	1	Q1	8,057	1	8,055.72 (1)
1	1	Q2	5,934	1	5,934 (1)
1	1	Q3	5,937	1	5,935.72 (1)
1	1	Q4	2,016	1	2,016 (1)
1	1	Q5	3,994	1	3,991.44 (1)
1	1	Q6	1,943	1	1,944.28 (1)
5	4	Q1	8,057	1	8,055.72 (4)
5	4	Q2	5,934	1	5,934 (4)

Continued on next page

Table B.18.: Per-query distinct SA cardinality estimates for the Market Basket query workload with $N = 10{,}000$, fixed $R = 1000$, and varying numbers of trials (continued).

Trials	Unique matrices	Query	True	# distinct	Distinct SA cardinalities returned
5	4	Q3	5,937	1	5,935.72 (4)
5	4	Q4	2,016	1	2,016 (4)
5	4	Q5	3,994	1	3,991.44 (4)
5	4	Q6	1,943	1	1,944.28 (4)
10	6	Q1	8,057	1	8,055.72 (6)
10	6	Q2	5,934	1	5,934 (6)
10	6	Q3	5,937	1	5,935.72 (6)
10	6	Q4	2,016	1	2,016 (6)
10	6	Q5	3,994	1	3,991.44 (6)
10	6	Q6	1,943	1	1,944.28 (6)
20	8	Q1	8,057	1	8,055.72 (8)
20	8	Q2	5,934	1	5,934 (8)
20	8	Q3	5,937	1	5,935.72 (8)
20	8	Q4	2,016	1	2,016 (8)
20	8	Q5	3,994	1	3,991.44 (8)
20	8	Q6	1,943	1	1,944.28 (8)
50	10	Q1	8,057	1	8,055.72 (10)
50	10	Q2	5,934	1	5,934 (10)
50	10	Q3	5,937	1	5,935.72 (10)
50	10	Q4	2,016	1	2,016 (10)
50	10	Q5	3,994	1	3,991.44 (10)
50	10	Q6	1,943	1	1,944.28 (10)

Table B.19.: Per-query distinct SQA cardinality estimates for the Market Basket query workload with $N = 10{,}000$, fixed $R = 1000$, and varying numbers of trials.

Trials	Unique matrices	Query	True	# distinct	Distinct SQA cardinalities returned
1	1	Q1	8,057	1	8,055.72 (1)
1	1	Q2	5,934	1	5,934 (1)
1	1	Q3	5,937	1	5,935.72 (1)
1	1	Q4	2,016	1	2,016 (1)
1	1	Q5	3,994	1	3,991.44 (1)
1	1	Q6	1,943	1	1,944.28 (1)
5	4	Q1	8,057	1	8,055.72 (4)
5	4	Q2	5,934	1	5,934 (4)
5	4	Q3	5,937	1	5,935.72 (4)
5	4	Q4	2,016	1	2,016 (4)
5	4	Q5	3,994	1	3,991.44 (4)
5	4	Q6	1,943	1	1,944.28 (4)
10	6	Q1	8,057	1	8,055.72 (6)

Continued on next page

Table B.19.: Per-query distinct SQA cardinality estimates for the Market Basket query workload with $N = 10{,}000$, fixed $R = 1000$, and varying numbers of trials (continued).

Trials	Unique matrices	Query	True	# distinct	Distinct SQA cardinalities returned
10	6	Q2	5,934	1	5,934 (6)
10	6	Q3	5,937	1	5,935.72 (6)
10	6	Q4	2,016	1	2,016 (6)
10	6	Q5	3,994	1	3,991.44 (6)
10	6	Q6	1,943	1	1,944.28 (6)
20	8	Q1	8,057	1	8,055.72 (8)
20	8	Q2	5,934	1	5,934 (8)
20	8	Q3	5,937	1	5,935.72 (8)
20	8	Q4	2,016	1	2,016 (8)
20	8	Q5	3,994	1	3,991.44 (8)
20	8	Q6	1,943	1	1,944.28 (8)
50	10	Q1	8,057	1	8,055.72 (10)
50	10	Q2	5,934	1	5,934 (10)
50	10	Q3	5,937	1	5,935.72 (10)
50	10	Q4	2,016	1	2,016 (10)
50	10	Q5	3,994	1	3,991.44 (10)
50	10	Q6	1,943	1	1,944.28 (10)